

Data_Analysis with Python

Aprenderemos a usar las principales librerías para analizar datos de Python (Numoym oandas, scikit learn).

Module 1: Importing Data Sets

Understanding the data

Dataset – Used Automobiles (CSV)

```
3,?,alfa-romero,gas,std,two,convertible,rwd,front,88.60,168.80,64.10,48.80,2548,dohc,four,130,mpfi,3.47,2.68,9.00,111,5000,21,27,13495
3,?,alfa-romero,gas,std,two,convertible,rwd,front,88.60,168.80,64.10,48.80,2548,dohc,four,130,mpfi,3.47,2.68,9.00,111,5000,21,27,16500
1,?,alfa-romero,gas,std,two,hatchback,rwd,front,94.50,171.20,65.50,52.40,2823,ohcv,six,152,mpfi,2.68,3.47,9.00,154,5000,19,26,16500
2,164,audi,gas,std,four,sedan,fwd,front,99.80,176.60,66.20,54.30,2337,ohc,four,109,mpfi,3.19,3.40,10.00,102,5500,24,30,13950
2,164,audi,gas,std,four,sedan,4wd,front,99.40,176.60,66.40,54.30,2824,ohc,five,136,mpfi,3.19,3.40,8.00,115,5500,18,22,17450
2,?,audi,gas,std,two,sedan,fwd,front,99.80,177.30,66.30,53.10,2507,ohc,five,136,mpfi,3.19,3.40,8.50,110,5500,19,25,15250
1,158,audi,gas,std,four,sedan,fwd,front,105.80,192.70,71.40,55.70,2844,ohc,five,136,mpfi,3.19,3.40,8.50,110,5500,19,25,17710
1,?,audi,gas,std,four,wagon,fwd,front,105.80,192.70,71.40,55.70,2954,ohc,five,136,mpfi,3.19,3.40,8.50,110,5500,19,25,18920
1,158,audi,gas,turbo,four,sedan,fwd,front,105.80,192.70,71.40,55.90,3086,ohc,five,131,mpfi,3.13,3.40,8.30,140,5500,17,20,23875
0,?,audi,gas,turbo,two,hatchback,4wd,front,99.50,178.20,67.90,52.00,3053,ohc,five,131,mpfi,3.13,3.40,7.00,160,5500,16,22,?
2,192,bmw,gas,std,two,sedan,rwd,front,101.20,176.80,64.80,54.30,2395,ohc,four,108,mpfi,3.50,2.80,8.80,101,5800,23,29,16430
0,192,bmw,gas,std,four,sedan,rwd,front,101.20,176.80,64.80,54.30,2395,ohc,four,108,mpfi,3.50,2.80,8.80,101,5800,23,29,16925
0,188,bmw,gas,std,two,sedan,rwd,front,101.20,176.80,64.80,54.30,2710,ohc,six,164,mpfi,3.31,3.19,9.00,121,4250,21,28,20970
0,188,bmw,gas,std,four,sedan,rwd,front,101.20,176.80,64.80,54.30,2765,ohc,six,164,mpfi,3.31,3.19,9.00,121,4250,21,28,21105
1,?,bmw,gas,std,four,sedan,rwd,front,103.50,189.00,66.90,55.70,3055,ohc,six,164,mpfi,3.31,3.19,9.00,121,4250,20,25,24565
0,?,bmw,gas,std,four,sedan,rwd,front,103.50,189.00,66.90,55.70,3230,ohc,six,209,mpfi,3.62,3.39,8.00,182,5400,16,22,30760
0,?,bmw,gas,std,two,sedan,rwd,front,103.50,193.80,67.90,53.70,3380,ohc,six,209,mpfi,3.62,3.39,8.00,182,5400,16,22,41315
0,?,bmw,gas,std,four,sedan,rwd,front,110.00,197.00,70.90,56.30,3505,ohc,six,209,mpfi,3.62,3.39,8.00,182,5400,15,20,36880
2,121,chevrolet,gas,std,two,hatchback,fwd,front,88.40,141.10,60.30,53.20,1488,1,three,61,2bbl,2.91,3.03,9.50,48,5100,47,53,5151
```

<https://archive.ics.uci.edu/dataset/10/automobile>

<https://archive.ics.uci.edu/dataset/10/automobile>

But in this example, the first row it's just another row of data.

Each of the attributes in the dataset

No.	Attribute name	attribute range	No.	Attribute name	attribute range
1	symboling	-3, -2, -1, 0, 1, 2, 3.	14	curb-weight	continuous from 1488 to 4066.
2	normalized-losses	continuous from 65 to 256.	15	engine-type	dohc, dohcv, l, ohc, ohcf, ohcv, rotor.
3	make	audi, bmw, etc.	16	num-of-cylinders	eight, five, four, six, three, twelve, two.
4	fuel-type	diesel, gas.	17	engine-size	continuous from 61 to 326.
5	aspiration	std, turbo.	18	fuel-system	1bbl, 2bbl, 4bbl, idi, mfi, mpfi, spdi, spfi.
6	num-of-doors	four, two.	19	bore	continuous from 2.54 to 3.94.
7	body-style	hardtop, wagon, etc.	20	stroke	continuous from 2.07 to 4.17.
8	drive-wheels	4wd, fwd, rwd.	21	compression-ratio	continuous from 7 to 23.
9	engine-location	front, rear.	22	horsepower	continuous from 48 to 288.
10	wheel-base	continuous from 86.6 to 120.9.	23	peak-rpm	continuous from 4150 to 6600.
11	length	continuous from 141.1 to 208.1.	24	city-mpg	continuous from 13 to 49.
12	width	continuous from 60.3 to 72.3.	25	highway-mpg	continuous from 16 to 54.
13	height	continuous from 47.8 to 59.8.	26	price	continuous from 5118 to 45400.

So here's the documentation on what each of the 26 columns represent.

There are a lot of columns, and I'll just go through a few of the column names.

But you can also check out the link at the bottom of the slide to go through the descriptions yourself.

The first attribute,

symboling, corresponds to the insurance risk level of a car. Cars are initially assigned a risk factor symbol associated with their price. Then, if an automobile is more risky, this symbol is adjusted by moving it up the scale. A value of +3 indicates that the auto is risky, -3, that it's probably pretty safe.

normalized losses, is the relative average loss payment per insured vehicle year. This value is normalized for all autos within a particular size classification (two door small, station wagons, sports specialty, etc), and represents the average loss per car per year. The values range from 65 to 256.

The other attributes are easy to understand.

The 26th attribute is price. This is **our target value** or label. In other words, this means price is the value that we want to predict from the data set, and the predictors should be all the other variables listed, like symboling, normalized losses, make, and so on.

Thus, the goal of this project is to predict price in terms of other car features.

Just a quick note, this data set is actually from 1985, so the car prices for the models may seem a little low. But just bear in mind that the goal of this exercise is to learn how to analyze the data.

[MUSIC]

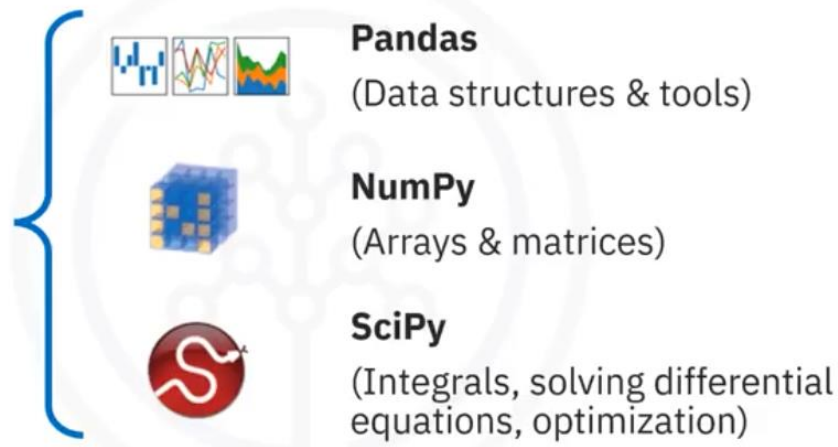
Python packages for data Science

A Python library is a **collection of functions and methods** that allow you to perform lots of actions without writing any code. The libraries usually contain built-in modules providing different functionalities which you can use directly.

We have divided the Python data analysis libraries into three groups.

Scientifics computing libraries in Python

1. Scientifics Computing Libraries



The first group is called Scientific Computing Libraries.

Pandas offers data structure and tools for effective data manipulation and analysis. It provides fast access to structured data. The primary instrument of Pandas is a **two dimensional table** consisting of column and row labels, which are called a **data frame**. It is designed to provide **easy indexing functionality**.

NumPy library uses **arrays** for its inputs and outputs. It can be extended to objects for **matrices**, and with minor coding changes, developers can perform fast array processing.

SciPy includes functions for some advanced math problems (Integrals, differential equations, optimization, as well as data visualization).

Visualization libraries in Python

2. Visualization Libraries



Matplotlib lib package is the most well-known library for data visualization. It is great for making graphs and plots. The graphs are also highly customizable.

Seaborn is based on Matplotlib. It's very easy to generate various plots, such as heat maps, time series, and violin plots.

Algorithmic libraries in Python

3. Algorithmic Libraries



With machine learning algorithms, We're able to develop a model using our data set and obtain predictions.

The algorithmic libraries tackle some machine learning tasks from basic too complex.

Scikit-learn library contains tools, for statistical modeling, including regression, classification, clustering, and so on. This library is built on NumPy, SciPy, and Matplotlib.

Statsmodels is also a Python module that allows users to explore data, estimate statistical models, and perform statistical tests.

Importing and exporting data with Python

Importing Data

- Process of loading and reading data into Python from various resources
- **Two important properties:**
 - Format
 - various formats: .csv, .json, .xlsx, .hdf
 - File Path of dataset
 - Computer: /Desktop/mydata.csv
 - Internet: <https://archive.ics.uci.edu/autos/imports-85.data>

```
url_data = https://archive.ics.uci.edu/ml/machine-learning-databases/autos/imports-85.data
df = pd.read_csv(url_data, header=None)
df.head(5)
df.tail(5)

df.columns = headers
```

Exporting a Pandas dataframe to CSV

- Preserve progress anytime by saving modified dataset using

```
path="C:\Windows\...\ automobile.csv"

df.to_csv(path)
```

Exporting to different formats in Python

Data Format	Read	Save
csv	pd.read_csv()	df.to_csv()
json	pd.read_json()	df.to_json()
Excel	pd.read_excel()	df.to_excel()
sql	pd.read_sql()	df.to_sql()

Getting Started Analyzing data in Python

Basic insights from the data

- Understand your data before you begin any analysis
- Should check:
 - Data Types
 - Data Distribution
- Locate potential issues with the data

Explore the dataset.

Pandas has **built in methods** to understand the data type of features or to look at the distribution of data within the dataset.

Using these methods gives an overview of the dataset and also points out potential issues such as the wrong data type of features which may need to be resolved later on.

Data has a variety of types. The main types stored in Pandas objects are object, float, int, and date time. The data type names **are somewhat different** from those in native Python.

This table shows the difference and similarities between them.

Some are very similar, such as the numeric data types `int` and `float`. The **object** pandas type functions **similar** to **string** in Python, save for the change in name.

While the date time Pandas type is a very useful type for handling time series data.

Basic Insights of Dataset - Data Types

Pandas Type	Native Python Type	Description
object	string	numbers and strings
int64	int	Numeric characters
float64	float	Numeric characters with decimals
datetime64, timedelta[ns]	N/A (but see the datetime module in Python's standard library)	time data.

Why check data types?

- potential info and type mismatch
- compatibility with python methods

There are two reasons to check data types in a dataset.

Pandas **automatically assigns types based on the encoding** it detects from the original data table. For a number of reasons, this assignment **may be incorrect**.

For example, it would be awkward if the car price column, which we should expect contain continuous numeric numbers, is assigned the data type of object. It would be more natural for it to have the float type. May be is better **manually change** the data type to float.

The second reason is it allows an experienced data scientist to see **which Python functions can be applied** to a specific column. For example, some math functions can only be applied to numerical data. If these functions are applied to non-numerical data, an error may result.

Basic Insights of Dataset - Data Types

- In pandas, we use `dataframe.dtypes` to check data types

`df.dtypes`

```
symboling      int64
normalized-losses  object
make           object
fuel-type      object
aspiration     object
num-of-doors   object
body-style     object
drive-wheels   object
engine-location object
wheel-base    float64
length        float64
width         float64
height        float64
curb-weight    int64
engine-type    object
num-of-cylinders object
engine-size    int64
fuel-system    object
bore          object
stroke         object
compression-ratio float64
horsepower     object
peak-rpm       object
city-mpg       int64
```

When the data type method is applied to the dataset. The **data type of each column is returned** in a series.

A good data scientist's intuition tells us that most of the data types make sense.

They make of cars, for examples are names. This information should be of type object.

The last one on the list would be an issue. As bore is a dimension of an engine, we should expect a numerical data type to be used. Instead, the object type is used.

- Returns a statistical summary

`df.describe()`

	symboling	wheel-base	length	width	height	curb-weight
count	205.000000	205.000000	205.000000	205.000000	205.000000	205.000000
mean	0.834146	98.756585	174.049268	65.907805	53.724878	2555.565854
std	1.245307	6.021776	12.337289	2.145204	2.443522	520.680204
min	-2.000000	86.600000	141.100000	60.300000	47.800000	1488.000000
25%	0.000000	94.500000	166.300000	64.100000	52.000000	2145.000000
50%	1.000000	97.000000	173.200000	65.500000	54.100000	2414.000000
75%	2.000000	102.400000	183.100000	66.900000	55.500000	2935.000000
max	3.000000	120.900000	208.100000	72.300000	59.800000	4066.000000

- Provides full summary statistics

`df.describe(include="all")`

	symboling	normalized-losses	make	fuel-type	aspiration	num-of-doors	body-style	drive-wheels	engine-location
count	205.000000	205	205	205	205	205	205	205	205
unique	NaN	52	22	2	2	3	5	3	2
top	NaN	?	toyota	gas	std	four	sedan	fwd	front
freq	NaN	41	32	185	168	114	96	120	202
mean	0.834146	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
std	1.245307	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
min	-2.000000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
25%	0.000000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
50%	1.000000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
75%	2.000000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
max	3.000000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

The statistical summary of each column to learn about the distribution of data in each column.

The statistical metrics can tell the data scientist if there are mathematical issues that may exist, such as **extreme outliers and large deviations**, the data scientists may have to address these issues later.

To get the quick statistics, we use the described method. It returns the number of terms in the column as **count**, **average** column value as mean, column **standard deviation** as STD, and **maximum minimum** values, as well as the boundary of each of the quartiles.

By **default**, the data frame described functions **skips rows and columns that do not contain numbers**.

It is possible to make the described method work for object type columns as well. To enable a summary of all the columns, we could add an argument, include equals, all inside the described function bracket.

Now the outcome shows the summary of all 26 columns including object type attributes.

A different set of statistics is evaluated:

- **Unique** is the number of distinct objects in the column.
- **Top** is most frequently occurring object.
- **Frequency** is the number of times the top object appears in the column.

Some values in the table are shown here as NaN, which stands for not a number. This is because that particular statistical metric cannot be calculated for that specific column data type.

Basic Insights of Dataset - Info

`dataframe.info()` provides a concise summary of your DataFrame.

`df.info()`

Row Number

0	3	7	alfa-romeo	gas	std
1	3	7	alfa-romeo	gas	std
2	1	7	alfa-romeo	gas	std
3	2	164	audi	gas	std
4	2	164	audi	gas	std
5	2	7	audi	gas	std
6	1	130	audi	gas	std
7	1	7	audi	gas	std
8	1	130	audi	gas	torso
9	0	7	audi	gas	torso
10	2	182	bmw	gas	std
11	0	182	bmw	gas	std
12	0	180	bmw	gas	std
13	0	180	bmw	gas	std
14	1	7	bmw	gas	std
15	0	7	bmw	gas	std
16	0	7	bmw	gas	std
17	0	7	bmw	gas	std
18	2	121	chevrolet	gas	std
19	1	80	chevrolet	gas	std
20	0	80	chevrolet	gas	std
21	1	118	citroen	gas	std
22	1	118	citroen	gas	std
23	1	118	citroen	gas	std
24	1	149	citroen	gas	std
25	1	149	citroen	gas	std
26	1	149	citroen	gas	std
27	1	149	citroen	gas	std
28	-1	118	citroen	gas	std
29	3	145	citroen	gas	std
...	...	...	...	...	...
176	-1	65	toyota	gas	std
177	-1	65	toyota	gas	std
178	3	187	toyota	gas	std
179	3	187	toyota	gas	std
180	-1	84	toyota	gas	std
181	-1	84	toyota	gas	std
182	2	122	volkswagen	citroen	std
183	2	122	volkswagen	gas	std
184	2	84	volkswagen	citroen	std
185	2	84	volkswagen	gas	std
186	2	84	volkswagen	gas	std
187	2	84	volkswagen	citroen	std
188	2	84	volkswagen	gas	std
189	2	7	volkswagen	gas	std
190	2	200	volkswagen	gas	std
191	0	7	volkswagen	gas	std
192	0	7	volkswagen	citroen	std
193	0	7	volkswagen	gas	std
194	-0	103	volvo	gas	std
195	-1	74	volvo	gas	std
196	-0	103	volvo	gas	std
197	-1	74	volvo	gas	std
198	-0	103	volvo	gas	std
199	-1	74	volvo	gas	std
200	-1	85	volvo	gas	std
201	-1	85	volvo	gas	std
202	-1	85	volvo	gas	std

Another method you can use to check your dataset is the `dataframe.info` function. This function gives a concise summary of the data frame. This method prints information about a data frame including the index D type and columns non-null values, and memory usage.

Accessing Data Databases with Python

There is a mechanism by which the Python program communicates with the DBMS.

The Python code connects to the database using API calls.

Accessing databases using Python

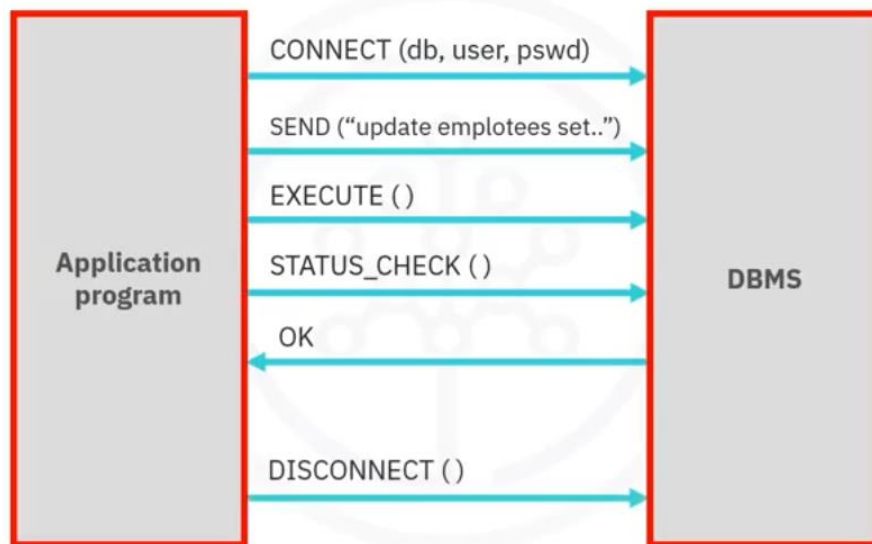


An application programming interface (API) is a set of functions that you can call to get access to some type of service.

A SQL API consists of library of function calls as an application programming interface (API) for the DBMS.

To pass SQL statements to the DBMS, an application program calls functions in the API, and it calls other functions to retrieve query results and status information from the DBMS.

What is a SQL API?



The basic operation of a typical SQL API is illustrated in the figure. The application program begins its database access with one or more API calls that **connects** the program to the DBMS.

To **send** a SQL statement to the DBMS, the program builds the statement as a text string in a buffer and then makes an API call to pass the buffer contents to the DBMS.

The application program makes API calls to **check the status** of its DBMS request and to handle errors.

The application program ends its database access with an API call that disconnects it from the database.

DB API is Python the standard API for accessing relational databases.

It is a standard that allows you to write a single program that works with multiple kinds of relational databases instead of writing a separate program for each one.

If you learn the DB API functions, then you can apply that knowledge to use any database with Python.

The two main concepts in the Python DB API are:

- **Connection** objects: To connect to a database and manage your transactions.
- **Query** objects: to run queries. You open a cursor object and then run queries. The cursor works similar to a cursor in a text processing system, where you scroll down in your result set and get your data into the application. Cursors are used to scan through the results of a database.

Concepts of the Python DB API



Connection Objects

- Database connections
- Manage transactions



Cursor Objects

- Database Queries

Here are the methods used with connection objects.

What are Connection methods?



- `cursor ()`
- `commit ()`
- `rollback ()`
- `close ()`

- Cursor method returns a new cursor object using the connection.
- Commit method is used to commit any pending transaction to the database.
- Rollback method causes the database to roll back to the start of any pending transaction.
- Closed method is used to close a database connection.

Writing code using DB-API

```
from dmodule import connect

#Create connection object
connection = connect( 'databasename' , 'username' , 'pswd' )

#Create a cursor object
cursor = connection.cursor ( )

#Run queries
cursor.execute( 'select * from mytable' )
results = cursor.fetchall ( )

#Free resources
cursor.close ( )
connection.close ( )
```

Uses the DB API to query a database.

1. **Import** your database module y using the connect API from that module.
2. Open a **connection** to the database. Pass in the parameters the database name, username, and password. The connect function returns a connection object.
3. Create a **cursor** object on the connection object. The cursor is used to run queries and fetch results.
4. **Closing** the connection avoids taking up resources.

Lesson Summary

- Each line in a dataset is a row, and commas separate the values.
- To understand the data, you must analyze the attributes for each column of data.

- Python libraries are collections of functions and methods that facilitate various functionalities without writing code from scratch and are categorized into Scientific Computing, Data Visualization, and Machine Learning Algorithms.
- Many data science libraries are interconnected; for instance, Scikit-learn is built on top of NumPy, SciPy, and Matplotlib.
- The data format and the file path are two key factors for reading data with Pandas.
- The **read_CSV** method in Pandas can read files in CSV format into a Pandas DataFrame.
- Pandas has unique data types like object, float, Int, and datetime.
- Use the **dtype** method to check each column's data type; misclassified data types might need manual correction.
- Knowing the correct data types helps apply appropriate Python functions to specific columns.
- Using **Statistical Summary** with **describe()** provides count, mean, standard deviation, min, max, and quartile ranges for numerical columns.
- You can also use **include='all'** as an argument to get summaries for object-type columns.
- The statistical summary helps identify potential issues like outliers needing further attention.
- Using the **info() Method** gives an overview of the top and bottom 30 rows of the DataFrame, useful for quick visual inspection.
- Some statistical metrics may return "NaN," indicating missing values, and the program can't calculate statistics for that specific data type.
- Python can connect to databases through specialized code, often written in Jupyter notebooks.
- SQL Application Programming Interfaces (APIs) and Python DB APIs (most often used) facilitate the interaction between Python and the DBMS.
- **SQL APIs** connect to DBMS with one or more API calls, build SQL statements as a text string, and use API calls to send SQL statements to the DBMS and retrieve results and statuses.
- **DB-API**, Python's standard for interacting with relational databases, uses **connection objects** to establish and manage database connections and **cursor objects** to run queries and scroll through the results.
- Connection Object methods include the **cursor()**, **commit()**, **rollback()**, and **close()** commands.
- You can import the database module, use the **Connect API** to open a connection, and then create a cursor object to run queries and fetch results.
- Remember to close the database connection to free up resources.

Lab Importing Data sets

Substitución de '?' por Non AlfaNumeric

```
Df1 = df.replace('?', np.NaN)
```

```
df.replace('?', np.NaN, inplace = True)
```

Module 2: Data Wrangling

Pre-processing Data in Python (data cleaning or data wrangling)

Is a **necessary** step in data analysis.

Data pre-processing

The process of converting or mapping data from the initial “raw” form into another format, in order to prepare the data for further analysis.

It is the process of converting or mapping data from one raw form into another format to make it ready for further analysis.

Learning objectives

- Identify and handle missing values
- Data Formatting
- Data Normalization (centering /scaling)
- Data Binning
- Turning Categorical to numeric variables

Identify and handle missing values: a data entry is left empty.

Data **formats**: Data from different sources may be in various formats, in different units, or in various conventions.

Data **Normalization**: Different columns of numerical data may have very different ranges, and direct **comparison** is often **not meaningful**. Centering and scaling are ways to bring all data into a similar range for more useful comparison. ly, we'll focus on

Data Binning creates bigger categories from a set of numerical values. It is particularly useful for comparison between groups of data.

Categorical variables conversion into numeric variables to make statistical modeling easier.

Simple dataframe operations

	symboling	normalized-losses	make	fuel-type	aspiration	num-of-doors	body-style	drive-wheels	engine-location	wheel-base	...	engine-size	fuel-system	bore	stroke	compression-ratio
0	3	?	alfa-romero	gas	std	two	convertible	rwd	front	88.6	...	130	mpfi	3.47	2.68	9.0
1	3	?	alfa-romero	gas	std	two	convertible	rwd	front	88.6	...	130	mpfi	3.47	2.68	9.0
2	1	?	alfa-romero	gas	std	two	hatchback	rwd	front	94.5	...	152	mpfi	2.68	3.47	9.0
3	2	164	audi	gas	std	four	sedan	fwd	front	99.8	...	109	mpfi	3.19	3.40	10.0
4	2	164	audi	gas	std	four	sedan	4wd	front	99.4	...	136	mpfi	3.19	3.40	8.0
5	2	?	audi	gas	std	two	sedan	fwd	front	99.8	...	136	mpfi	3.19	3.40	8.5
6	1	158	audi	gas	std	four	sedan	fwd	front	105.8	...	136	mpfi	3.19	3.40	8.5
7	1	?	audi	gas	std	four	wagon	fwd	front	105.8	...	136	mpfi	3.19	3.40	8.5
8	1	158	audi	gas	turbo	four	sedan	fwd	front	105.8	...	131	mpfi	3.13	3.40	8.3
9	0	?	audi	gas	turbo	two	hatchback	4wd	front	99.5	...	131	mpfi	3.13	3.40	7.0

In Python, we usually perform operations along columns. Each row of the column represents a sample.

You access a column by specifying the name of the column. Each column is a Pandas series.

There are many ways to manipulate data frames in Python:

Add a value (5) to each entry of a column.

```
Df["symbolin"] = Df["symbolin"] + 5
```

This changes each value of the data frame column by adding one to the current value.

Dealing with missing values in python

When **no data value** is stored for feature for a particular observation, we say this feature has a **missing value**.

Missing values

- What is missing value?
- Missing values occur when no data value is stored for a variable (feature) in an observation.
- Could be represented as “?”, “N/A”, 0 or just a blank cell.

	symboling	normalized-losses	make	fuel-type	aspiration	num-of-doors	body-style	drive-wheels	engine-location
0	3	NaN ←	alfa-romero	gas	std	two	convertible	rwd	front

Usually, missing value in dataset appears as question mark, N/A, zero or just a blank cell.

There are many ways to deal with missing values, and this is regardless of Python, R or whatever tool you use. Of course, each situation is different and should be judged differently. However, these are the typical options you can consider.

How to deal with missing data?

Check with the data collection source

Drop the missing values

- drop the variable
- drop the data entry

Replace the missing values

- replace it with an average (of similar datapoints)
- replace it by frequency
- replace it based on other functions

Leave it as missing data

Check if the person or group that collected the data can go back and find what the actual value should be.

Remove the data where that missing value is found. When you drop data, you could either drop the whole variable (**column-wise deletion**) or just the single data entry (**Row-wise deletion**) with the missing value. If you don't have a lot of observations with missing data, usually dropping the particular entry is the best.

If you're removing data, you want to look to do something that has **the least amount of impact**.

Replacing data is better since no data is wasted. However, it's less accurate since we need to replace missing data with a guess of what the data should be. One standard replacement technique is **to replace missing values by the average value of the entire variable**.

As an example, suppose we have some entries that have missing values for the normalized losses column and the column average for entries with data is 4,500. While there is no way for us to get an accurate guess of what the missing values under the normalized losses column should have been, you can approximate their values using the average value of the column, 4,500.

But what if the values cannot be averaged as with **categorical variables**?

For a variable like fuel type, there isn't an average fuel type since the variable values are not numbers. In this case, one possibility is to try **using the mode** the most common, like gasoline.

Finally, sometimes we may find another way to guess the missing data. This is usually because **the data gatherer knows something additional about the missing data**. For example, he may know that the missing values tend to be old cars, and the normalized losses of old cars are significantly higher than the average vehicle.

And of course, finally in some cases you may simply want to leave the missing data as missing data. For one reason or another it may be useful to keep that observation even if some features are missing.

Now, let's go into how to drop missing values or replace missing values in Python.

To remove data that contains missing values, Pandas library has a built in method called `dropna`.

How to drop missing values in Python

- Use `dataframes.dropna()`:

highway-mpg	price
...	...
20	23875
22	NaN
29	16430
...	...

highway-mpg	price
...	...
20	23875
29	16430
...	...

axis=0 drops the entire row
axis=1 drops the entire column

```
df.dropna(subset=["price"], axis=0, inplace = True)
```

Essentially, with the `dropna` method, you can choose to drop rows or columns that contain missing values like NaN. So you'll need to **specify axis = 0 to drop the rows, or axis = 1 to drop the columns** that contain the missing values.

In this example, there is a missing value in the price column. Since the price of used cars is what we're trying to predict in our upcoming analysis, we'd have to remove the cars, the

rows that don't have a listed price. it can simply be done in one line of code using `dataframe.dropna`.

Setting the argument in place to True allows the modification to be done on the data set directly, `inplace = True` just writes the result back into the data frame.

```
Df.dropna(subset=["Price"], axis=0, inplace = True)
```

This is equivalent to this line of code.

```
Df = df. dropna(subset=["Price"], axis=0)
```

Don't forget

```
df.dropna(subset=["price"], axis=0)
```



```
df.dropna(subset=["price"], axis=0, inplace = True)
```

Don't forget that first line of code does not change the data frame but it's a good way to make sure that you are performing the correct operation. To modify the data frame, you have to set the parameter `inplace = True`.

How to replace missing values in Python

Use `dataframe.replace(missing_value, new_value)`:



normalized-losses	make
...	...
164	audi
164	audi
NaN	audi

normalized-losses	make
...	...
164	audi
164	audi
162	audi
158	audi
...	...

```
mean = df["normalized-losses"].mean()
```

```
df["normalized-losses"].replace(np.nan, mean)
```

To replace missing values like NaNs with actual values, Pandas library has a built in method called `Replace`, which can be used to fill in the missing values with the newly calculated values.

As an example, assume that we want to replace the missing values of the variable `normalized losses` by the mean value of the variable. Therefore, the missing value should be replaced by the average of the entries within that column. In Python, first we calculate the mean of the column. Then we use the method `Replace` to specify the value we would like to be replaced as the first parameter, in this case `NaN`. The second parameter is the value we would like to replace it with, the mean in this example.

This is a fairly simplified way of replacing missing values. There are, of course, other techniques such as replacing missing values for the average of the group instead of the entire data set.

Data Formatting in Python

Data is usually collected from different places, by different people, which may be stored in different formats. Data formatting means bringing data into a common standard of expression that allows users to make **meaningful comparisons**. As a part of dataset cleaning, data formatting ensures that data is **consistent and easily understandable**.

Data Formatting

- Data is usually collected from different places and stored in different formats
- Bringing data into a common standard of expression allows users to make meaningful comparison

Non-formatted:

- confusing
- hard to aggregate
- hard to compare

City
N.Y.
Ny
NY
New York



City
New York
New York
New York
New York

Formatted:

- more clear
- easy to aggregate
- easy to compare

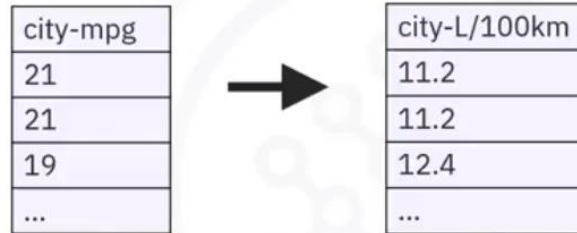
For example, people may use different expressions to represent New York City, such as N.Y., Ny, NY, and New York. Sometimes this unclean data is a good thing to see.

For example, if you're looking at the different ways people tend to write New York, then this is exactly the data that you want. Or if you're looking for ways to spot fraud, perhaps writing N.Y. is more likely to predict an anomaly than if someone wrote out New York in full.

But perhaps, more often than not, we just simply want to treat them all as the same entity or format to make statistical analysis easier down the road.

Applying calculations to an entire column

- Convert “mpg” to “L/100km” in Car dataset



city-mpg
21
21
19
...

city-L/100km
11.2
11.2
12.4
...

```
df["city-mpg"] = 235/df["city-mpg"]
```

```
df.rename(columns={"city_mpg": "city-L/100km"}, inplace=True)
```

Referring to our used car dataset, there's a feature named city-miles per gallon in the dataset, which refers to a car fuel consumption in miles per gallon unit. However, you may be someone who lives in a country that uses metric units, so you would want to convert those values to liters per 100 kilometers, the metric version. To transform miles per gallon to liters per 100 kilometers, we need to divide 235 by each value in the city miles per gallon column.

In Python, this can easily be done in one line of code:

In the second line of code, rename column name from city-miles per gallon to city-liters per 100 kilometers using the data frame rename method.

Incorrect data types

- Sometimes the wrong data type is assigned to a feature

```
df["price"].tail(5)

200    16845
201    19045
202    21485
203    22470
204    22625
Name: price, dtype: object
```

For a number of reasons, including when you imported dataset into Python, the **data type may be incorrectly established**. For example, here we notice that the assigned data type to the price feature is object although the expected data type should really be an integer or float type. It is important for later analysis to explore the feature's data type and convert them to the correct data types. Otherwise, the developed models later on may behave strangely and totally valid data may end up being treated like missing data.

Data types in Python and Pandas

- There are many data types in Pandas
- Objects : "A", "Hello"..
- Int64 : 1,3,5
- Float64 : 2.123, 632.31,0.12

There are many data types in Pandas. Objects can be letters or words. Int64 are integers, and floats are real numbers.

Correcting data types

To *identify* data types:

- Use `dataframe.dtypes()` to identify data type

To *convert* data types:

- Use `dataframe.astype()` to convert data type

Example: Convert data type to integer in column “price”

```
df["price"] = df["price"].astype("int")
```

To identify a feature's data type in Python, we can use the `dataframe.dtypes()` method and check the data type of each variable in a data frame.

In the case of wrong data types, the method `dataframe.astype()` can be used to convert a data type from one format to another.

For example, using `astype("int")` for the price column, you can convert the object column into an integer type variable.

Data Normalization in Python

Data normalization is an important technique to understand in data preprocessing.

Data Normalization

- Uniform the features value with different range.

length	width	height
168.8	64.1	48.8
168.8	64.1	48.8
171.2	65.5	52.4
176.6	66.2	54.3
176.6	66.4	54.3
177.3	66.3	53.1
192.7	71.4	55.7
192.7	71.4	55.7
192.7	71.4	55.9

scale	[150,250]	[50,100]	[50,100]
impact	large	small	small

When we take a look at the used car data set, we notice in the data that the feature length ranges from 150 to 250, while feature width and height ranges from 50 to 100. We may want to normalize these variables so that the range of the values is consistent.

This normalization can make some statistical analyses easier down the road.

By making the ranges consistent between variables. **Normalization enables a fairer comparison between the different features, making sure they have the same impact.**

Data Normalization

age	income
20	100000
30	20000
40	500000



age	income
0.2	0.2
0.3	0.04
0.4	1

Not-normalized

- “age” and “income” are in different range.
- hard to compare
- “income” will influence the result more

Normalized

- similar value range.

Consider a data set containing two features: age and income, where age ranges from 0 to 100, while income ranges from 0 to 20,000 and higher. Income is about 1,000 times larger than age and ranges from 20,000 to 500,000. So these two **features are in very different ranges**.

When we do further analysis, like linear regression, for example, the attribute "income" **will intrinsically influence the result more due to its larger value**. But this doesn't necessarily mean it is more important as a predictor. So the nature of the data biases the linear regression model to weigh income more heavily than age.

To avoid this, we can normalize these two variables into values that range from 0 to 1. After normalization, both variables now have a **similar influence** on the models we will build later.

There are several ways to normalize data.

Methods of normalizing data

Several approaches for normalization:

①

$$x_{new} = \frac{x_{old}}{x_{max}}$$

Simple Feature scaling

②

$$x_{new} = \frac{x_{old} - x_{min}}{x_{max} - x_{min}}$$

Min-Max

③

$$x_{new} = \frac{x_{old} - \mu}{\sigma}$$

Z-score

Simple Feature scaling: divides each value by the maximum value for that feature. This makes the **new values range between 0 and 1**.

Simple Feature Scaling in Python

With Pandas:

length	width	height
168.8	64.1	48.8
168.8	64.1	48.8
180.0	65.5	52.4
...	...	...



length	width	height
0.81	64.1	48.8
0.81	64.1	48.8
0.87	65.5	52.4
...	...	...

```
df["length"] = df["length"]/df["length"].max()
```

Min max: takes each value x underscore old, subtracts it from the minimum value of that feature, then divides by the range of that feature. Again, the resulting **new values range between 0 and 1**.

Min-max in Python

With Pandas:

length	width	height
168.8	64.1	48.8
168.8	64.1	48.8
180.0	65.5	52.4
...	...	...



length	width	height
0.41	64.1	48.8
0.41	64.1	48.8
0.58	65.5	52.4
...	...	...

```
df["length"] = (df["length"]-df["length"].min())/
                (df["length"].max()-df["length"].min())
```

Z-score or standard score: for each value, you subtract the mu, which is the average of the feature and then divide by the standard deviation sigma. New values hover around zero and typically range between -3 and +3 three, but can be higher or lower.

Z-score in Python

With Pandas:

length	width	height
168.8	64.1	48.8
168.8	64.1	48.8
180.0	65.5	52.4
...	...	...



length	width	height
-0.034	64.1	48.8
-0.034	64.1	48.8
0.039	65.5	52.4
...	...	...

```
df["length"] = (df["length"]-df["length"].mean())/df["length"].std()
```

Binning in Python

Binning as a method of data preprocessing.

Binning is when you group values together into bins. Sometimes binning can **improve accuracy** of the predictive models.

Binning

- Binning: Grouping of values into bins
- Converts numeric into categorical variables
- Group a set of numerical values into a set of “bins”
- “price” is an feature range from 5,000 to 45,500



In addition, sometimes we use data binning to **group a set of numerical values into a smaller number of bins to have a better understanding of the data distribution**. As example, "price" here is an attribute range from 5,000 to 45,500. Using binning, we **categorize the price** into three bins: low price, medium price and high prices.

In the actual car dataset, price is a numerical variable ranging from 5,188 to 45,400. It has 201 unique values. We can categorize them into **three bins**, low, medium and high price cars.

Binning in Python Pandas

price		price	price-binned
13495		13495	Low
16500		16500	Low
18920		18920	Medium
41315		41315	High
5151		5151	Low
6295		6295	Low
...		...	...

```
bins = np.linspace(min(df["price"]), max(df["price"]), 4)
```

```
group_names = ["Low", "Medium", "High"]
```

```
df["price-binned"] = pd.cut(df["price"], bins, labels=group_names, include_lowest=True)
```

In Python, we can easily implement the binning.

We would like **three bins of equal bin width, so we need four numbers as dividers that are equal distance apart**. First, we use the NumPy function `linspace` to return the array bins that contains four equally spaced numbers over the specified interval of the price.

```
>>> import numpy as np
>>> np.linspace(2.0, 3.0, num=5)
array([2. , 2.25, 2.5 , 2.75, 3.  ])
>>> np.linspace(2.0, 3.0, num=5, endpoint=False)
array([2. , 2.2, 2.4, 2.6, 2.8])
>>> np.linspace(2.0, 3.0, num=5, retstep=True)
(array([2. , 2.25, 2.5 , 2.75, 3.  ]), 0.25)
```

We create a list `group_names` that contains the different bin names.

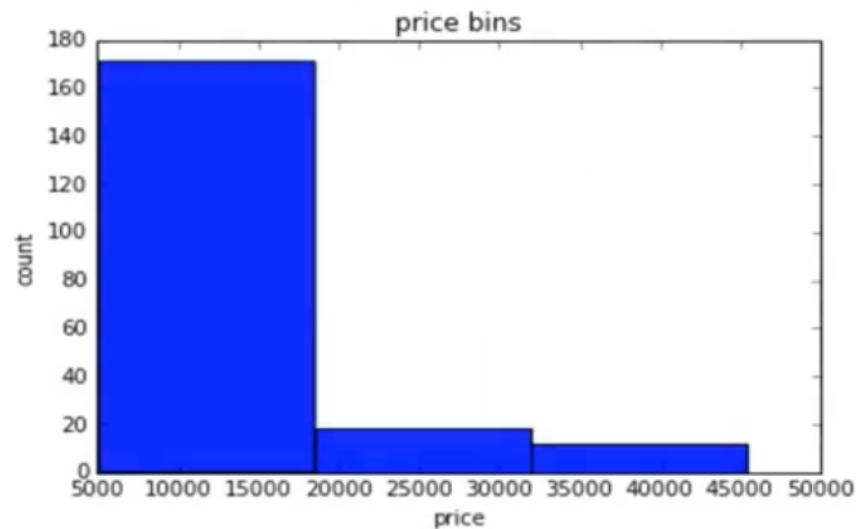
We use the Pandas function `cut` to **segment and sort the data values into bins**

```
>>> pd.cut(np.array([1, 7, 5, 4, 6, 3]),
...        3, labels=["bad", "medium", "good"])
['bad', 'good', 'medium', 'medium', 'good', 'bad']
Categories (3, object): ['bad' < 'medium' < 'good']
```

You can then use histograms to visualize the distribution of the data after they've been divided into bins.

Visualizing binned data

- Histograms



This is the histogram that we plotted based on the binning that we applied in the price feature. From the plot, it is clear that most cars have a low price and only very few cars have high price.

Turning categorical variables into quantitative variables in Python

Most **statistical models** cannot take in objects or strings as input, and for model training, **only take the numbers** as inputs.

Categorical variables

Problem:

- Most statistical models cannot take in the objects/strings as input

Car	Fuel	...
A	gas	...
B	diesel	...
C	gas	...
D	gas	...

Categorical → Numeric

Solution:

- Add dummy variables for each unique category
- Assign 0 or 1 in each category

Car	Fuel	...	gas	diesel
A	gas	...	1	0
B	diesel	...	0	1
C	gas	...	1	0
D	gas	...	1	0

“One-hot encoding”

In the car data set, the fuel type feature as a **categorical variable** has two values; gas or diesel which are in string format.

We encode the values by **adding new features corresponding to each unique element** in the original feature we would like to encode.

In the case where the feature fuel has two unique values, gas and diesel, we create two new features, gas and diesel. When a value occurs in the original feature, we set the corresponding value to one in the new feature, the rest of the features are set to zero. This technique is often called **one hot encoding**.

In pandas, we can use `get_dummies` method to convert categorical variables to dummy variables.

Dummy variables in Python pandas

- Use `pandas.get_dummies()` method.
- Convert categorical variables to dummy variables (0 or 1)



fuel
gas
diesel
gas
gas

gas	diesel
1	0
0	1
1	0
1	0

```
pd.get_dummies(df['fuel'])
```

In Python, transforming categorical variables to dummy variables is simple. Following the example `pd.get_dummies` method gets the fuel type column and creates the data frame `dummy_variable_one`. The `get_dummies` method automatically generates a list of numbers, each one corresponding to a particular category of the variable.

Lesson Summary

- Data formatting is critical for making data from various sources consistent and comparable.
- Master the techniques in Python to convert units of measurement, like transforming "city miles per gallon" to "city-liters per 100 kilometers" for ease of comparison and analysis.
- Acquire skills to identify and correct data types in Python, ensuring the data is accurately represented for subsequent statistical analyses.
- Data normalization helps make variables comparable and helps eliminate inherent biases in statistical models.
- You can apply Feature Scaling, Min-Max, and Z-Score to normalize data and apply each technique in Python using pandas' methods.
- Binning is a method of data pre-processing to improve model accuracy and data visualization.

- Run binning techniques in Python using numpy's "linspace" and pandas' "cut" methods, particularly for numerical variables like "price."
- Utilize histograms to visualize the distribution of binned data and gain insights into feature distributions.
- Statistical models generally require numerical inputs, making it necessary to convert categorical variables like "fuel type" into numerical formats.
- You can implement the one-hot encoding technique in Python using pandas' **get_dummies** method to transform categorical variables into a format suitable for machine learning models.

Laboratory

```
missing_data = df.isnull()
```

nos crea un DF de booleanos

```
missing_data.columns.values.tolist()
```

Crea una lista con los nombres de las columnas

```
missing_data[column].value_counts()
```

Devuelve **la frecuencia de los valores únicos** de la Serie de valores de una columna.

```
df['num-of-doors'].value_counts().idxmax()
```

Devuelve el **valor más frecuente**

```
df.reset_index(drop=True, inplace=True)
```

Hay que usarlo so borramos filas del dataframe

```
df[["bore", "stroke"]] = df[["bore", "stroke"]].astype("float")
```

```
df[["normalized-losses"]] = df[["normalized-losses"]].astype("int")
```

```
df[["price"]] = df[["price"]].astype("float")
```

```
df[["peak-rpm"]] = df[["peak-rpm"]].astype("float")
```

Para cambiar los tipos de datos

```
bins = np.linspace(min(df["horsepower"]), max(df["horsepower"]), 4)
```

```
group_names = ['Low', 'Medium', 'High']
```

```
df['horsepower-binned'] = pd.cut(df['horsepower'], bins, labels=group_names,
include_lowest=True )
```

Para **hacer agrupar en tres categorías** los valores del una variable

```
common_screen_size = df['Screen_Size_cm'].value_counts().idxmax()
```

Para saber el **valor más frecuente** de una columna

```
# Data standardization: convert weight from kg to pounds
f["Weight_kg"] = df["Weight_kg"]*2.205
df.rename(columns={'Weight_kg':'Weight_pounds'}, inplace=True)
```

Tras el cambio de unidades para **renombrar una columna**.

```
dummy_variable_1 = pd.get_dummies(df["fuel-type"])
dummy_variable_1.rename(columns={'gas':'fuel-type-gas', 'diesel':'fuel-type-diesel'},
inplace=True)

# merge data frame "df" and "dummy_variable_1"
df = pd.concat([df, dummy_variable_1], axis=1)
# drop original column "fuel-type" from "df"
df.drop("fuel-type", axis = 1, inplace=True)
```

```
df[['Screen_Size_cm']] = np.round(df[['Screen_Size_cm']],2)
```

Para redondear los valores de una columna a dos decimales

Module 3: Exploratory Data Analysis

Exploratory data analysis (EDA), is an approach to analyze data to summarize main characteristics of the data, gain better understanding of the data set, uncover relationships between different variables and extract important variables for the problem we're trying to solve.

Main question: what are the characteristics that have the most impact on the dependent variable?

To answer this question we will use different EDA techniques:

- **Descriptive statistics:** describe basic features of a data set and obtains a short summary about the sample and measures of the data.
- Basic of **grouping** data: Help to transform our data set.
- ANOVA, the **analysis of variance** a statistical method in which the variation in a set of observations is divided into distinct components.

- **Correlation** between different variables.
- Advanced correlation: Pearson correlation and correlation heatmaps.

Exploratory Data Analysis (EDA)

- Preliminary step in data analysis to:
 - Summarize main characteristics of the data
 - Gain better understanding of the data set
 - Uncover relationships between variables
 - Extract important variables
- Question:
“What are the characteristics which have the most impact on the car price?”

Learning Objectives

In this lesson you will learn about:

- Descriptive Statistics
- GroupBy
- ANOVA
- Correlation
- Correlation - Statistics

Descriptive Statistics

Descriptive Statistics

- Explore data before building complicated models
- Calculate descriptive statistics for your data
- Describe basic features of data
- Giving short summaries about the sample and measures of the data

Explore your data before you spend time building complicated models, Calculating some statistics for your data to describe basic features of a dataset and obtains a short summary about the sample and measures of the data.

Let's show you a couple different useful methods.

Describe()

Pandas `df.describe()` computes basic statistics (count, mean, std, min, Quartiles (1,2,3) max) for all **numerical variables**. Any **NaN values are automatically skipped** in these statistics.

This function will give you a clearer idea of the distribution of your different variables.

Descriptive Statistics: Describe()

- Summarize statistics using pandas **describe()** method

```
df.describe()
```

	Unnamed: 0	symboling	normalized-losses	wheel-base	length	width	height	curb-weight	engine-size	bore	stroke
count	201.000000	201.000000	164.000000	201.000000	201.000000	201.000000	201.000000	201.000000	201.000000	201.000000	201.000000
mean	100.000000	0.840796	122.000000	98.797015	174.200995	65.889055	53.766667	2555.666667	126.875622	3.319154	3.256766
std	58.167861	1.254802	35.442168	6.066366	12.322175	2.101471	2.447822	517.296727	41.546834	0.280130	0.316049
min	0.000000	-2.000000	65.000000	86.600000	141.100000	60.300000	47.800000	1488.000000	61.000000	2.540000	2.070000
25%	50.000000	0.000000	NaN	94.500000	166.800000	64.100000	52.000000	2169.000000	98.000000	3.150000	3.110000
50%	100.000000	1.000000	NaN	97.000000	173.200000	65.500000	54.100000	2414.000000	120.000000	3.310000	3.290000
75%	150.000000	2.000000	NaN	102.400000	183.500000	66.600000	55.500000	2926.000000	141.000000	3.580000	3.410000
max	200.000000	3.000000	256.000000	120.900000	208.100000	72.000000	59.800000	4066.000000	326.000000	3.940000	4.170000

Values_counts()

You could have also **categorical variables** in your dataset. These are variables that can be divided up into different categories or groups and have discrete values.

One way you can summarize the categorical data is by using the function `value_counts`.

We can change the name of the column to make it easier to read.

Descriptive Statistics: Value_Counts()

- Summarize the categorical data is by using the **value_counts()** method

```
drive_wheels_counts=df["drive-wheels"].value_counts()
```

```
drive_wheels_counts.rename(columns={'drive-wheels':'value_counts' inplace=True)  
drive_wheels_counts.index.name= 'drive-wheels'
```

	value_counts
drive-wheels	
fwd	118
rwd	75
4wd	8

Box Plots

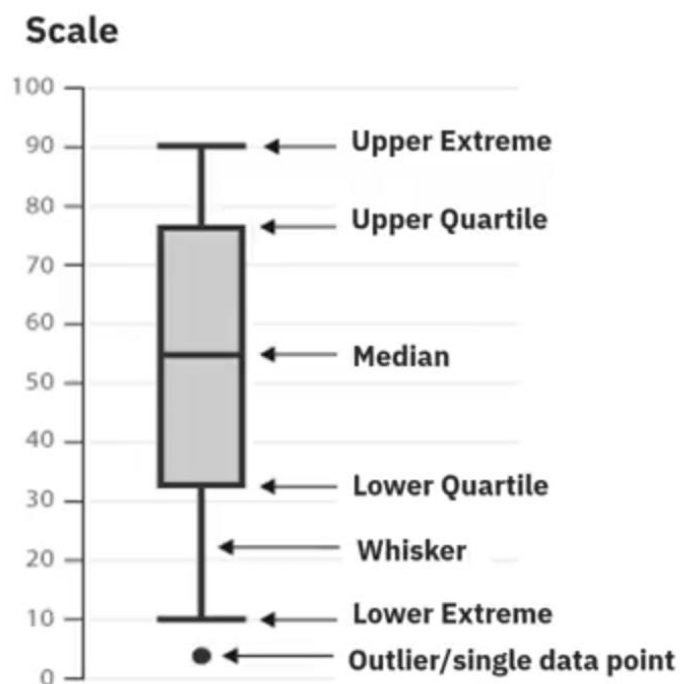
Visualize numeric data to compare various distributions of the data.

Box plot shows:

- **median** which represents where the middle data point is.
- **upper quartile** shows where the 75th percentile is,
- **lower quartile** shows where the 25th percentile is.
- **Inter-quartile range:** The data between the upper and lower quartile
- **Lower and upper extremes:** calculated as 1.5 times the inter-quartile range above the 75th percentile, and as 1.5 times the IQR below the 25th percentile.
- **Outliers:** data that occur outside the upper and lower extremes.

With box plots, you can easily spot outliers and also see the **distribution and skewness** of the data.

Descriptive Statistics: Box Plots

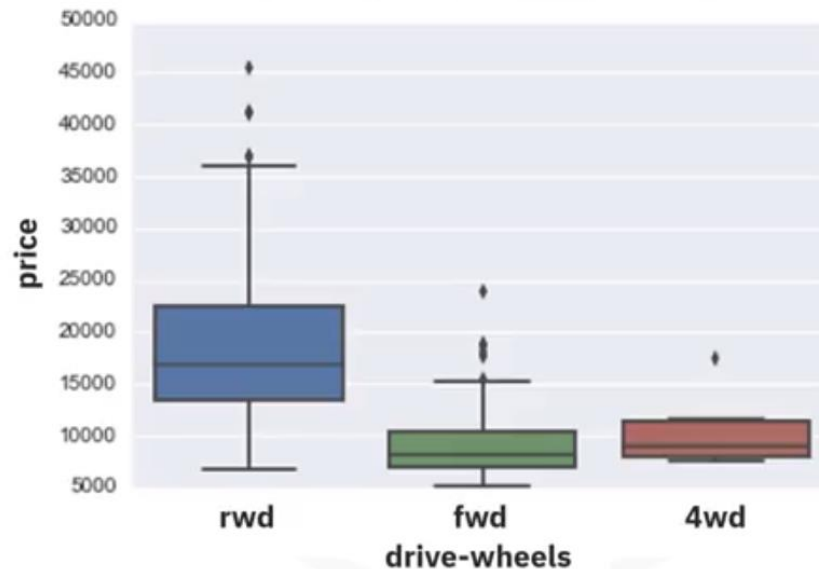


Box plots make it easy to compare between groups

In this example, using box plot, we can see the distribution of different categories of the drive-wheels feature over price feature. We can see that the distribution of price between the rear wheel-drive (rwd) and the other categories are distinct. But the price for front wheel-drive and four wheel-drive are almost indistinguishable.

Box Plot: Example

```
sns.boxplot(x= "drive-wheels", y= "price", data=df)
```



Scatter Plot

Oftentimes, we tend to see **continuous variables** in our data.

Each observation in a scatter plot is represented as **a point**.

This plot shows the **relationship between two variables**.

- The **predictor variable** is the variable that you are using to predict an outcome. variable on the **x-axis**.
- The **target variable** is the variable that you are trying to predict. variable on
- the **y-axis**

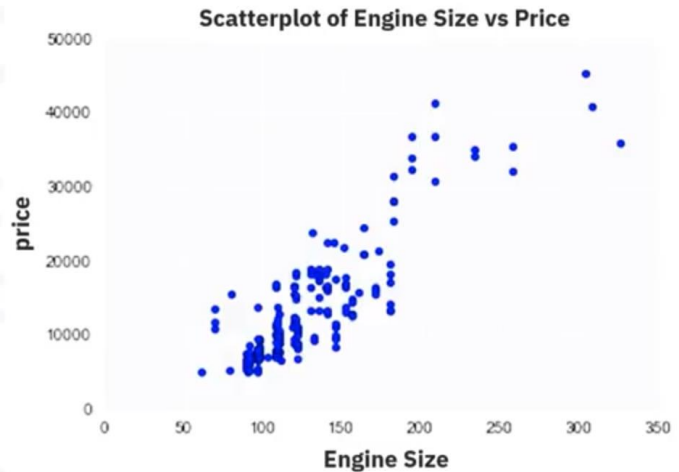
We are using the Matplotlib function scatter here, taking in x and a y variable.

Something to note is that it's always important to **label your axes** and write a general plot title so that you know what you're looking at.

Scatterplot: Example

```
y=df["price"]
x=df["engine-size"]
plt.scatter(x,y)

plt.title("Scatterplot of Engine
Size vs Price")
plt.xlabel("Engine Size")
plt.ylabel("Price")
```



Now, how is the variable engine size related to price?

From the scatter plot, we see that as the engine size goes up, the price of the car also goes up. This is giving us an initial indication that there is a positive linear relationship between these two variables.

Group by in python

Which type of drive system adds the most value to a vehicle?

Groupby()

The groupby method is used **on categorical variables**, groups the data into subsets according to the different categories of that variable, you can group by a single variable, or you can group by multiple variables by passing in multiple variable names.

Grouping data

Question:

Is there any relationship between the different types of “drive system” and the “price” of the vehicles?

- Use Panda **dataframe.groupby()** method:
 - Can be applied to categorical variables
 - Group data into categories
 - Single or multiple variables

As an example, let's say we are interested in finding the average price of vehicles and observe how they differ between different types of body styles and drive wheels variables.

To do this, we first pick out the three data columns we are interested in, which is done in the first line of code.

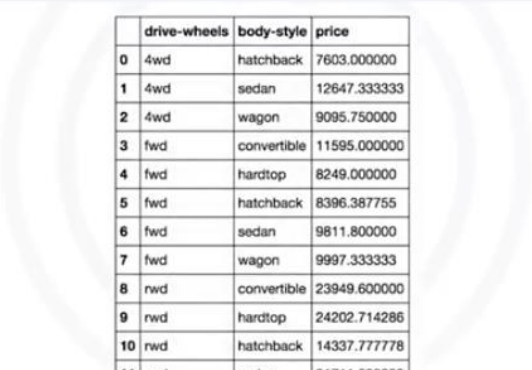
We then group the reduced data according to drive wheels and body style in the second line.

Since we are interested in knowing how the average price differs across the board, we can take **the mean** of each group and append at this bit at the very end of the Line 2.

The data is now grouped into subcategories, and only the average price of each subcategory is shown.

groupby(): Example

```
df_test = df[['drive-wheels', 'body-style', 'price']]
df_grp = df_test.groupby(['drive-wheels', 'body-style'], as_index=False).mean()
df_grp
```



	drive-wheels	body-style	price
0	4wd	hatchback	7603.000000
1	4wd	sedan	12647.333333
2	4wd	wagon	9095.750000
3	fwd	convertible	11595.000000
4	fwd	hardtop	8249.000000
5	fwd	hatchback	8396.387755
6	fwd	sedan	9811.800000
7	fwd	wagon	9997.333333
8	rwd	convertible	23949.600000
9	rwd	hardtop	24202.714286
10	rwd	hatchback	14337.777778
11	rwd	sedan	21711.833333
12	rwd	wagon	16994.222222

Setting `as_index=False` in `groupby()` provides a more "SQL-style" output where the grouped-by columns are retained as regular columns, making the resulting DataFrame easier to handle, query, and integrate with other operations and functions. It avoids the creation of a new index from the grouping keys, which is the default behavior.

Pivot()

A table of this form isn't the easiest to read and also not very easy to visualize. To make it easier to understand, we can transform this table to a pivot table by using the pivot method.

A pivot table has one variable displayed along the columns and the other variable displayed along the rows.

Pandas method: Pivot()

- One variable is displayed along the columns, and the other variable is displayed along the rows

```
df_pivot = df_grp.pivot(index='drive-wheels', columns='body-style')
```

	price				
body-style	convertible	hardtop	hatchback	sedan	wagon
drive-wheels					
4wd	0.0	0.000000	7603.000000	12647.333333	9095.750000
fwd	11595.000000	8429.000000	8396.387755	9811.800000	9997.333333
rwd	23949.600000	24202.714286	14337.777778	21711.833333	16994.222222

Just with one line of code and by using the Pandas pivot method, we can pivot the body style variable so it is displayed along the columns, and the drive wheels will be displayed along the rows.

The price data now becomes a rectangular grid, which is easier to visualize.

This is like what is usually done in Excel spreadsheets.

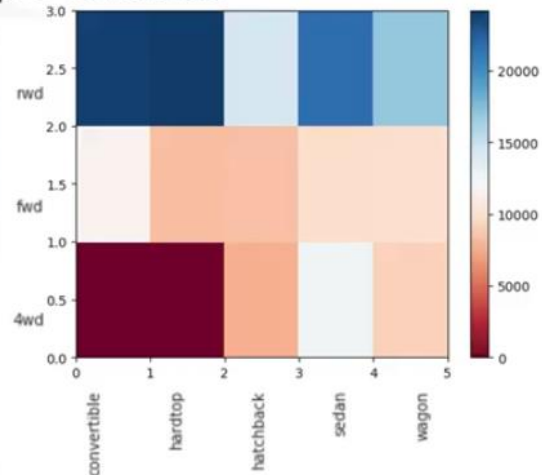
Heatmap()

Heat map takes a rectangular grid of data and assigns a color intensity based on the data value at the grid points. It is a great way to plot the target variable over multiple variables, and through this, get visual clues of the relationship between these variables and the target.

Heatmap

- Plot target variable against multiple variables

```
plt.pcolor(df_pivot, cmap='RdBu')  
plt.colorbar()  
plt.show()
```



In this example, we use Pyplot's pcolor method to plot heat map and **convert the previous pivot table into a graphical form**. We specified the red-blue color scheme.

In the output plot, each type of body style is numbered along the x-axis, and each type of drive wheels is numbered along the y-axis.

The average prices are plotted with varying colors based on their values according to the color bar.

Correlation

Correlation

What is Correlation?

- Measures to what extent different variables are interdependent
- For example:
 - Lung cancer → Smoking
 - Rain → Umbrella
- Correlation doesn't imply causation

Correlation is a statistical metric for measuring to what extent different **variables are interdependent**. if one variable changes, how does this affect change in the other variable?

It is important to know that correlation doesn't imply causation.

In fact, we can say that umbrella and rain are correlated, but we would not have enough information to say whether the umbrella caused the rain or the rain caused the umbrella.

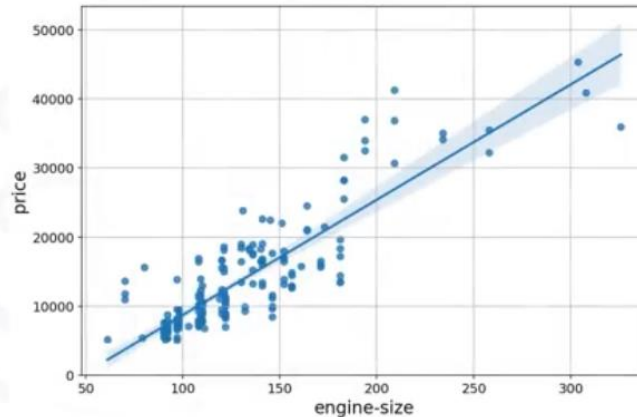
In this example, you can see that the straight line through the data points is very steep, which shows that there is **a positive linear relationship between the two variables**.

We can use seaborn reg plot to create the scatter plot.

Correlation: Positive linear relationship

- Correlation between two features (engine-size and price)

```
sns.regplot(x="engine-size", y="price",  
data=df)  
plt.ylim(0,)
```



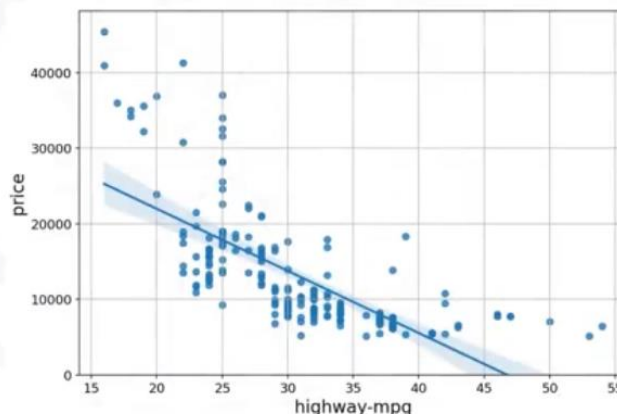
Therefore, there is a **negative linear relationship** between highway miles per gallon and price.

Although this relationship is negative, the **slope of the line is steep**, which means that the highway miles per gallon is still a good predictor of price. These two variables are said to have a negative correlation.

Correlation: Negative linear relationship

- Correlation between two features (highway-mpg and price)

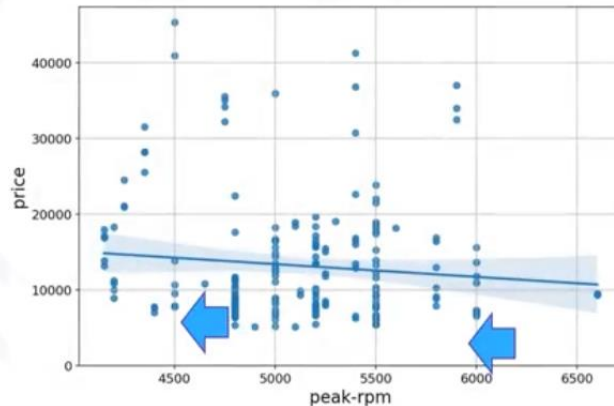
```
sns.regplot(x="highway-mpg", y="price", data=df)  
plt.ylim(0,)
```



Correlation: Negative linear relationship

- Weak correlation between two features (peak-rpm and price)

```
sns.regplot(x="peak-rpm", y="price", data=df)  
plt.ylim(0,)
```



Finally, we have an example of a **weak correlation**. For example, both low peak RPM and high values of peak RPM have low and high prices. Therefore, we cannot use RPM to predict the values

Correlation Statistics

One way to measure the strength of the **correlation between continuous numerical variables** is by using a method called Pearson Correlation.

Pearson Correlation method will give you two values:

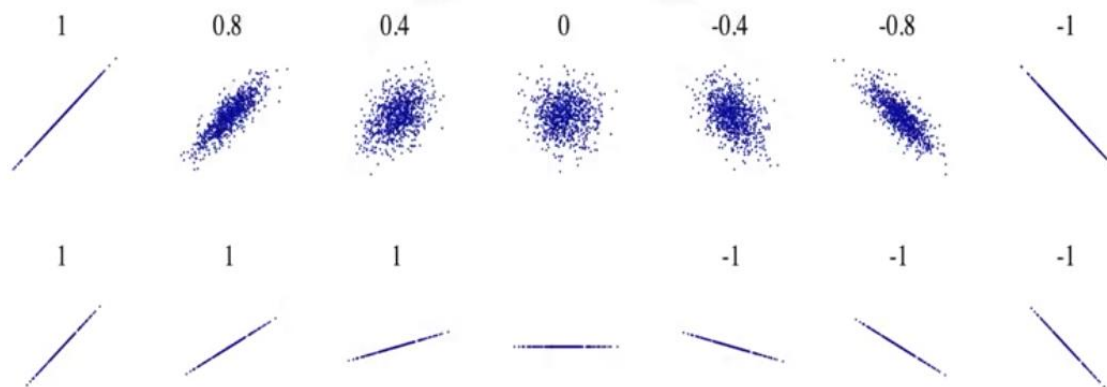
- Correlation coefficient:
 - a value close to one implies a large **positive** correlation.
 - while a value close to -1 implies a large **negative** correlation.
 - and a value close to zero implies **no** correlation between the variables.
- p-value will tell **us how certain** we are about the correlation that we calculated.
 - a value less than 0.001 gives us a **strong** certainty about the correlation coefficient that we calculated
 - a value between 0.001 and 0.05 gives us **moderate** certainty.
 - a value between 0.05 and 0.1 will give us a **weak** certainty.
 - a value larger than 0.1 will give us **no** certainty of correlation at all.

We can say that there is a **strong correlation** when the correlation coefficient is close to one or -1 and the p-value is less than 0.001.

Pearson Correlation

- Measure the strength of the correlation between two features
 - Correlation coefficient
 - P-value
- Correlation coefficient
 - Close to +1: Large Positive relationship
 - Close to -1: Large Negative relationship
 - Close to 0: No relationship
- Strong correlation
 - Correlation coefficient close to 1 or -1
 - P value less than 0.001
- P-value
 - P-value < 0.001 Strong certainty in the result
 - P-value < 0.05 Moderate certainty in the result
 - P-value < 0.1 Weak certainty in the result
 - P-value > 0.1 No certainty in the result

Pearson Correlation



https://en.wikipedia.org/wiki/Correlation_and_dependence

In this example, we want to look at the correlation between the variables horsepower and car price. See how easy you can calculate the Pearson Correlation using the Scipy stats package?

We can see that the correlation coefficient is approximately 0.8 and this is close to one, so there's a **strong positive correlation**. We can also see that the p-value is very small, much

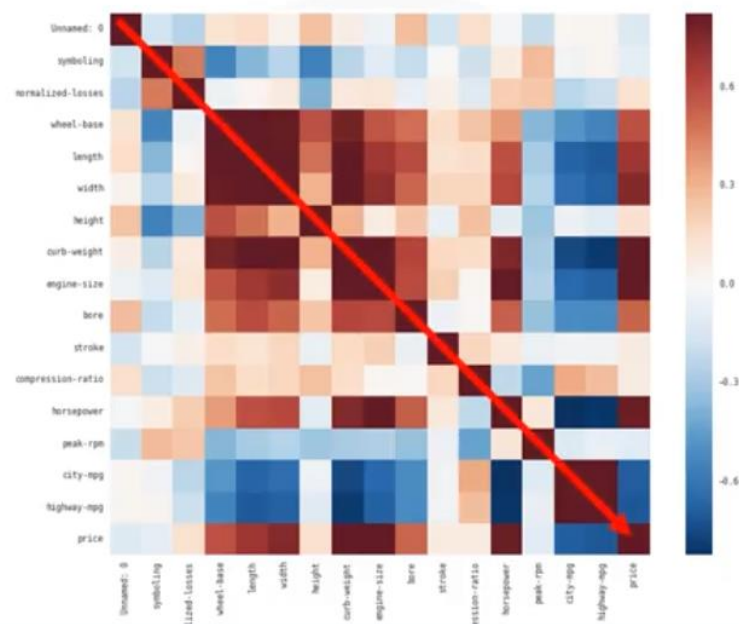
smaller than 0.001, and so we can conclude that we are certain about the strong positive correlation.

Pearson Correlation- Example

```
pearson_coef, p_value = stats.pearsonr(df['horsepower'], df['price'])
```

- Pearson correlation: 0.81
- P-value : 9.35 e-48

Correlation- Heatmap



Taking all variables into account, we can now create a heat map that indicates the correlation between each of the variables with one another. The **color** scheme **indicates** the **Pearson correlation coefficient**, indicating the strength of the correlation between two variables.

This correlation heat map gives us a good overview of how the different variables are related to one another, and most importantly, how these variables are related to price.

Chi-Square Test for Categorical Variables

Partimos de la tabla con el número de ocurrencias de los valores de la variables categoricas.

$$\chi^2 = \sum \frac{(O_i - E_i)^2}{E_i}$$

La O_i es el número de ocurrencias **observadas** (20, 30, 25, 25)

E_i es número de ocurrencias **esperadas** calculada para (like, male) como $(50 * 45 / 100)$

Category	Like	Dislike	Total
Male	20	30	50
Female	25	25	50
Total	45	55	100

$$E_{Male, Like} = \frac{(50 \times 45)}{100} = 22.5$$

$$E_{Male, Dislike} = \frac{(50 \times 55)}{100} = 27.5$$

$$E_{Female, Like} = \frac{(50 \times 45)}{100} = 22.5$$

$$E_{Female, Dislike} = \frac{(50 \times 55)}{100} = 27.5$$

Step 2: Compute Chi-Square Statistic

$$\chi^2 = \frac{(20 - 22.5)^2}{22.5} + \frac{(30 - 27.5)^2}{27.5} + \frac{(25 - 22.5)^2}{22.5} + \frac{(25 - 27.5)^2}{27.5}$$

$$\chi^2 = \frac{(2.5)^2}{22.5} + \frac{(2.5)^2}{27.5} + \frac{(2.5)^2}{22.5} + \frac{(2.5)^2}{27.5}$$

$$\chi^2 = \frac{6.25}{22.5} + \frac{6.25}{27.5} + \frac{6.25}{22.5} + \frac{6.25}{27.5}$$

$$\chi^2 = 0.277 + 0.227 + 0.277 + 0.227$$

$$\chi^2 = 1.008$$

Ese 1.008 lo comparamos con el valor que obtenemos de la table de distribución X-cuadrado para un determinado grado de Libertad y una determinada significancia.

Si ese 1.008 es **menor** que el valor obtenido de la tabla **aceptamos** la hipotesis nula que indica que no hay asociación entre las variables.

Si ese 1.008 es **mayor** que el valor obtenido de la tabla **rechazamos** la hipótesis nula, y nos quedamos con la hipótesis alternativa que indica que hay asociación entre las variables.

Null Hypothesis and Alternative Hypothesis

The chi-square test involves formulating two hypotheses:

Null Hypothesis (H_0)(H0) - Assumes that there is no association between the categorical variables, implying that any observed differences are due to random chance.

Alternative Hypothesis (H_1)(H1) - Assumes that there is a significant association between the variables, indicating that the observed differences are not due to chance alone.

▼ ▼

Lesson Summary

Congratulations! You have completed this lesson. At this point in the course, you know:

- Tools like the '**describe**' function in pandas can quickly calculate key statistical measures like mean, standard deviation, and quartiles for all numerical variables in your data frame.
- Use the '**value_counts**' function to summarize data into different categories for categorical data.
- Box plots offer a more visual representation of the data's distribution for numerical data, indicating features like the median, quartiles, and outliers.
- Scatter plots are excellent for exploring relationships between continuous variables, like engine size and price, in a car data set.
- Use Pandas' '**groupby**' method to explore relationships between categorical variables.
- Use pivot tables and heat maps for better data visualizations.
- Correlation between variables is a statistical measure that indicates how the changes in one variable might be associated with changes in another variable.
- When exploring correlation, use scatter plots combined with a regression line to visualize relationships between variables.
- Visualization functions like **regplot**, from the **seaborn** library, are especially useful for exploring correlation.

- The **Pearson correlation**, a key method for assessing the correlation between continuous numerical variables, provides two critical values—the coefficient, which indicates the strength and direction of the correlation, and the P-value, which assesses the certainty of the correlation.
- A correlation coefficient close to 1 or -1 indicates a strong positive or negative correlation, respectively, while one close to zero suggests no correlation.
- For P-values, values less than .001 indicate strong certainty in the correlation, while larger values indicate less certainty. Both the coefficient and P-value are important for confirming a strong correlation.
- Heatmaps provide a comprehensive visual summary of the strength and direction of correlations among multiple variables.

Labs

Creación de un sub dataframe en función de los tipos de datos

```
numeric_df = df.select_dtypes(include=['float64', 'int64'])
```

Esto nos asegura que la correlación va a trabajar solamente con datos numéricos.

Para la EDA (count, cuantos valores diferentes hay, top value, freq top value) de las variable categóricas

```
df.describe(include=['object'])
```

```
df.describe(include=['object'])
```

	make	aspiration	num-of-doors	body-style	drive-wheels	engine-location	engine-type	num-of-cylinders	fuel-system	horsepower-binned
count	201	201	201	201	201	201	201	201	201	200
unique	22	2	2	5	3	2	6	7	8	3
top	toyota	std	four	sedan	fwd	front	ohc	four	mpfi	Low
freq	32	165	115	94	118	198	145	157	92	115

Para transformar la salida de value counts en un dataframe y cambiar los nombres de índices y columnas.

```
df['drive-wheels'].value_counts().to_frame()
```

count	
drive-wheels	
fwd	118
rwd	75
4wd	8

Let's repeat the above steps but save the results to the dataframe "drive_wheels_counts" and rename the column 'drive-wheels' to 'value_counts'.

```
drive_wheels_counts = df['drive-wheels'].value_counts().to_frame()
drive_wheels_counts.reset_index(inplace=True)
drive_wheels_counts=drive_wheels_counts.rename(columns={'drive-wheels': 'value_counts'})
drive_wheels_counts
```

	value_counts	count
0	fwd	118
1	rwd	75
2	4wd	8

Now let's rename the index to 'drive-wheels':

```
drive_wheels_counts.index.name = 'drive-wheels'
drive_wheels_counts
```

	value_counts	count
drive-wheels		
0	fwd	118
1	rwd	75
2	4wd	8

La función de agregación de un groupby solamente puede ser invocada directamente sobre la variable no agrupada si todas las demas lo están:

En este df:

```
df_group_one = df[['drive-wheels','body-style','price']]
```

OK => grouped = df_group_one.groupby(['drive-wheels'], as_index=False).agg({'price': 'mean'})

OK => grouped = df_group_one.groupby(['drive-wheels','body-style'],as_index=False).mean()

KO => grouped = df_group_one.groupby(['drive-wheels'], as_index=False).mean()

Para etiquetar los valores de los ejes de un grafico pcolor de pyplot

Para invertir el orden de las etiquetas en un `sns.heatmap()` tengo que acceder al objeto `plt` subyacente de `matplotlib`

```
ax=sns.heatmap(data=grouped_pivot, cmap='RdBu')
ax.invert_yaxis()
```

Estas dos líneas equivalen a:

```
fig, ax = plt.subplots()
im = ax.pcolor(grouped_pivot, cmap='RdBu')

#label names
row_labels = grouped_pivot.columns.levels[1]
col_labels = grouped_pivot.index

#move ticks and labels to the center
ax.set_xticks(np.arange(grouped_pivot.shape[1]) + 0.5, minor=False)
ax.set_yticks(np.arange(grouped_pivot.shape[0]) + 0.5, minor=False)

#insert labels
ax.set_xticklabels(row_labels, minor=False)
ax.set_yticklabels(col_labels, minor=False)

#rotate label if too long
plt.xticks(rotation=90)

fig.colorbar(im)
plt.show()
```

Module 4: Model Development

Learning Objectives

In this module you will learn about:

1. Simple and Multiple Linear Regression
2. Model Evaluation using Visualization
3. Polynomial Regression and Pipelines
4. R-squared and MSE for In-Sample Evaluation
5. Prediction and Decision Making

- Question

- **How can you determine a fair value for a used car?**

Model Development

- A model can be thought of as a mathematical equation used to predict a value given one or more other values
- Relating one or more independent variables to dependent variables

independent variables
or features

'highway-mpg'

55 mpg



Model

Model Development

- Usually the more **relevant data** you have the more accurate your model is

'highway-mpg'
'curb-weight'
'engine-size'

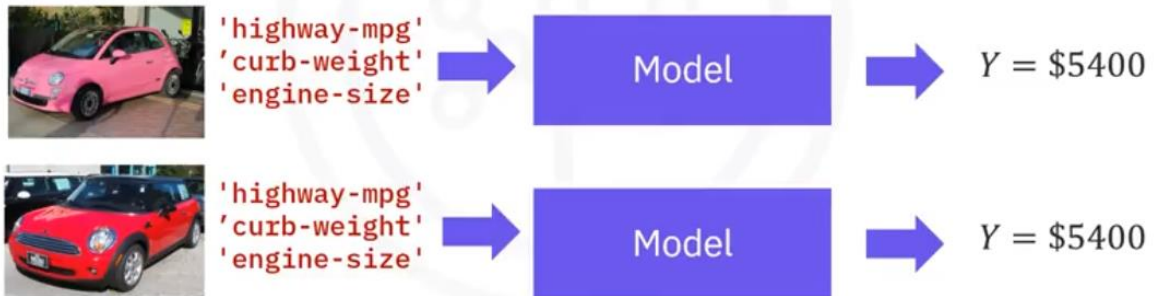


Model

Model Development

- To understand why more data is important, consider the following situation:

1. You have two almost identical cars
2. Pink cars sell for significantly less



A **model** or estimator can be thought of as a **mathematical equation** used to predict a value given one (simple) or more other values (Multiple linear), relating one or more independent variables or features to dependent variables.

Usually, the **more relevant data** you have, **the more accurate** your model is.

You have two almost identical cars, but pink cars sell for significantly less. You want to use your model to determine the price of two cars, one pink, one red. If your model's independent variables or features do not include color, your model will predict the same price for cars that may sell for much less.

Model Development

In addition to getting more data, you can try different types of models.

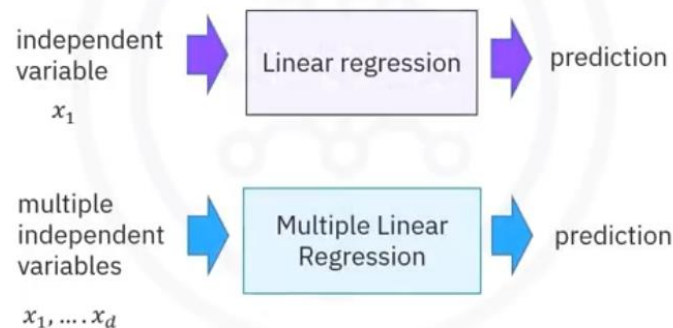
In this course you will learn about:

1. Simple Linear Regression
2. Multiple Linear Regression
3. Polynomial Regression

Linear Regression and Multiple Linear Regression

Introduction

- Linear regression will refer to one independent variable to make a prediction
- Multiple Linear Regression will refer to multiple independent variables to make a prediction



Simple linear regression, or SLR, is a method **to help us understand the relationship between two variables:**

- The predictor (independent variable x)
- The target (dependent variable y).

Simple linear regression

1. The predictor (independent) variable - x
2. The target (dependent) variable - y

$$y = b_0 + b_1 x$$

The equation $y = b_0 + b_1 x$ is shown. Below the term b_0 is a purple arrow pointing upwards, and below the term b_1 is another purple arrow pointing upwards.

- b_0 : the intercept
- b_1 : the slope

We would like to come up with a linear relationship between the variables shown here. $Y = b_0 + b_1 * x$

The parameter b_0 is the intercept,

the parameter b_1 is the slope.

When we fit or train the model we will come up with these parameters. This step requires lots of math, so we will not focus on this part.

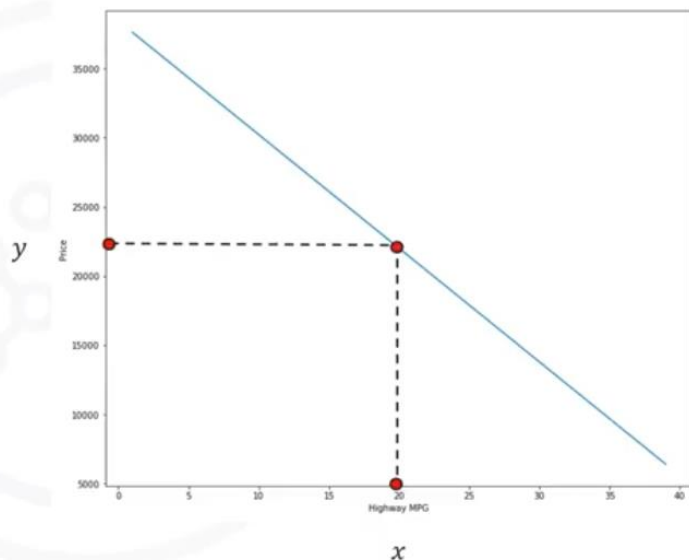
Let's clarify the prediction step:

It's hard to figure out how much a car costs.

The highway miles per gallon is in the owner's manual. If we assume there is a linear relationship between these variables, we can use this relationship to formulate a model to determine the price of the car.

Simple linear regression: Prediction

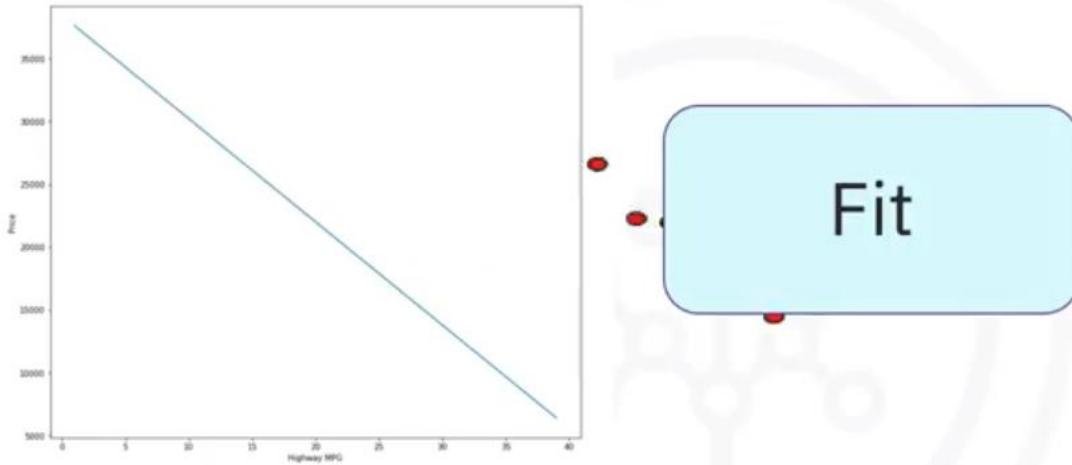
$$\begin{aligned} y &= 38423 - 821x \\ &= 38423 - 821(20) \\ &= 22\,003 \end{aligned}$$



If the highway miles per gallon is 20, we can input this value into the model to obtain a prediction of \$22,000.

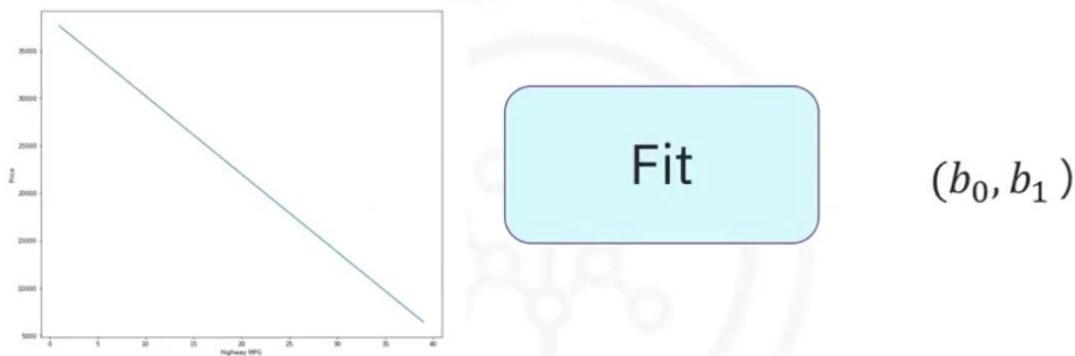
In order to determine the line, we take data points from our data set marked in red here.

Simple linear regression: Fit



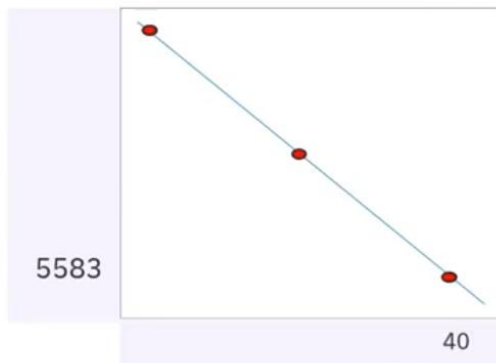
We then use these training points to **fit** our model. The **results** of the training points are the **parameters**.

Simple linear regression: Fit



We usually store the data points in **two data frame or NumPy arrays**. The value we would like to predict is called the target that we store in the array y . We store the dependent variable in the data frame or array x .

Simple linear regression: Fit



$$X = \begin{bmatrix} 0 \\ 20 \end{bmatrix}$$

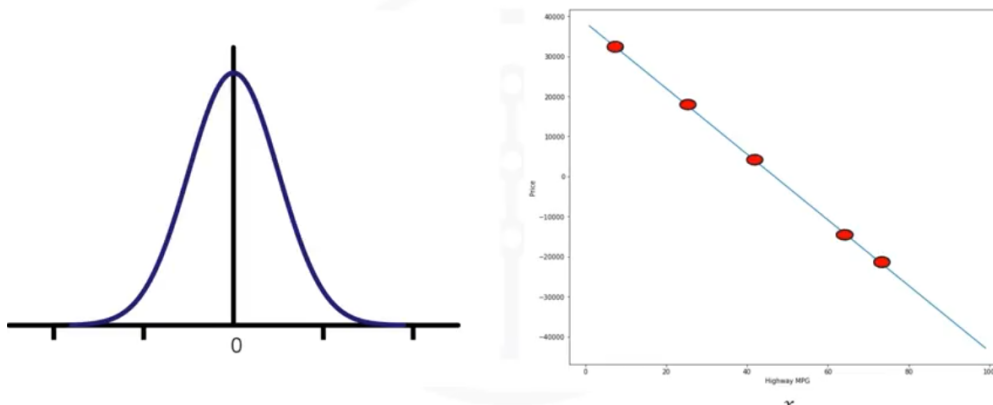
$$Y = \begin{bmatrix} 38423 \\ 22003 \end{bmatrix}$$

Each sample corresponds to a **different row in each data frame or array**.

In many cases, **many factors influence** how much people paid for a car, for example, make or how old the car is.

In this model, this uncertainty is taken into account by assuming a small random value is added to the point on the line. This is called **noise**.

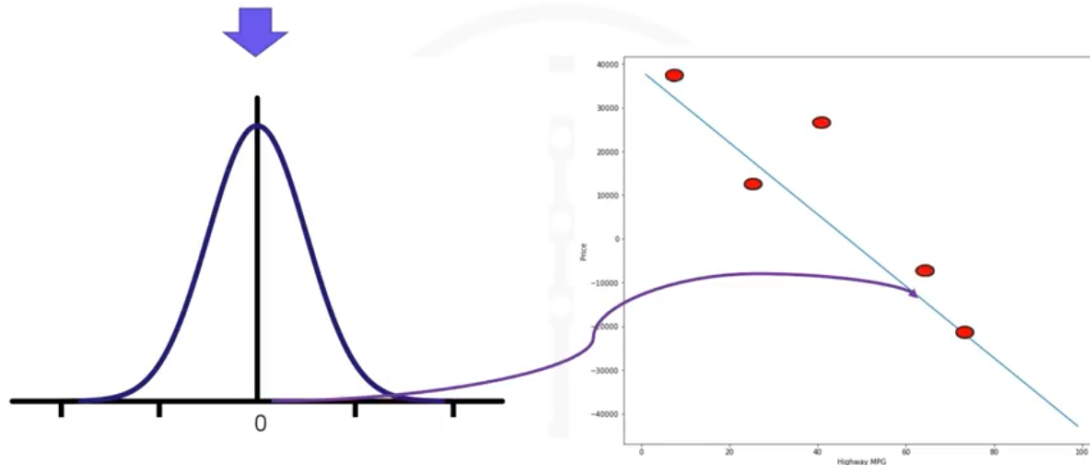
Simple linear regression: Fit



The figure on the left shows the distribution of the noise. The vertical axis shows the value added and the horizontal axis illustrates the probability that the value will be added.

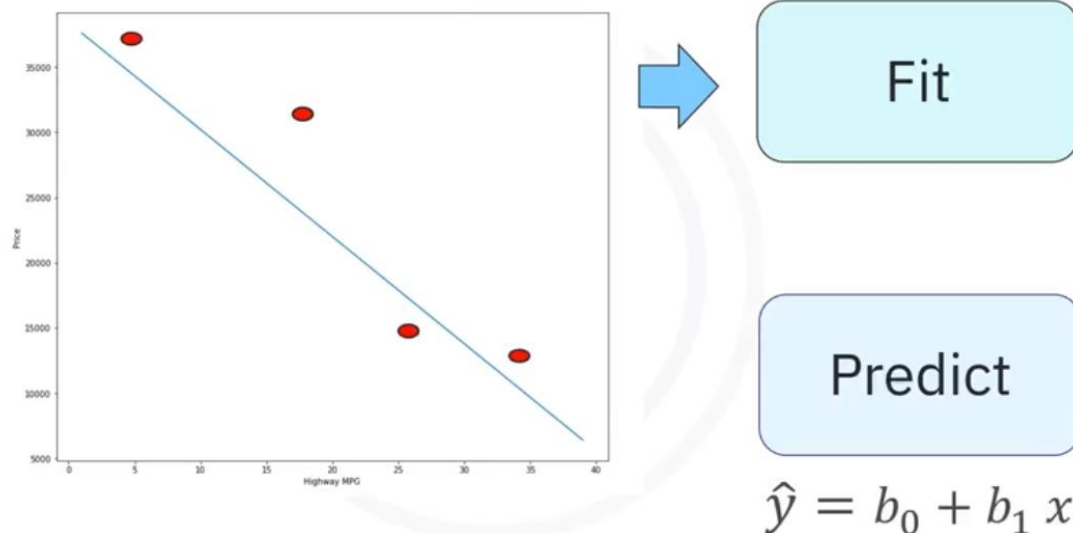
Usually, a small positive value is added or a small negative value. Sometimes large values are added, but for the most part, the values added are near zero.

Simple linear regression: Fit



We can summarize the process like this. We have a set of training points. We use these training points to fit or train the model and get parameters. We then use these parameters in the model, we now have a model.

Simple linear regression

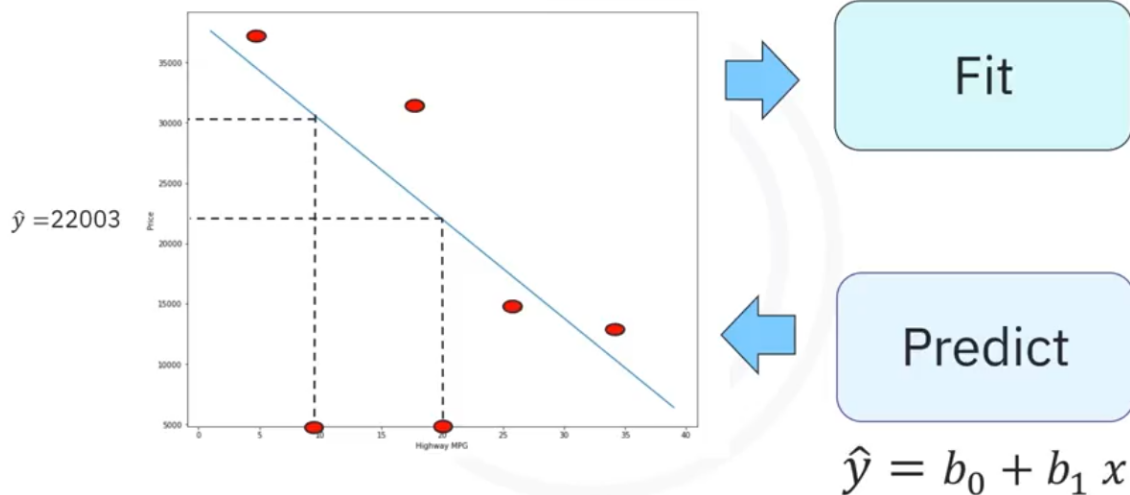


We use the **hat** on the y to **denote** the model is an **estimate**.

We can use this model to **predict values that we haven't seen**, for example, we have no car with 20 highway miles per gallon.

We can use our model to make a prediction for the price of this car but don't forget our model is not always correct.

Simple linear regression



We can see this by **comparing the predicted value to the actual value**. We have a sample for ten highway miles per gallon, but the predicted value does not match the actual value. If the linear assumption is correct, this error is due to the noise but there can be other reasons.

To fit the model in Python, first we import linear model from scikit-learn. Then create a linear regression object using the constructor.

Fitting a simple linear model estimator

- X : Predictor variable
- Y: Target variable

1. Import linear_model from scikit-learn

```
from sklearn.linear_model import LinearRegression
```

1. Create a Linear Regression Object using the constructor:

```
lm=LinearRegression()
```

We define the predictor variable and target variable. Then use the method fit to fit the model and find the parameters b_0 and b_1 , the input are the features and the targets. We can obtain a prediction using the method predict.

Fitting a simple linear model

- We define the predictor variable and target variable

```
X = df[['highway-mpg']]  
Y = df['price']
```

- Then use `lm.fit(X, Y)` to fit the model, i.e find the parameters b_0 and b_1

```
lm.fit(X, Y)
```

- We can obtain a prediction

```
Yhat=lm.predict(X)
```

Yhat	X
2	5
:	
3	4

The **output** is an array. The array has the **same number of samples as the input x**.

The intercept b_0 is an **attribute of the object** `lm`, the slope b_1 is also an **attribute of the object** `lm`. The relationship between price and highway miles per gallon is given by this equation in bold:

SLR - Estimated linear model

- We can view the intercept (b_0): `lm.intercept_`
38423.305858
- We can also view the slope (b_1): `lm.coef_`
-821.73337832
- The Relationship between Price and Highway MPG is given by:
- **Price = 38423.31 - 821.73 * highway-mpg**


$$\hat{Y} = b_0 + b_1x$$

price = 38,423.31 - 821.73 times highway miles per gallon, like the equation we discussed before.

Multiple linear regression is used to explain the relationship between one continuous target (y) variable and two or more predictor (x) variables.

Multiple Linear Regression (MLR)

This method is used to explain the relationship between:

- One continuous target (Y) variable
- Two or more predictor (X) variables

If we have, for example, four predictor variables, then b_0 intercept $x=0$, b_1 , the coefficient or parameter of x_1 , b_2 , the coefficient of parameter x_2 , and so on.

Multiple Linear Regression (MLR)

$$\hat{Y} = b_0 + b_1x_1 + b_2x_2 + b_3x_3 + b_4x_4$$

- b_0 : **intercept** ($X=0$)
- b_1 : the **coefficient** or **parameter** of x_1
- b_2 : the **coefficient** of **parameter** x_2 and so on..

If there are only two variables, then we can visualize the values. Consider the following function the variables x_1 and x_2 can be visualized on a 2D plane, let's do an example on the next slide.

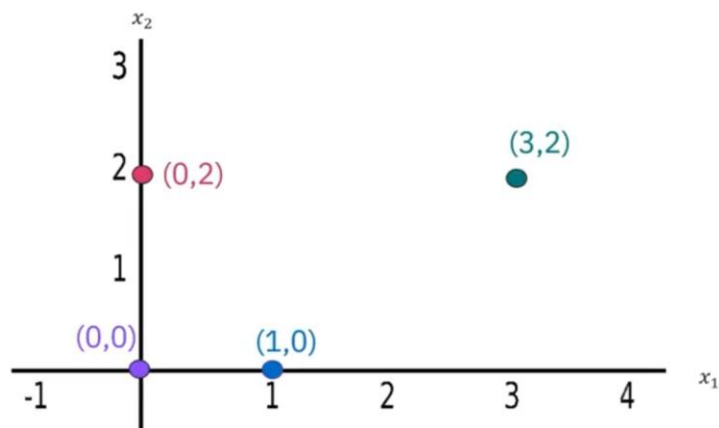
Multiple Linear Regression (MLR)

$$\hat{Y} = 1 + 2x_1 + 3x_2$$

- The variables x_1 and x_2 can be visualized on a 2D plane, let's do an example on the next slide

The table contains different values of the predictor variables x_1 and x_2 . The position of each point is placed on the 2D plane, color coded accordingly.

n	x_1	x_2
1	0	0
2	0	2
3	1	0
4	3	2

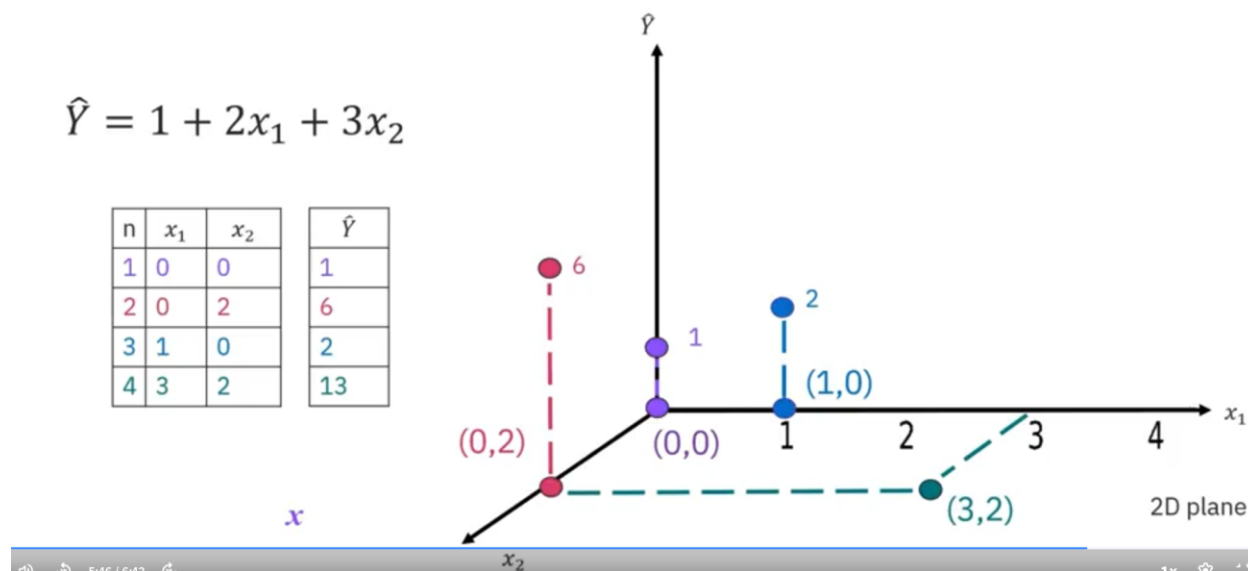


Each value of the predictor variables x_1 and x_2 will be mapped to a new value y , $\hat{y}$. The new values of y , $\hat{y}$, are mapped in the vertical direction with height proportional to the value that $\hat{y}$ takes.

- This is shown below where

$$\hat{Y} = 1 + 2x_1 + 3x_2$$

n	x_1	x_2	$\hat{Y}$
1	0	0	1
2	0	2	6
3	1	0	2
4	3	2	13



We can fit the multiple linear regression as follows. We can extract the four predictor variables and store them in the variable z then train the model as before using the method `fit` or dependent variables and the targets column. We can also obtain a prediction using the method `predict`.

Fitting a multiple linear model estimator

1. We can extract the for 4 predictor variables and store them in the variable Z

```
z = df[['horsepower', 'curb-weight', 'engine-size', 'highway-mpg']]
```

2. Then train the model as before:

```
lm.fit(Z, df['price'])
```

3. We can also obtain a prediction

```
Yhat=lm.predict(X)
```

x_1	x_2	x_3	x_4	Yhat
3	5	-4	3	2
:	:	:	:	:
2	4	2	-4	3

In this case, the input is an Array or data frame with four columns. The number of rows corresponds to the number of samples. The output is an array with the same number of elements as number of samples.

The intercept is an attribute of the object and the coefficients are also attributes. It is helpful to visualize the equation replacing the independent variable names with actual names.

MLR – Estimated linear model

1. Find the intercept (b_0)

```
lm.intercept_  
-15678.742628061467
```

2. Find the coefficients (b_1, b_2, b_3, b_4)

```
lm.coef_  
array([52.65851272, 4.69878948, 81.95906216, 33.58258185])
```

The Estimated Linear Model:

- **Price** = -15678.74 + (52.66) * **horsepower** + (4.70) * **curb-weight** + (81.96) * **engine-size** + (33.58) * **highway-mpg**

$$\hat{Y} = b_0 + b_1x_1 + b_2x_2 + b_3x_3 + b_4x_4$$

Model Evaluation Using Visualization

Regression Plot

Comparamos la distribución de los valores predichos con los valores reales.

Regression plot

Why use regression plot?

It gives us a good estimate of:

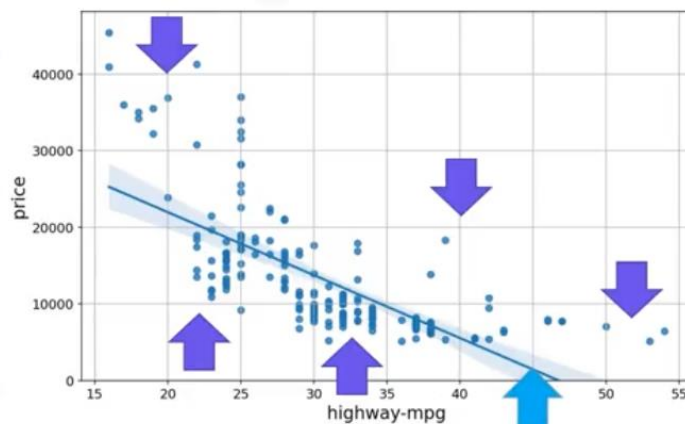
- The relationship between two variables
- The strength of the correlation
- The direction of the relationship (positive or negative)

The **horizontal** axis is the **independent** variable. The **vertical** axis is the **dependent** variable.

Regression plot

Regression Plot shows us a combination of:

- The scatterplot: where each point represents a different y
- The fitted linear regression line ($\hat{y}$)

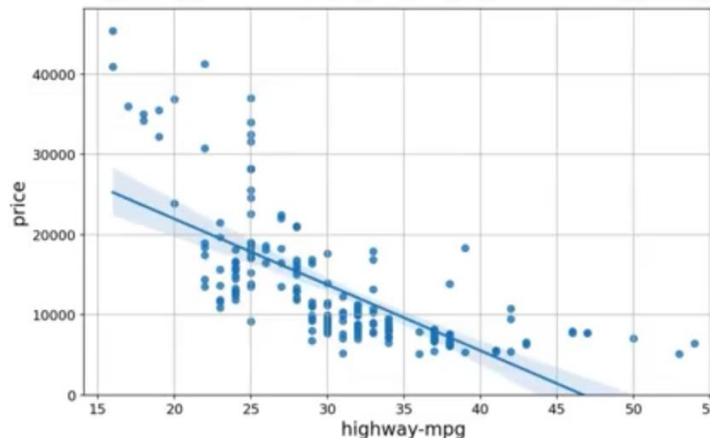


Each point represents a different target point. The fitted line represents the predicted value.

Regression plot

```
import seaborn as sns
```

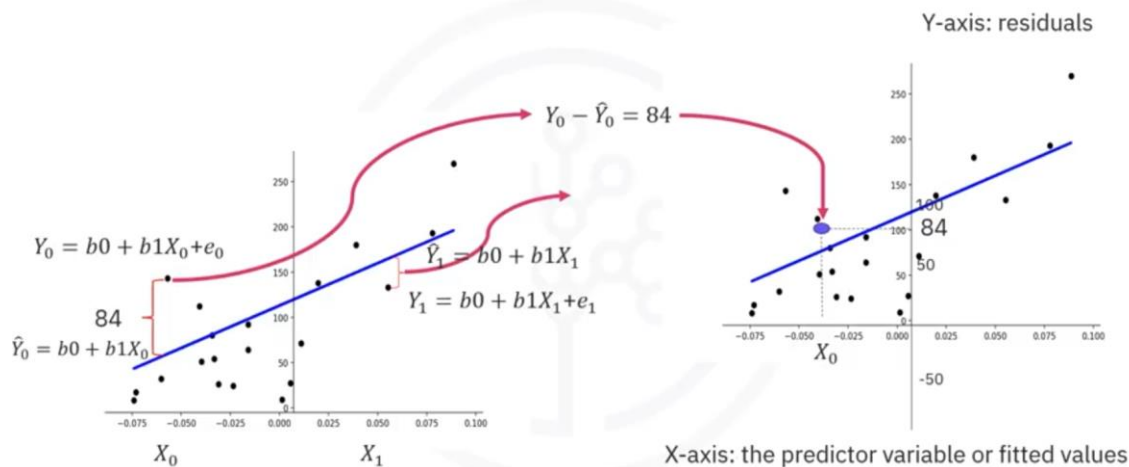
```
sns.regplot(x="highway-mpg", y="price", data=df)  
plt.ylim(0,)
```



Residual Plot

The residual plot represents the **error between** the actual value.

Residual plot

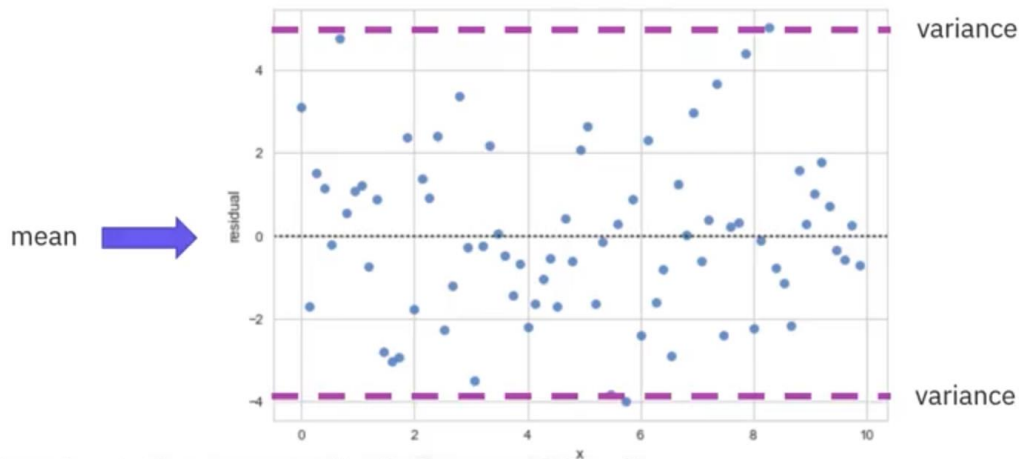


Examining the predicted value and actual value, we see a **difference**. We obtained that value by subtracting the **predicted** value and the actual **target** value. We then plot that value on the vertical axis **with the dependent variable as the horizontal axis**. Similarly, for the

second sample, we repeat the process, subtracting the target value from the predicted value, then plotting the value accordingly. Looking at the **plot gives us some insight into** our data.

We expect to see the results to have **zero mean** distributed evenly around the x-axis with similar variance. There is no curvature. This type of residual plot suggests a linear plot is appropriate.

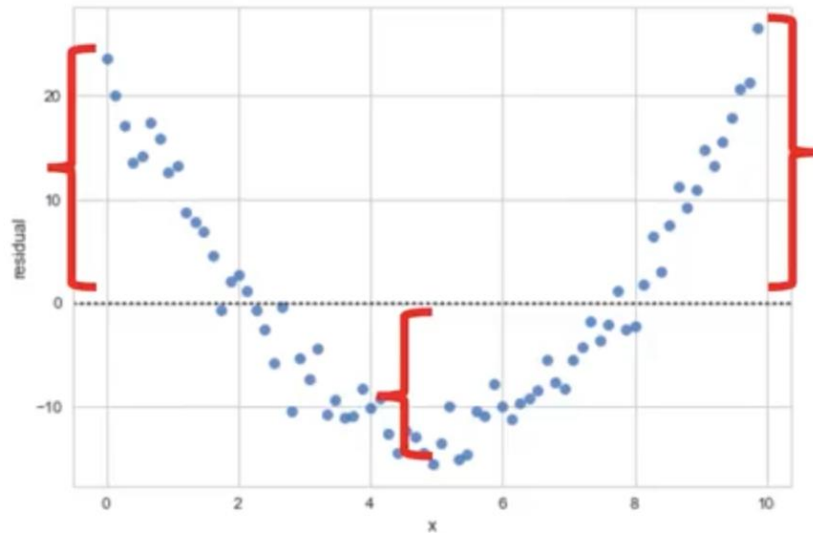
Residual plot



- Look at the **spread of the residuals**:
 - Randomly spread out around x-axis then a linear model is appropriate

In this residual plot, there is a **curvature**. The values of the error change with x.

Residual plot

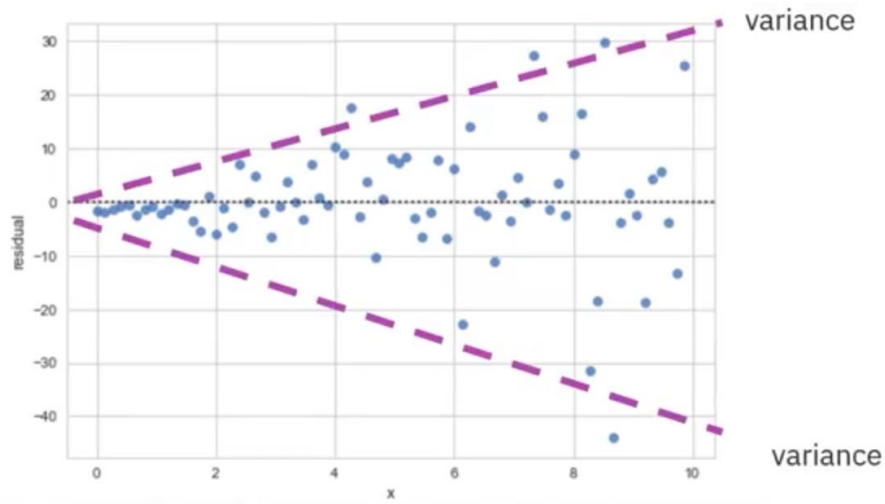


- Not randomly spread out around the x-axis
- Nonlinear model may be more appropriate

For example, in the region all the residual errors are positive. In this area, the residuals are negative. In the final location, the error is large. The residuals are not randomly separated. This suggests **the linear assumption is incorrect**. This plot suggests a nonlinear function.

In this plot, we see that variance of the residuals **increases** with x. Therefore, our **model is incorrect**.

Residual plot



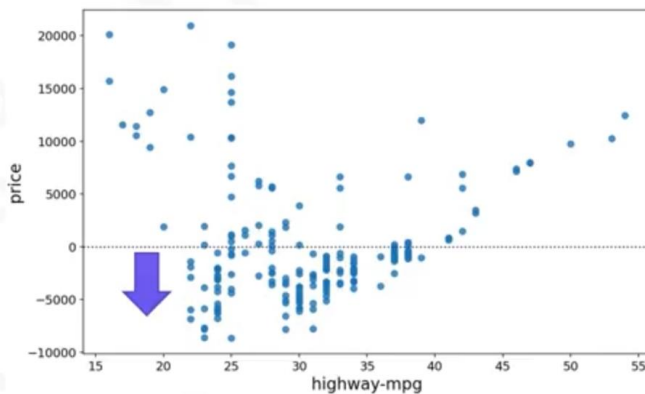
- Not randomly spread out around the x-axis
- Variance appears to change with x axis

We can use Seaborn to create a residual plot.

Residual plot

```
import seaborn as sns
```

```
sns.residplot(df['highway-mpg'],  
df['price'])
```

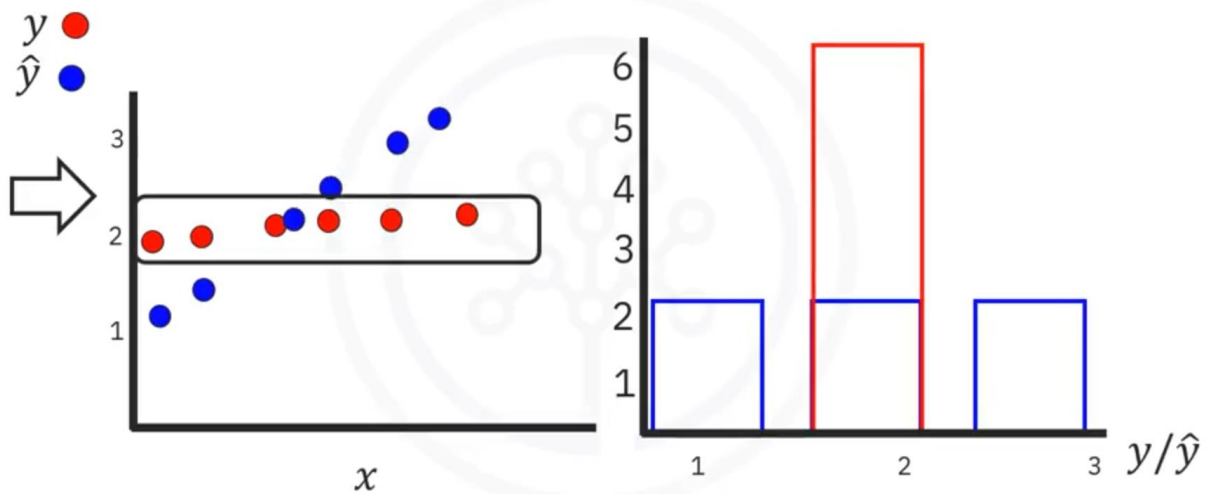


First, import Seaborn, we use the `residplot` function. The first parameter is a series of independent variable or feature. The second parameter is a series of dependent variable or target. We see in this case the residuals have a **curvature**.

Distribution Plot

A distribution plot **counts** the **predicted value versus the actual value**.

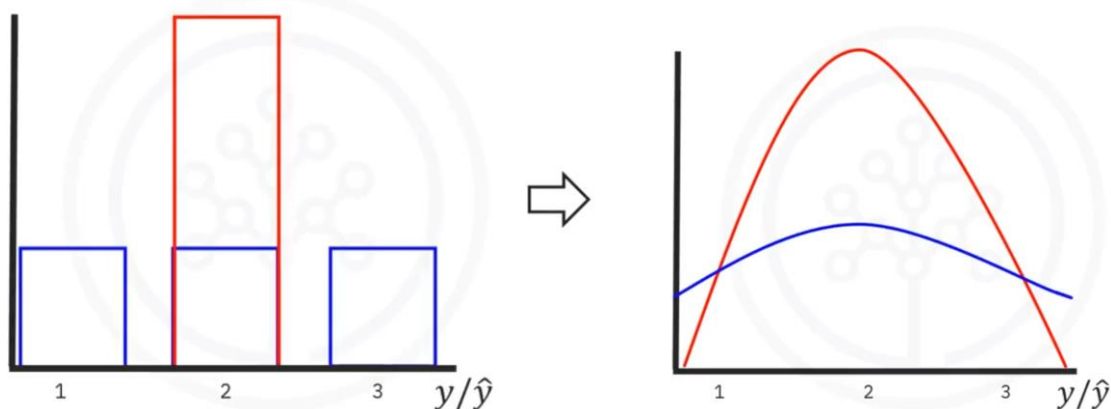
Distribution plots



These plots are extremely useful for **visualizing models with more than one independent variable** or feature.

Let's look at a simplified example. We examine the vertical axis. We then count and plot the number of predicted points that are approximately equal to one. We then count and plot the number of predicted points that are approximately equal to two. We repeat the process for predicted points that are approximately equal to three. Then we repeat the process for the target values. In this case, all the target values are approximately equal to two. The values are the targets and predicted values are continuous.

A histogram is for discrete values.



Therefore Pandas will convert them to a distribution. **The vertical axis is scaled to make the area under the distribution equal to one.**

This is an example of using a distribution plot.

Distribution plots

Compare the distribution plots:

- The fitted values that result from the model
- The actual values



The dependent variable or feature is price. The fitted values that result from the model are in blue. The actual values are in red. We see the predicted values for prices in the range from **40,000 to 50,000 are inaccurate**. The prices in the region from 10,000 to 20,000 are much closer to the target value.

In this example, we use multiple features or independent variables comparing it to the plot on the last slide. We see predicted values are much closer to the target values.

MLR: Distribution plots



Here is the code to create a distribution plot.

Distribution plots

```
import seaborn as sns
```

```
ax1 = sns.distplot(df['price'], hist=False, color="r", label="Actual Value")
```

```
sns.distplot(Yhat, hist=False, color="b", label="Fitted Values" , ax=ax1)
```

The actual values are used as a parameter. We want a distribution instead of a histogram. So we want the hist parameters set to false. The color is red. The label is also included. The predicted values are included for the second plot.

Polynomial regression and pipelines

Polynomial Regression

What do we do when a linear model is not the best fit for our data?

We transform our data into a polynomial, then use **linear regression regression** to fit the parameter.

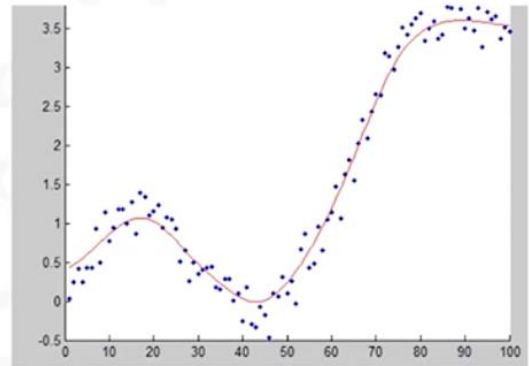
Polynomial regression is a **special case** of the general **linear** regression

Polynomial Regression

- A special case of the general linear regression model
- Useful for describing curvilinear relationships

Curvilinear relationships:

By squaring or setting higher-order terms of the predictor variables



This method is beneficial for describing curvilinear relationships. What is a curvilinear relationship? It's what you get by squaring or **setting higher-order terms of the predictor variables in the model transforming the data.**

Polynomial Regression

- Quadratic – 2nd order

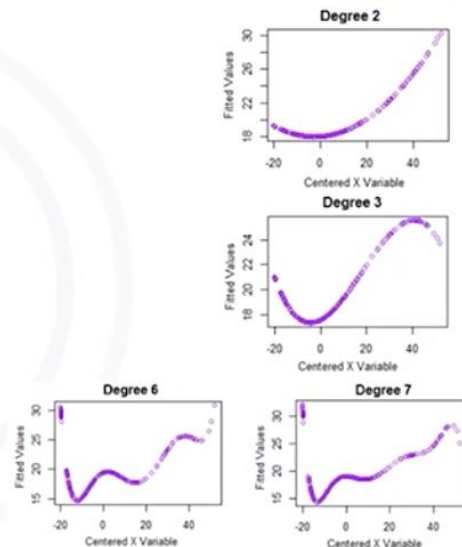
$$\hat{Y} = b_0 + b_1 x_1 + b_2 (x_1)^2$$

- Cubic – 3rd order

$$\hat{Y} = b_0 + b_1 x_1 + b_2 (x_1)^2 + b_3 (x_1)^3$$

- Higher order

$$\hat{Y} = b_0 + b_1 x_1 + b_2 (x_1)^2 + b_3 (x_1)^3 + \dots$$



The model can be **quadratic**, which means that the **predictor variable** in the model is **squared**. We use a bracket to indicate it as an exponent. This is a **2nd order polynomial regression** with a figure representing the function.

The model can be **cubic**, which means that the **predictor variable is cubed**. This is a **3rd order polynomial regression**. We see by examining the figure that the function has more variation.

There also exists **higher-order** polynomial regressions when a good fit hasn't been achieved by 2nd or 3rd order.

We can see in figures how much the graphs change when we change the order of the polynomial regression. The degree of the regression makes a big difference and can result in a better fit if you pick the right value.

In all cases, the relationship between the variable and the parameter is always linear.

Polynomial Regression

1. Calculate Polynomial of 3rd order

```
f=np.polyfit(x,y,3)
```

```
p=np.poly1d(f)
```

2. We can print out the model

```
print (p)
```

$$-1.557(x_1)^3 + 204.8(x_1)^2 + 8965x_1 + 1.37 \times 10^5$$

In Python, we do this by using the **polyfit** function. In this example, we develop a 3rd order polynomial regression model base. We can print out the model. The symbolic form for the model is given by the following expression:

-1.557 (x₁) cubed + 204.8 (x₁) squared + 8965 x₁ + 1.37 times 10 to the power of 5.

Polynomial Regression with more than one dimension

- We can also have multi dimensional polynomial linear regression

$$\hat{Y} = b_0 + b_1 X_1 + b_2 X_2 + b_3 X_1 X_2 + b_4 (X_1)^2 + b_5 (X_2)^2 + \dots$$

Let's look at an example from our data where we generate a polynomial regression model.

We can also have multidimensional polynomial linear regression. The expression can get complicated. Here are just some of the terms for a two-dimensional 2nd order polynomial. NumPy's **polyfit function cannot perform this type of regression.**

Polynomial Regression with more than one dimension

- The "preprocessing" library in scikit-learn:

```
from sklearn.preprocessing import PolynomialFeatures
pr=PolynomialFeatures(degree=2, include_bias=False)
x_polly=pr.fit_transform(x[['horsepower', 'curb-weight']])
```

We use the preprocessing library in scikit-learn to create a polynomial feature object. The constructor **takes the degree of the polynomial as a parameter**. Then we transform the features into a polynomial feature with the fit_transform method.

Polynomial Regression with more than one dimension

```
pr=PolynomialFeatures(degree=2)
```

X_1	X_2
1	2

```
pr.fit_transform([1,2], include_bias=False)
```



X_1	X_2	X_1X_2	X_1^2	X_2^2
1	2	(1) 2	1	(2) ²

Let's do a more intuitive example. Consider the features shown here. Applying the method, we transform the data. We now have a new set of features that are a transformed version of our original features.

Pre-processing

Pre-processing

- For example we can Normalize the each feature simultaneously:

```
from sklearn.preprocessing import StandardScaler  
  
SCALE=StandardScaler()  
SCALE.fit(x_data[['horsepower', 'highway-mpg']])  
  
x_scale=SCALE.transform(x_data[['horsepower', 'highway-mpg']])
```

As the dimension of the data gets larger, we may want to normalize multiple features in scikit-learn. Instead, we can use the preprocessing module to simplify many tasks. For example, we can standardize each feature simultaneously. We import standard scalar. We train the object, fit the scale object, then transform the data into a new data frame on array `x_scale`. There are more normalization methods available in the preprocessing library as well as other transformations.

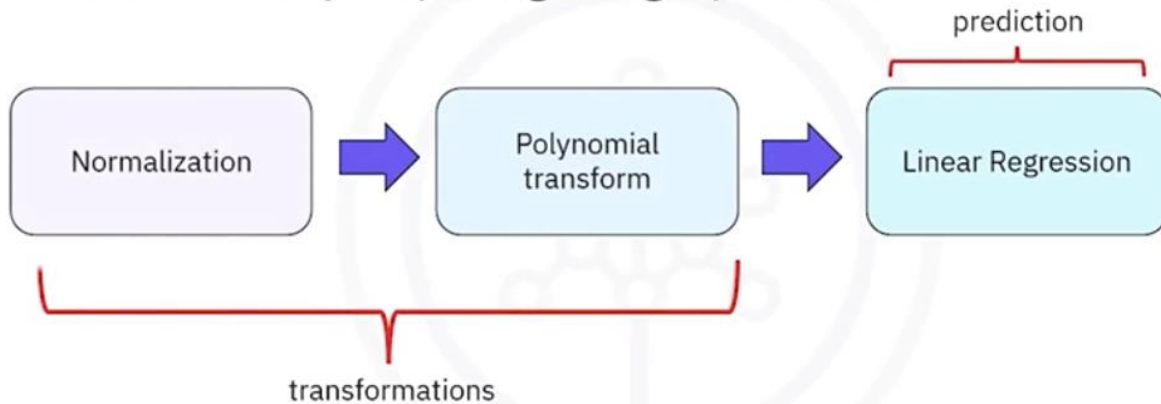
Pipelines

We can simplify our code by using a pipeline library.

There are many steps to getting a prediction. For example, polynomial transform, normalization, and linear regression.

Pipelines

- There are many steps to getting a prediction



We simplify the process using a pipeline. Pipelines sequentially perform a series of transformations. The last step carries out a prediction.

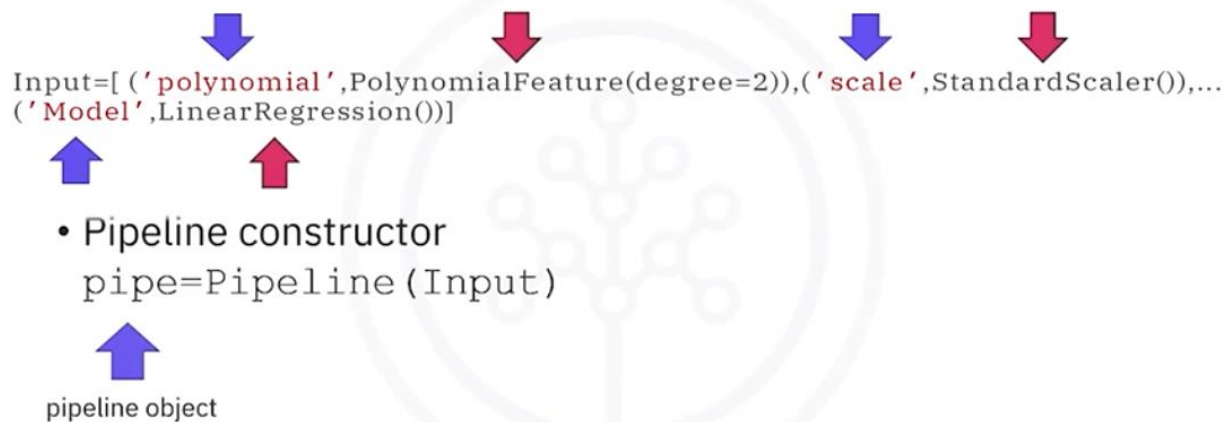
First, we import all the modules we need. Then we import the library pipeline.

Pipelines

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
```

We create a list of **tuples**.

Pipeline constructor



The first element in the tuple contains the **name of the estimator, model**. The second element contains **model constructor**.

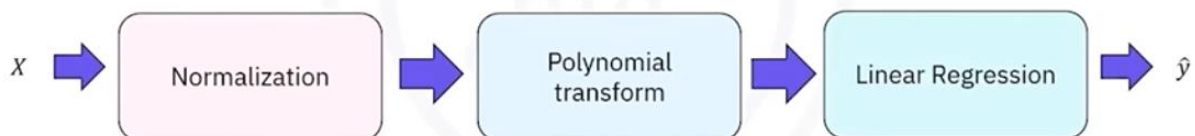
We input the list in the pipeline constructor. We now have a pipeline object.

Pipeline constructor

- We can train the pipeline object

```
Pipe.fit(df[['horsepower', 'curb-weight', 'engine-size', 'highway-mpg']], y)

yhat=Pipe.predict(X[['horsepower', 'curb-weight', 'engine-size', 'highway-mpg']])
```



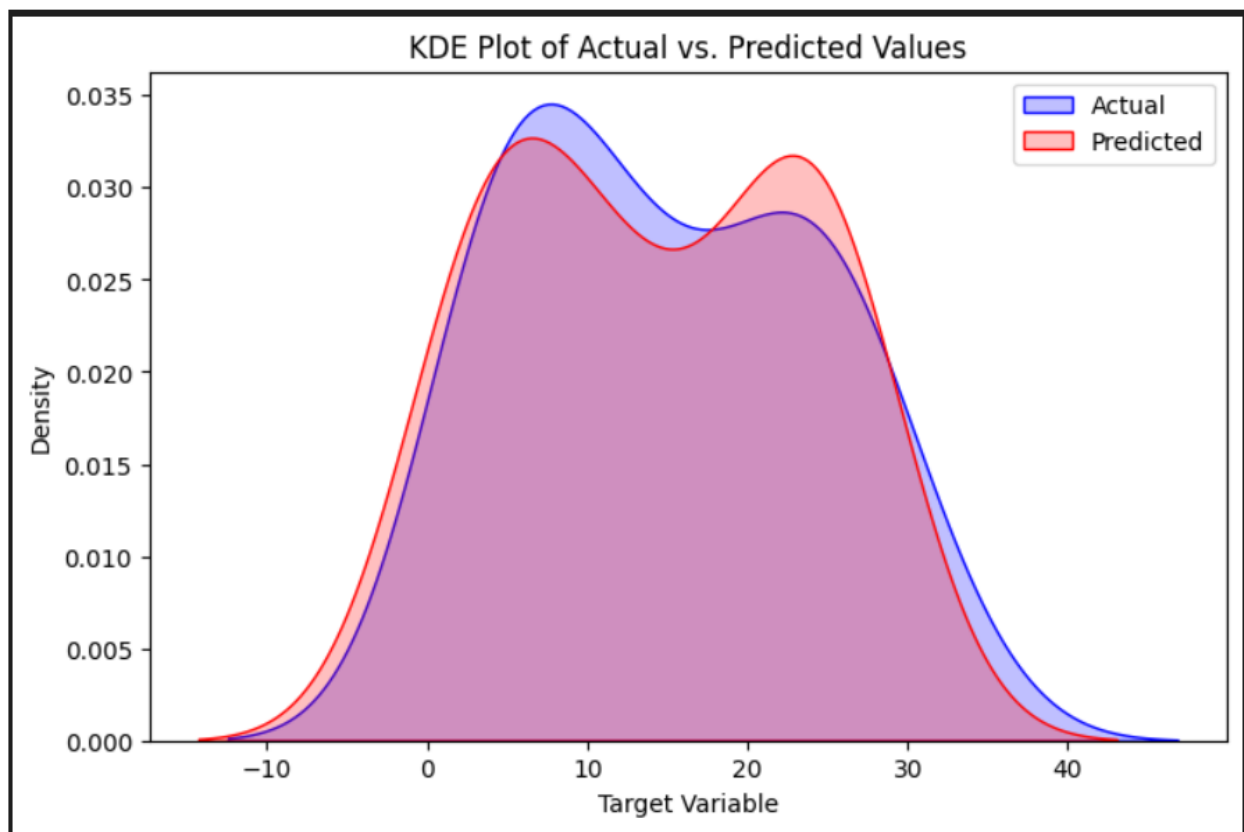
We can train the pipeline by applying the train method to the pipeline object. We can also produce a prediction as well. The method normalizes the data, performs a polynomial transform, then outputs a prediction.

Kernel Density Estimation (KDE) plots for model evaluation.

With the deprecation of Seaborn `distplot()`, KDE plots serve as a modern and effective method for assessing model performance.

- They provide a smooth approximation of the data distribution.
- They help compare the true vs. predicted distributions effectively.
- Unlike histograms, KDE plots are **not sensitive to bin sizes**.
- They **can highlight deviations** between observed and predicted values.

```
# Plotting KDE for Observed vs. Predicted Values
plt.figure(figsize=(8, 5))
sns.kdeplot(y_test, label='Actual', fill=True, color='blue')
sns.kdeplot(y_pred, label='Predicted', fill=True, color='red')
plt.xlabel('Target Variable')
plt.ylabel('Density')
plt.title('KDE Plot of Actual vs. Predicted Values')
plt.legend()
plt.show()
```



Interpretation of the KDE Plot

- **Overlap Between Distributions:** The two curves have a significant overlap, indicating that the model has captured the general distribution of the actual target values reasonably well. However, the predicted values slightly deviate from the actual values in some regions.
- **Peak Differences (Mode Shifts):** The blue (actual) curve peaks slightly higher than the red curve, meaning that the actual values are more concentrated around certain values. The red (predicted) curve has a second peak, suggesting that the model may be slightly misestimating certain ranges of the target variable.
- **Spread of the Distributions:** The actual values (blue) seem to have a wider spread, indicating more variation in real-world values. The predicted values (red) appear to be narrower, which suggests the model might be slightly underestimating variance (a sign of over-smoothing or bias).
- **Tails of the Distributions:** The tails of the predicted values closely follow the actual values, meaning the model does not generate extreme outliers beyond what was observed in the data. If there was a significant mismatch in the tails, it could indicate that the model struggles with extreme cases.

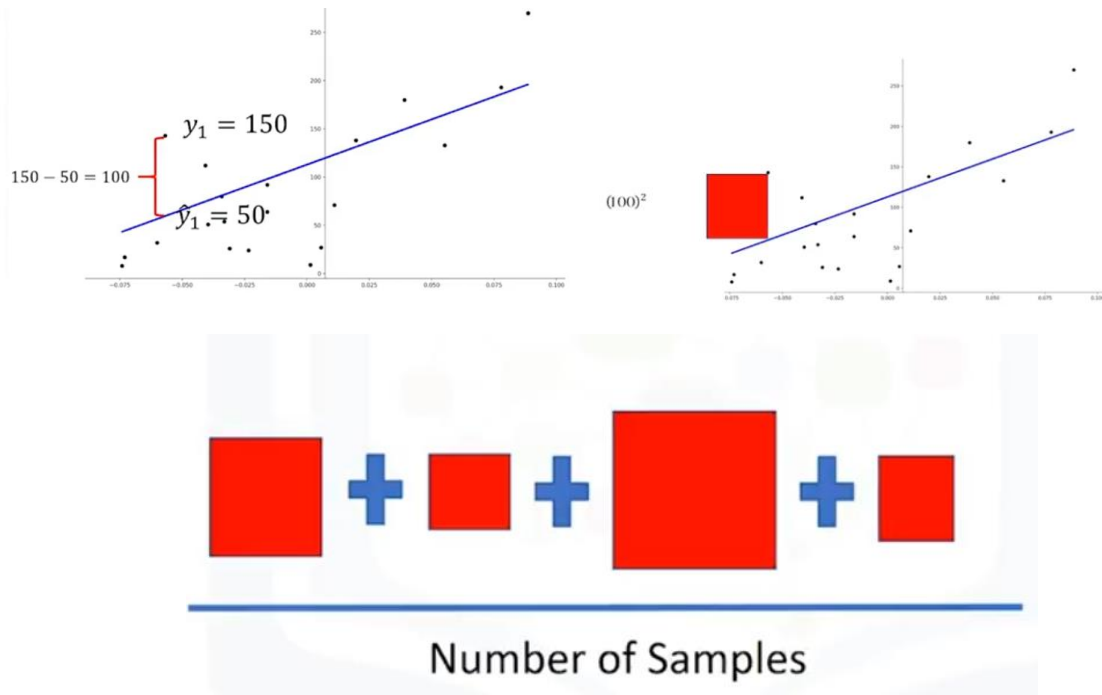
Measures for in sample Evaluation

We want to numerically evaluate our models.

Measures for In-Sample Evaluation

- A way to numerically determine how good the model fits on dataset.
- Two important measures to determine the fit of a model:
 - **Mean Squared Error (MSE)**
 - **R-squared (R^2)**

Mean Squared Error



To measure the MSE, we find the **difference** between the **actual** value y and the **predicted** value $\hat{y}$ then square it. In this case, the actual value is 150. The predicted value is 50. Subtracting these points, we get 100. We then square the number. We then take the mean or **average of all** the errors by adding them all together and dividing by the number of samples.

To find the MSE in Python, we can import the `mean_squared_error` from `sklearn.metrics`. The `mean_squared_error` function gets two inputs, the actual value of target variable and the predicted value of target variable.

```
from sklearn.metrics import mean_squared_error

mean_squared_error(df['price'], Y_predict_simple_fit)

3163502.944639888
```

R-Squared

R-squared/ R^2

- The Coefficient of Determination or R squared (R^2)
- Is a measure to determine how close the data is to the fitted regression line.
- R^2 : the percentage of variation of the target variable (Y) that is explained by the linear model.
- Think about as comparing a regression model to a simple model i.e the mean of the data points

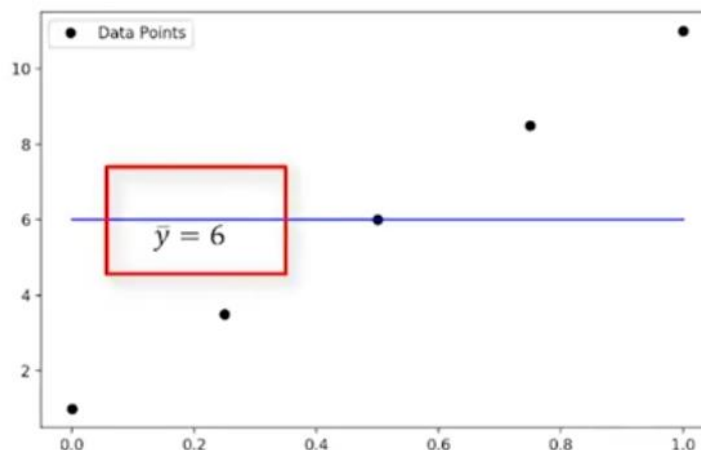
R squared is also called the coefficient of determination. It's a measure to determine **how close the data is to the fitted regression line**.

So how close is our actual data to our estimated model?

Think about it as **comparing a regression model to a simple model**, i.e., **the mean of the data points**.

If the variable x is a good predictor, our model should perform much better than just the mean.

- In this example the average of the data points $\bar{y}$ is 6

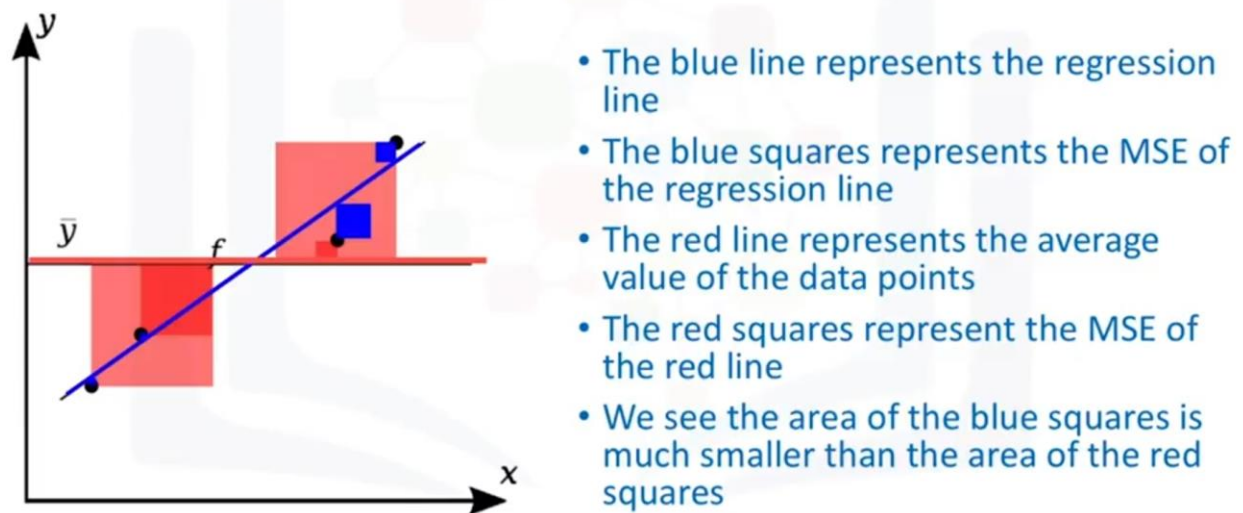


In this example, the average of the data points $\bar{y}$ is six.

$$R^2 = \left(1 - \frac{\text{MSE of regression line}}{\text{MSE of the average of the data}} \right)$$

Coefficient of determination R squared is one minus the ratio of the MSE of the **regression** line **divided** by the MSE of the **average** of the data points. For the most part, it takes values between zero and one.

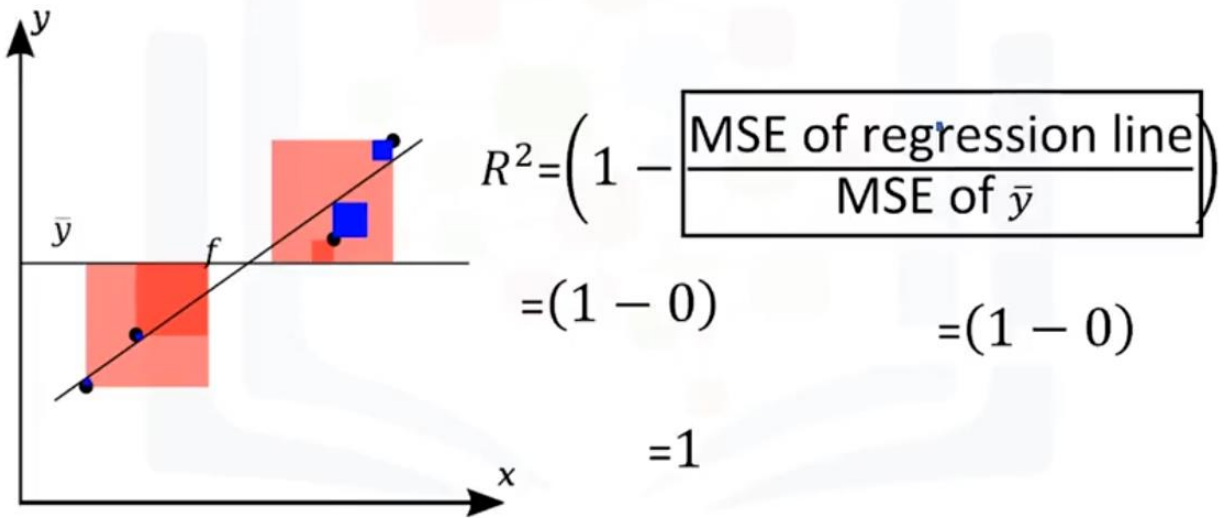
Let's look at a case where the line provides a relatively good fit.



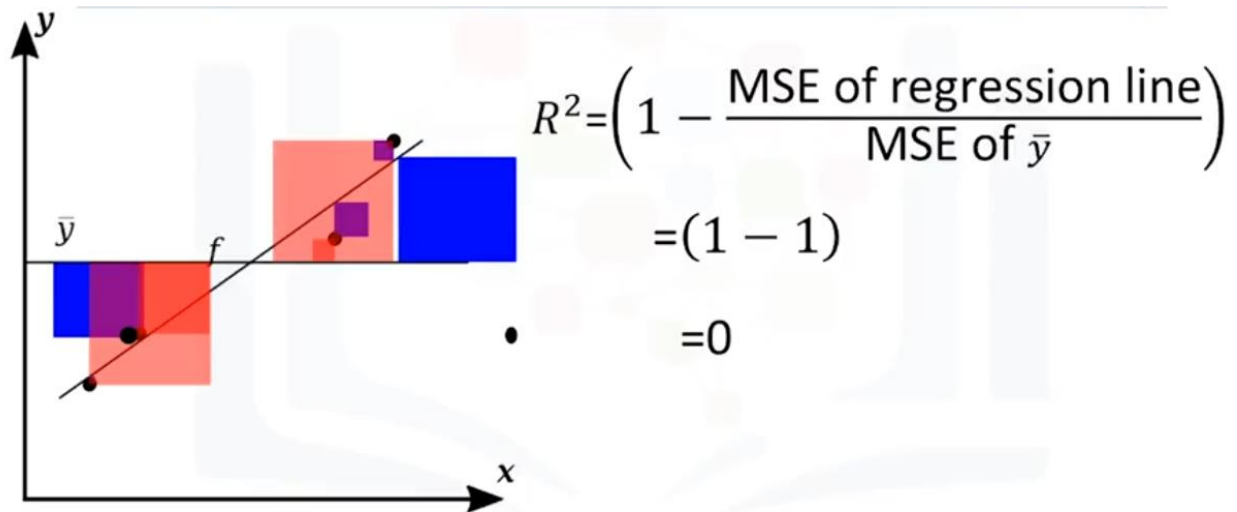
In this case, because the line **is a good fit**, the mean squared error is small. Therefore, the numerator is small. The mean squared error of the average of the data is large as the denominator is large. A small number divided by a larger number is an even smaller number. Taken to an extreme, this value tends to zero. If we plug in this value from the previous slide for R squared, we get a **value near one**. This means the line is a good fit for the data.

- In this case ratio of the areas of MSE is close to zero

$$\frac{\text{MSE of regression line}}{\text{MSE of } \bar{y}} = \frac{\text{small blue squares} + \text{plus sign}}{\text{large red squares} + \text{plus sign}}$$



Here is an example of a line **that does not fit** the data well.



If we just examine the area of the red squares compared to the blue squares, we see the area is almost identical. The ratio of the areas is close to one. In this case, the R squared **is near zero**. This line performs about the same as just using the average of the data points. Therefore, this line did not perform well.

We find the R squared value in Python by using the score method in the linear regression object. From the value that we get from this example, we can say that approximately

49.659% of the variation of price is explained by this simple linear model.

- Generally the values of the MSE are between 0 and 1.
- We can calculate the R^2 as follows

```
X = df[['highway-mpg']]
Y = df['price']
```

```
lm.fit(X, Y)
```

```
lm.score(X, y)
0.496591188
```

Your R squared value is usually between zero and one. If your r square is **negative**, it can be due to **overfitting** that we will discuss in the next module.

Prediction and decision Making

Decision making: Determining a good model fit

To determine final best fit, we look at a combination of:

- Do the predicted values make sense
- Visualization
- Numerical measures for evaluation
- Comparing models

How can we **determine** if our **model** is **correct**?

The first thing you should do is make sure your **model results make sense**.

You should always use visualization, numerical measures for evaluation, and comparing between different models.

Predicted values make sense?

Do the predicted values make sense?

- First we train the model

```
lm.fit(df['highway-mpg'], df['prices'])
```

- Let's predict the price of a car with 30 highway-mpg

```
lm.predict(np.array(30.0).reshape(-1,1))
```

- Result: \$ 13771.30

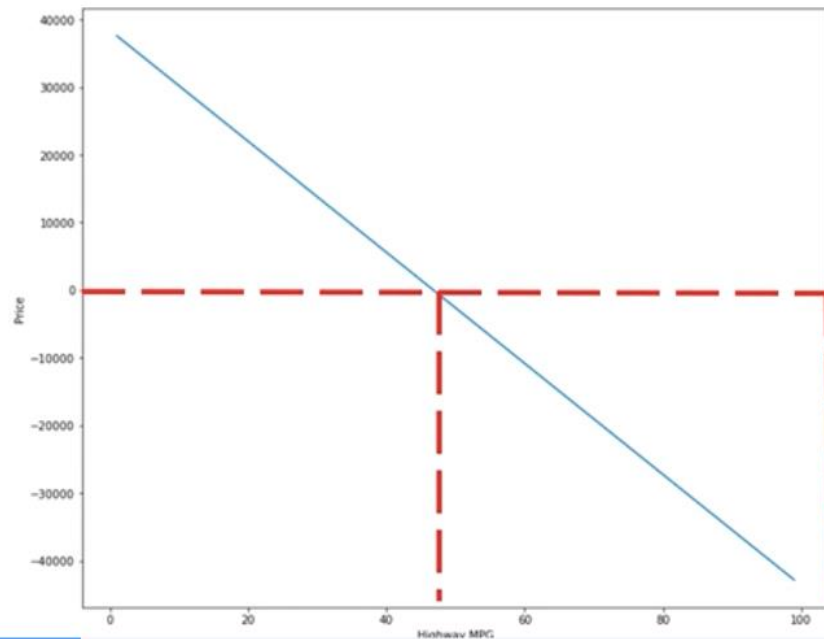
```
lm.coef_  
-821.73337832
```

- **Price** = $38423.31 - 821.73 * \text{highway-mpg}$

We trained the model using the fit method. Now we want to find out what the price would be for a car that has a highway miles per gallon of 30. Plugging this value into the predict method gives us a resulting price of \$13,771.30. This seems to make sense. For example, the value is not negative, extremely high, or extremely low. We can look at the coefficients by examining the coef underscore attribute. If you recall, the expression for the simple linear model that predicts price from highway miles per gallon. This value corresponds to the multiple of a highway miles per gallon feature. As such, an increase of 1 unit in highway miles per gallon, the value of the car decreases approximately \$821. This value also seems reasonable.

Sometimes your model will produce values that don't make sense.

Do the predicted values make sense?



For example, if we plot the model out for highway miles per gallon in the ranges of 0 to 100, we **get negative values for the price**. This could be because the **values in that range are not realistic**. The linear assumption is incorrect, or **we don't have data** for cars in that range. In this case, it is unlikely that a car will have fuel mileage in that range, so our model seems valid.

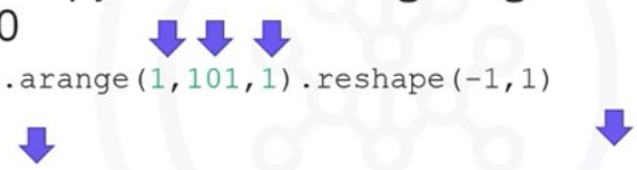
To generate a sequence of values in a specified range, import numpy, then use the numpy arrange function to generate the sequence. The sequence starts at 1 and increments by 1 till we reach 100. The first parameter is the starting point of the sequence. The second parameter is the endpoint plus 1 of the sequence. The final parameter is the step size between elements in the sequence. In this case, it's 1, so we increment the sequence one step at a time, from 1 to 2, and so on.

- First, we import numpy

```
import numpy as np
```

- We use the numpy function `arange` to generate a sequence from 1 to 100

```
new_input=np.arange(1,101,1).reshape(-1,1)
```



1	2	...	99	100
---	---	-----	----	-----

We can use the output to predict new values. The output is a numpy array. Many of the values are negative.

- We can predict new values

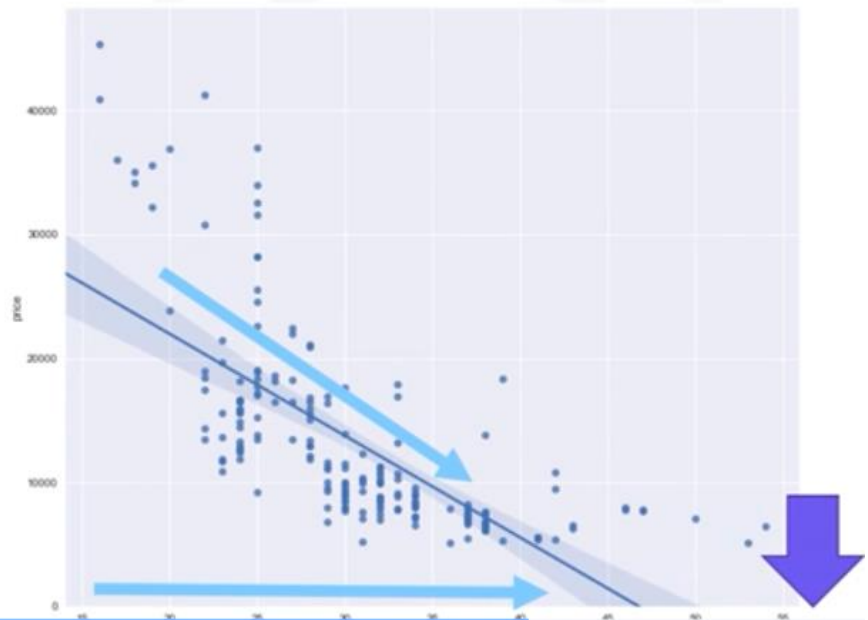
```
yhat=lm.predict(new_input)
```

```
array([[ 37601.57247984,  36779.83910151,  35958.10572319,  35136.37234487,
        34314.63896655,  33492.90558823,  32671.1722099 ,  31849.43883158,
        31027.70545326,  30205.97207494,  29384.23869662,  28562.50531829,
        27740.77193997,  26919.03856165,  26097.30518333,  25275.57180501,
        24453.83842668,  23632.10504836,  22810.37167004,  21988.63829172,
        21166.9049134 ,  20345.17153508,  19523.43815675,  18701.70477843,
        17879.97140011,  17058.23802179,  16236.50464347,  15414.77126514,
        14593.03788682,  13771.3045085 ,  12949.57113018,  12127.83775186,
        11306.10437353,  10484.37099521,  9662.63761689,  8840.90423857,
        8019.17086025,  7197.43748192,  6375.7041036 ,  5553.97072528,
        4732.23734696,  3910.50396864,  3088.77059031,  2267.03721199,
        -1841.62967962, -2663.36305794, -3485.09643626, -4306.82981458,
        -5128.5631929 , -5950.29657123, -6772.02994955, -7593.76332787,
        -8415.49670619, -9237.23008451, -10058.96346284, -10880.69684116,
        -11702.43021948, -12524.1635978 , -13345.89697612, -14167.63035445,
        -14989.36373277, -15811.09711109, -16632.83048941, -17454.56386773,
        -18276.29724606, -19098.03062438, -19919.7640027 , -20741.49738102,
        -21563.23075934, -22384.96413767, -23206.69751599, -24028.43089431,
        -24850.16427263, -25671.89765095, -26493.63102927, -27315.3644076 ,
        -28137.09778592, -28958.83116424, -29780.56454256, -30602.29792088,
        -31424.03129921, -32245.76467753, -33067.49805585, -33889.23143417,
        -34710.96481249, -35532.69819082, -36354.43156914, -37176.16494746,
        -37997.89832578, -38819.6317041 , -39641.36508243, -40463.09846075,
        -41284.83183907, -42106.56521739, -42928.29859571])
```

Visualization

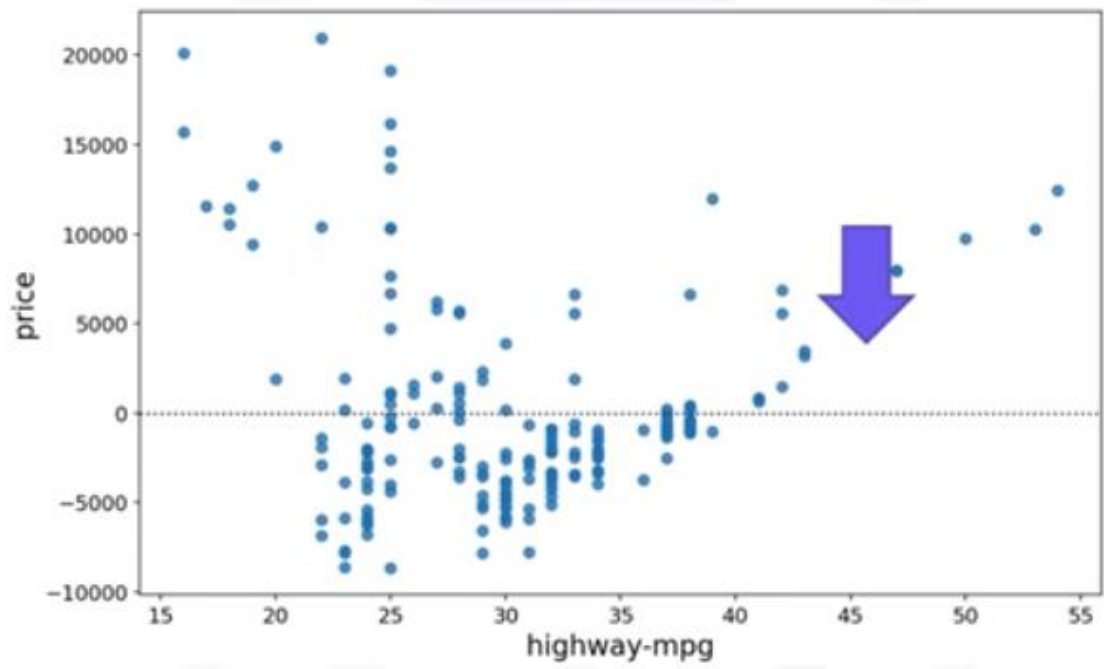
Using a regression plot to visualize your data is the first method you should try. See the labs for examples of how to plot polynomial regression. For this example, the effect of the independent variable is evident in this case.

- Simply visualizing your data with a regression

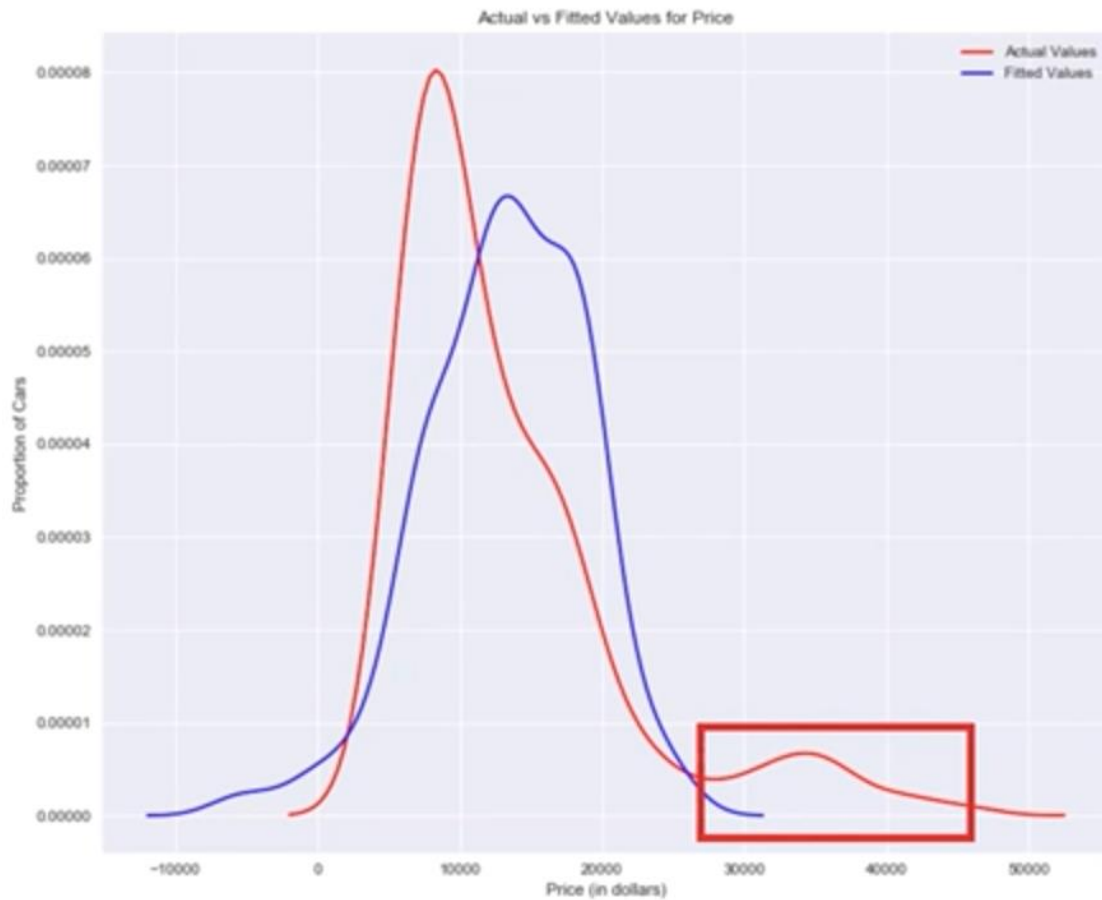


The data trends down as the dependent variable increases. The plot **also shows some non-linear behavior**.

Examining the residual plot, we see in this case the residuals have a **curvature suggesting non-linear behavior**.



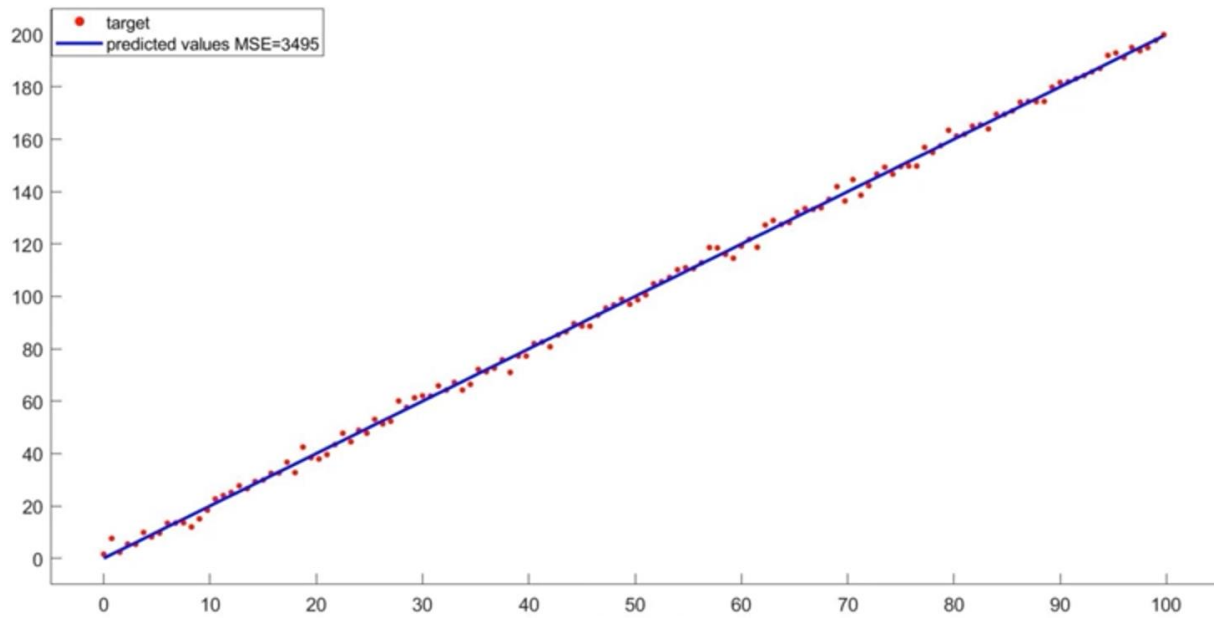
A distribution plot is a good method for multiple linear regression.



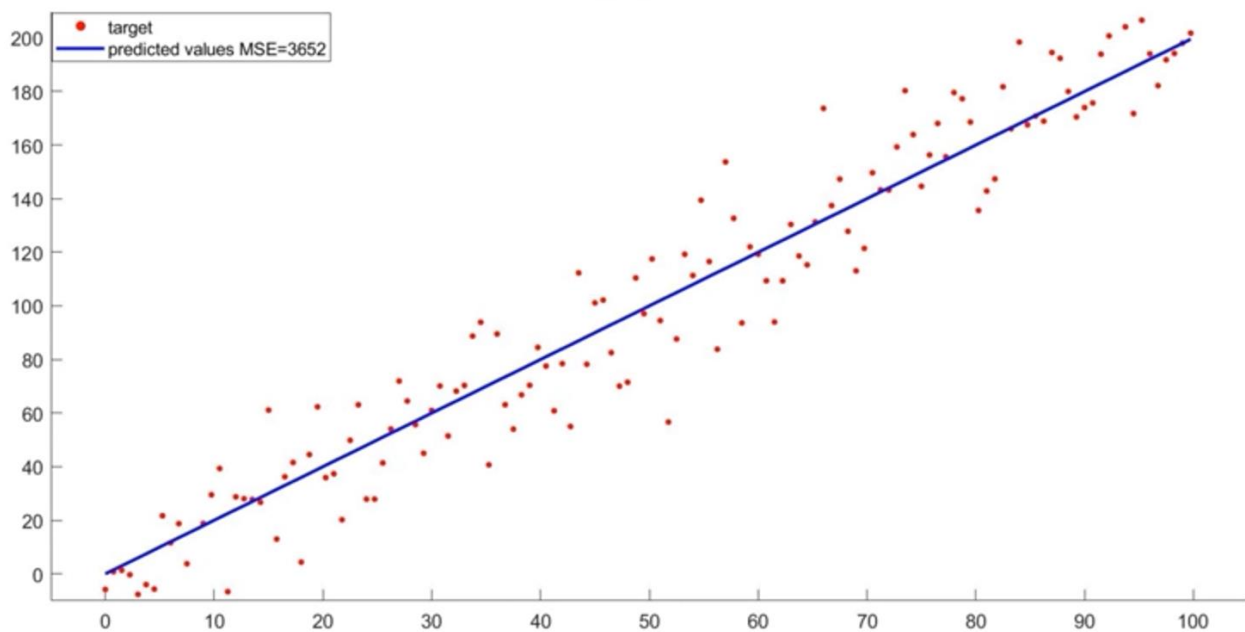
For example, we see the predicted values for prices in the range from \$30,000 to \$50,000 are inaccurate. This suggests a **non-linear model may be more suitable**, or we **need more data in this range**.

mean square error

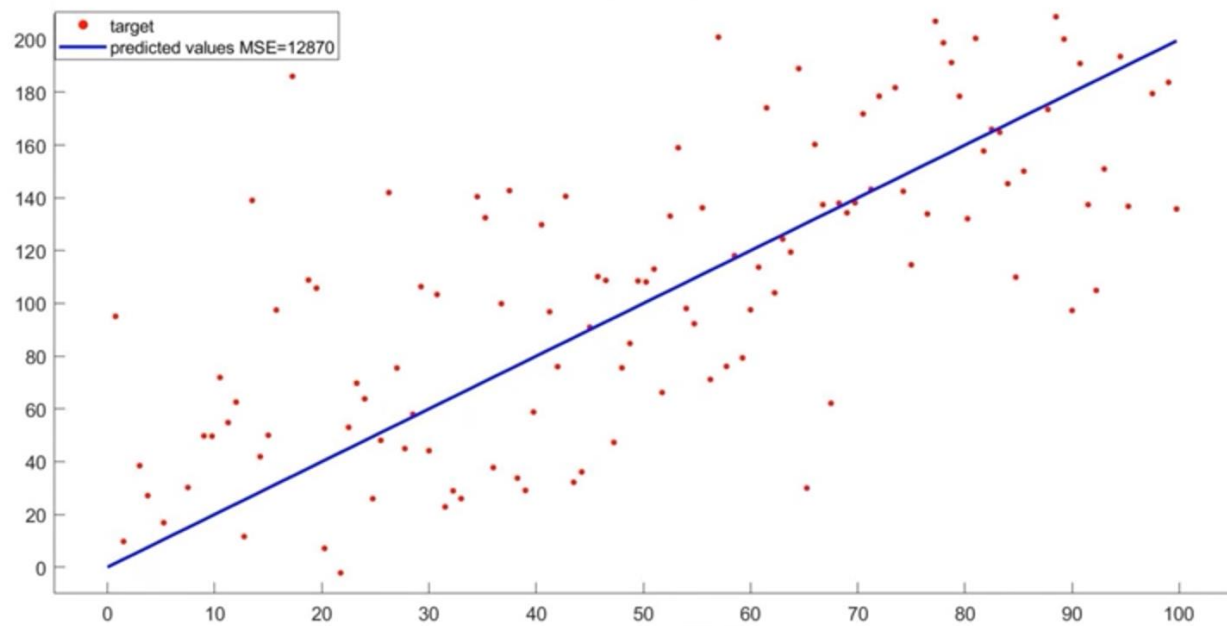
The mean square error is perhaps the most intuitive numerical measure for determining if a model is good or not.



The figure shows an example of a mean square error of 3,495.



This example has a mean square error of 3,652.



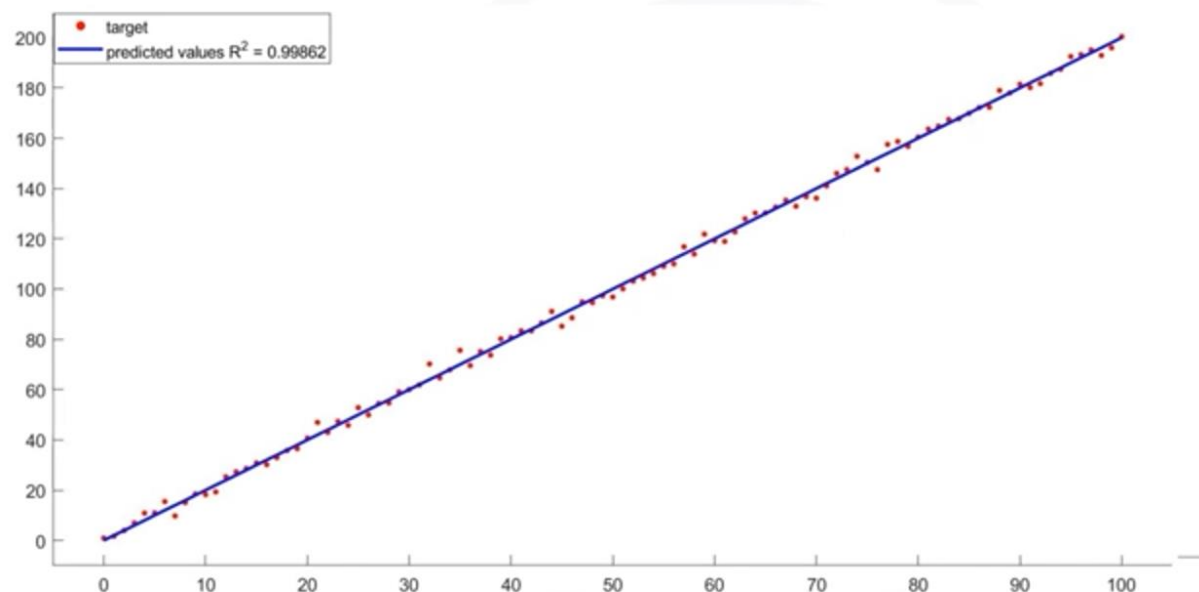
The final plot has a mean square error of 12,870.

As the **square error increases**, the targets get further from the predicted points.

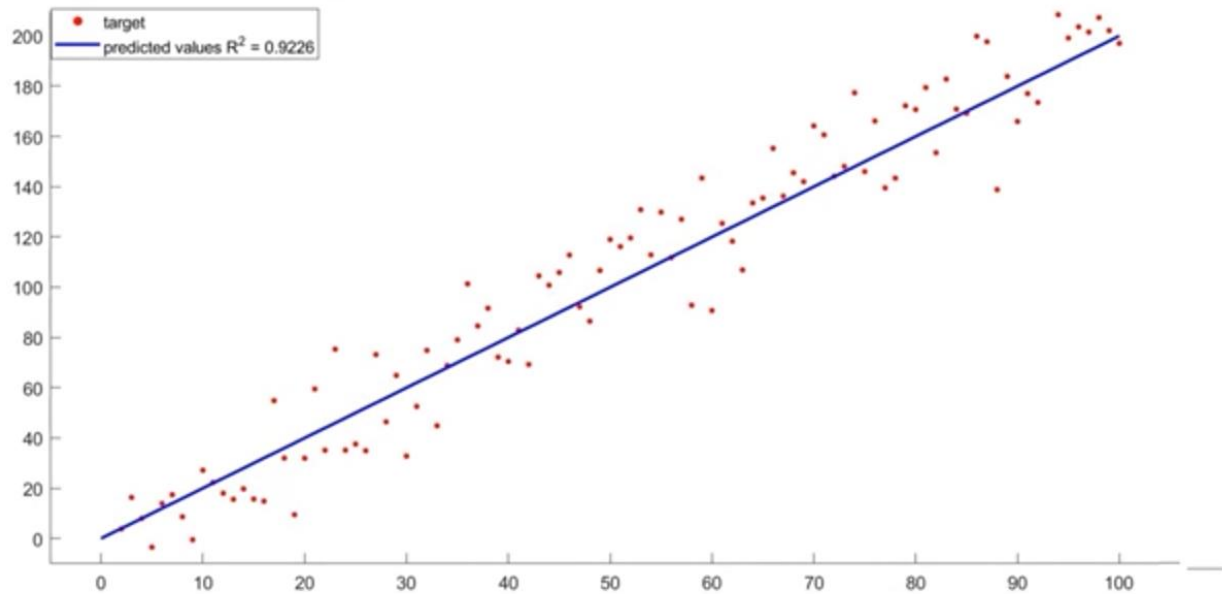
R-squared

As we discussed, R-squared is another popular method to evaluate your model.

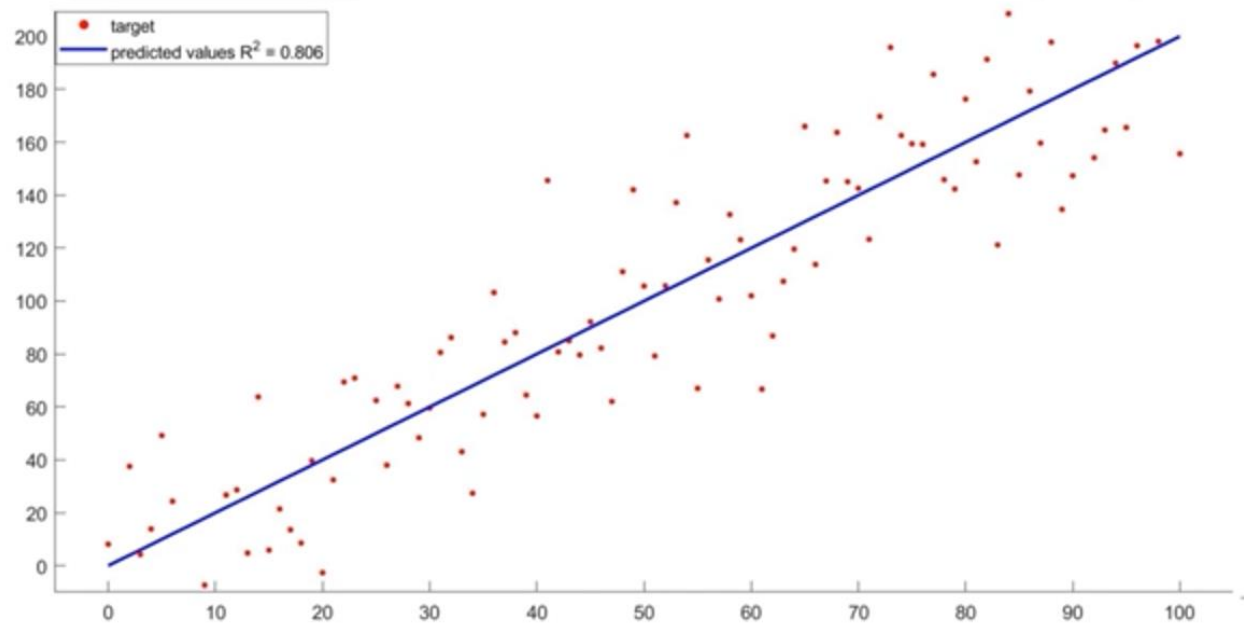
In this plot, we see the target points in red and the predicted line in blue. An R-squared of 0.9986. The model appears to be a good fit.



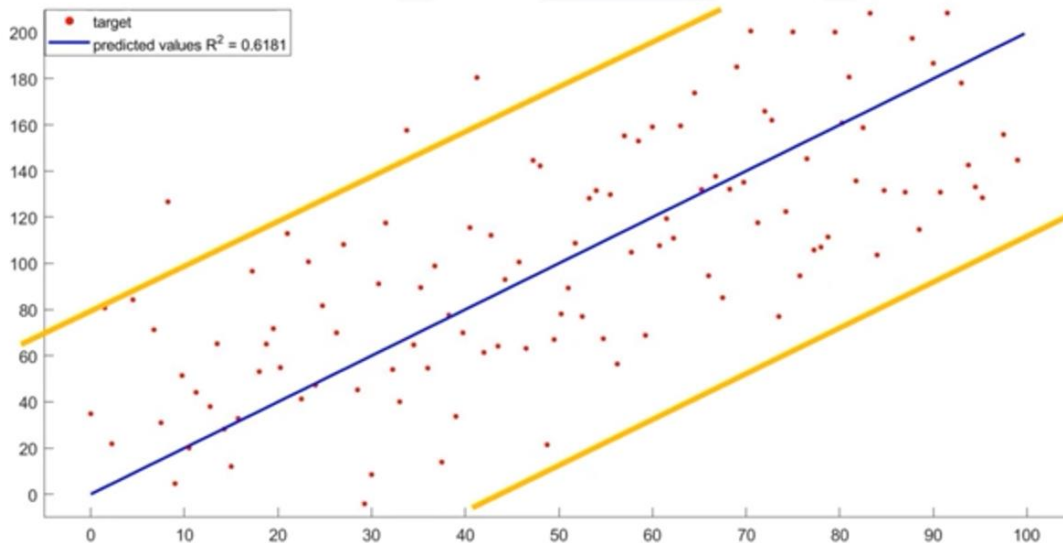
This model has an R-squared of 0.9226. There still is a strong linear relationship.



An R-squared of 0.806. The data is a lot more messy, but the linear relation is evident.



An R-squared of 0.61. The linear function is harder to see, but on closer inspection, we see the data is increasing with the independent variable.



An **acceptable value for R-squared depends on what field you're studying**. Some authors suggest a value should be equal to or greater than 0.10.

Comparing MLR and SLR:

Comparing MLR and SLR

1. Is a lower MSE always implying a better fit?
 - Not necessarily
2. MSE for an MLR model will be smaller than the MSE for an SLR model since the errors of the data will decrease when more variables are included in the model
3. Polynomial regression will also have a smaller MSE than regular regression
4. A similar inverse relationship holds for R^2

is a lower MSE always implying a better fit? Not necessarily. MSE for an MLR model will be smaller than the MSE for an SLR model, since the **errors** of the data will **decrease when more variables are included** in the model.

Polynomial regression will also have a **smaller** MSE than **regular** regression. A similar inverse relationship holds for R-squared. In the next section, we'll look at better ways to evaluate the model.

Lesson Summary

- Linear regression refers to using one independent variable to make a prediction.
- You can use multiple linear regression to explain the relationship between one continuous target *y* variable and two or more predictor *x* variables.
- Simple linear regression, or SLR, is a method used to understand the relationship between two variables, the predictor independent *variable x* and the target dependent *variable y*.
- Use the **regplot** and **residplot** functions in the Seaborn library to create regression and residual plots, which help you identify the strength, direction, and linearity of the relationship between your independent and dependent variables.
- When using residual plots for model evaluation, residuals should ideally have zero mean, appear evenly distributed around the *x*-axis, and have consistent variance. If these conditions are not met, consider adjusting your model.
- Use distribution plots for models with multiple features: Learn to construct distribution plots to compare predicted and actual values, particularly when your model includes more than one independent variable. Know that this can offer deeper insights into the accuracy of your model across different ranges of values.
- The order of the polynomials affects the fit of the model to your data. Apply Python's **polyfit** function to develop polynomial regression models that suit your specific dataset.
- To prepare your data for more accurate modeling, use feature transformation techniques, particularly using the **preprocessing** library in scikit-learn, transform your data using polynomial features, and use the modules like **StandardScaler** to normalize the data.
- Pipelines allow you to simplify how you perform transformations and predictions sequentially, and you can use pipelines in scikit-learn to streamline your modeling process.
- You can construct and train a pipeline to automate tasks such as normalization, polynomial transformation, and making predictions.
- To determine the fit of your model, you can perform sample evaluations by using the Mean Square Error (MSE), using Python's **mean_squared_error** function from scikit-learn, and using the score method to obtain the R-squared value.

- A model with a high R-squared value close to 1 and a low MSE is generally a good fit, whereas a model with a low R-squared and a high MSE may not be useful.
- Be alert to situations where your R-squared value might be negative, which can indicate overfitting.
- When evaluating models, use visualization and numerical measures and compare different models.
- The mean square error is perhaps the most intuitive numerical measure for determining whether a model is good.
- A distribution plot is a suitable method for multiple linear regression.
- An acceptable r-squared value depends on what you are studying and your use case.
- To evaluate your model's fit, apply visualization, methods like regression and residual plots, and numerical measures such as the model's coefficients for sensibility:
- Use Mean Square Error (MSE) to measure the average of the squares of the errors between actual and predicted values and examine R-squared to understand the proportion of the variance in the dependent variable that is predictable from the independent variables.
- When analyzing residual plots, residuals should be randomly distributed around zero for a good model. In contrast, a residual plot curve or inaccuracies in certain ranges suggest non-linear behavior or the need for more data.

Module 5: Model Evaluation and Refinement

Model Evaluation

- In-sample evaluation tells us how well our model will fit the data used to train it
- Problem?
 - It does not tell us how well the trained model can be used to predict new data
- Solution?
 - In-sample data or training data
 - Out-of-sample evaluation or test set

Model evaluation tells us how our model performs in the real world.

In the previous module, we talked about **in-sample evaluation** that tells us how well our model fits the data already given to train it. It does not give us an estimate of how well the trained model can predict new data.

The solution is **to split our data** up.

Use **the in-sample data** or training data to train the model.

Use **the out-of-sample** or test data to approximate how the model performs in the real world.

Separating data into training and testing sets is an important part of model evaluation. We use the test data to get an idea how our model will perform in the real world.

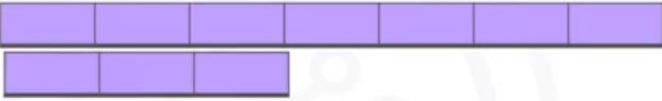
When we split a dataset, usually the **larger portion of data is used for training** and a smaller part is used for testing. For example, we can use 70% of the data for training. We then use 30% for testing.

We use training set to build a model and discover predictive relationships.

We then use a testing set to evaluate model performance. When we have completed testing our model, **we should use all the data to train the model**.

Training/Testing sets

Data:

- Split dataset into:
 - training set (70%) 
 - testing set (30%)
- Build and train the model with a training set
- Use testing set to assess the performance of a predictive model
- When we have completed testing our model we should use all the data to train the model to get the best performance

A popular function in the Scikit-learn package for splitting datasets is the `train_test_split` function.

This function randomly splits a dataset into training and testing subsets from the example code snippet.

This method is imported from `sklearn.model_selection`. The input parameters `y_data` is the target variable. In the car appraisal example it would be the price, and `x_data`, the list of predictor variables. In this case, it would be all the other variables in the car dataset that we are using to try to predict the price.

The output is an array, `x_train` and `y_train`. The subsets for training `x_test`, and `y_test` the subsets for testing. In this case, the test size is a percentage of the data for the testing set. Here it is 30%. The random state is a random seed for random data set splitting.

Function `train_test_split()`

- Split data into random train and test subsets

```
from sklearn.model_selection import train_test_split  
  
x_train, x_test, y_train, y_test = train_test_split(x_data, y_data, test_size=0.3, random_state=0)
```



- **x_data**: features or independent variables
- **y_data**: dataset target: `df['price']`
- **x_train, y_train**: parts of available data as training set
- **x_test, y_test**: parts of available data as testing set
- **test_size**: percentage of the data for testing (here 30%)
- **random_state**: number generator used for random sampling

Generalization error is a measure of how well our data does at predicting **previously unseen** data. **The error we obtain using our testing data approximates this error.**

Generalization performance

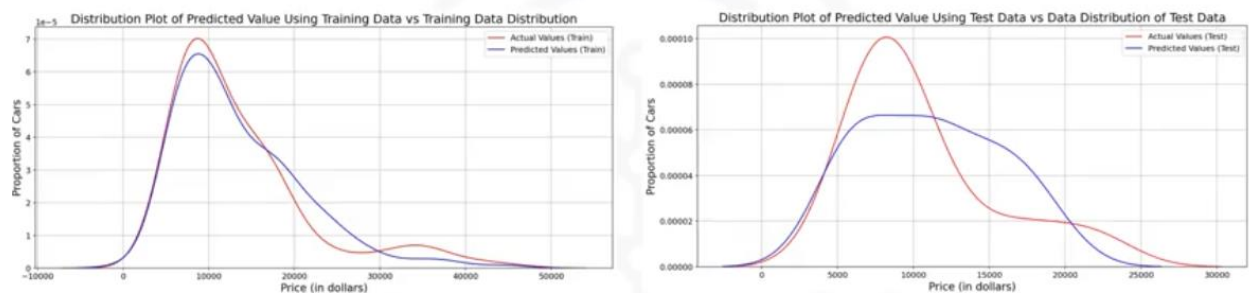
- Generalization error is a measure of how well our data does at predicting previously unseen data
- The error we obtain using our testing data is an approximation of this error

This figure shows the distribution of the actual values (**y_train**) in red compared to the predicted values (**y_train_hat**) from a linear regression in blue. We see the distributions are somewhat similar.

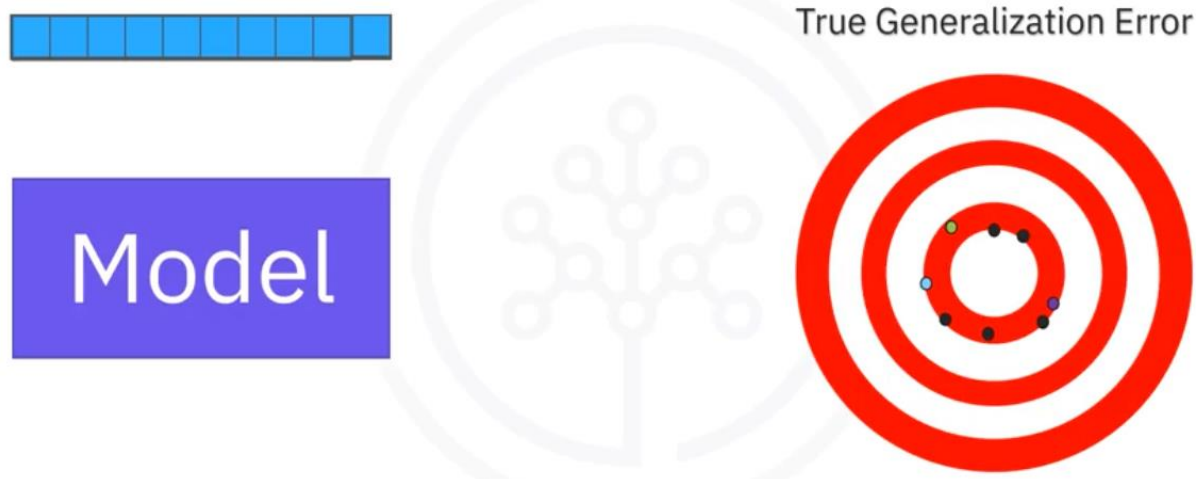
If we generate the same plot using the test data (**y_test** vs **t_test_hat**), we see the distributions are relatively different.

The **difference** is due to a **generalization error** and represents what we see in the real world.

Generalization error



Lots of training data



Using **a lot of data** for **training** gives us an accurate means of determining how well our model will perform in the real world, but the **precision** of the performance will be **low**.

Let's clarify this with an example. The center of this bull's eye represents the correct generalization error.

Let's say we take a random sample of the data using **90% of the data for training** and 10% for testing. The first time we experiment; we get a good estimate of the training data.

If we experiment again, training the model with a different combination of samples, we also get a good result, but the results will be different relative to the first time we run the experiment.

Repeating the experiment again with a different combination of training and testing samples, the results are relatively close to the generalization error, but distinct from each other.

Repeating the process, we get good approximation of the generalization error, but the precision is poor, i.e. all the results were extremely different from one another.

If we use **fewer data points to train** the model and more to test the model, the **accuracy of the generalization performance will be less**, but the **model will have good precision**.

The figure above demonstrates this. All our error estimates are relatively close together, but they are further away from the true generalization performance.

Lots of training data



To overcome this problem, we use cross validation.

One of the most **common out-of-sample evaluation metrics** is cross validation.

In this method, the dataset is split into k equal groups or folds.

Some of the folds can be used as a training set which we use to train the model, and the remaining parts are used as a test set which we use to test the model.

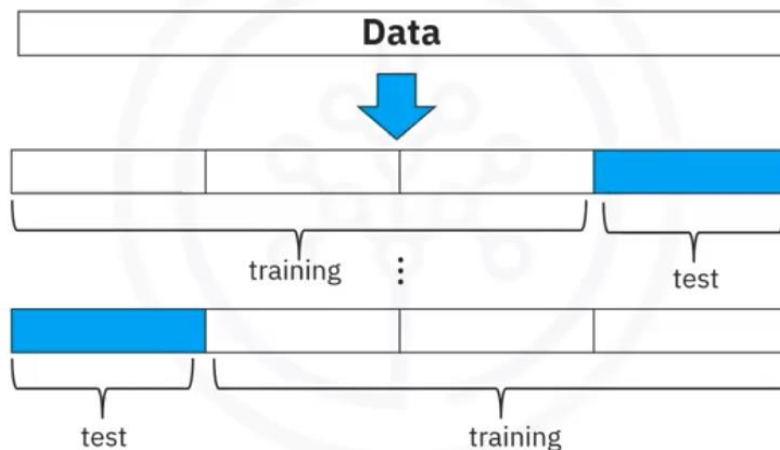
This is repeated until each fold is used for both training and testing.

At the end, we use the average results as the **estimate of out-of-sample error**.

The evaluation metric depends on the model. For example, the R^2 .

Cross validation

- Most common out-of-sample evaluation metrics
- More effective use of data (each observation is used for both training and testing)



The simplest way to apply cross validation is to call the `cross_val_score` function which **performs multiple out-of-sample evaluations**. This method is imported from sklearn's model selection package. We then use the function `cross_val_score`.

The first input parameter is the type of **model we are using** to do the cross validation. In this example, we initialize the linear regression model or object `lr`, which we passed the `cross_val_score` function.

The other parameters are `x_data`, the **predictor variable** data, and `y_data`, the **target variable** data.

We can manage the **number of partitions** or **folds** with the `cv` parameter. Here, `cv=3`, which means the data set is split into three equal partitions. The function returns an array of scores, one for each partition that was chosen as the testing set.

We can average the result together to estimate out-of-sample R^2 using the `mean` function in NumPy.

Function cross_val_score()

```
from sklearn.model_selection import cross_val_score
```

```
scores = cross_val_score(lr, x_data, y_data, cv=3)
```

```
np.mean(scores)
```

Let's see the result of the score array in the last slide. First we split the data into three folds. We use two folds for training the remaining fold for testing.

The model will produce an output, we will use the output to calculate a score. In the case of the R^2 , i.e. coefficient of determination, we will store that value in an array.

we will repeat the process using two folds for training and one fold for testing, save the score.

Then use a different combination for training and the remaining fold for testing. We store the final result.

Evaluating cross-validation score

Fold 1	Fold 2	Fold 3
Test	Train	Train
Train	Test	Train

Data set 1

Data set 2

Data set 3

Model



$$R^2 = \left(1 - \frac{\text{MSE of regression line}}{\text{MSE of } \bar{y}} \right)$$

scores

R^2 set 1

Function `cross_val_predict()`

- It returns the prediction that was obtained for each element when it was in the test set.
- Has a similar interface to `cross_val_score()`

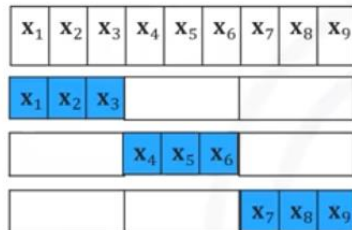
```
from sklearn.model_selection import cross_val_predict  
  
yhat=cross_val_predict (lr2e, x_data, y_data, cv=3)
```

The `cross_val_score` function **returns the score value to tell us the cross validation result.**

What if we want a little more information? What if we want to know the **actual predicted values** supplied by our model before the R^2 values are calculated?

To do this, we use the `cross_val_predict` function. The input parameters are exactly the same as the `cross_val_score` function, but the output is a prediction.

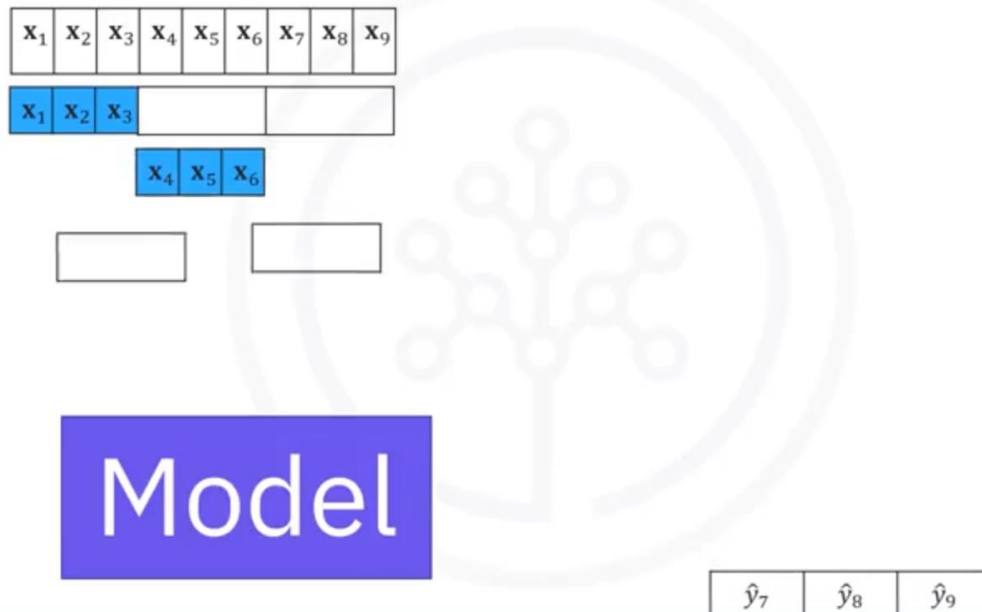
Evaluating cross-validation predictions



Model

First, we split the data into three folds. We use two folds for training, the remaining fold for testing. The model will produce an output and we will store it in an array.

Evaluating cross-validation predictions



We will repeat the process using two folds for training, one for testing. The model produces an output again.

Evaluating cross-validation predictions

x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9
-------	-------	-------	-------	-------	-------	-------	-------	-------

Model

$\hat{y}_1$	$\hat{y}_2$	$\hat{y}_3$
-------------	-------------	-------------

$\hat{y}_4$	$\hat{y}_5$	$\hat{y}_6$
-------------	-------------	-------------

$\hat{y}_7$	$\hat{y}_8$	$\hat{y}_9$
-------------	-------------	-------------

Finally, we use the last two folds for training, then we use the testing data. This final testing fold produces an output. These predictions are stored in an array.

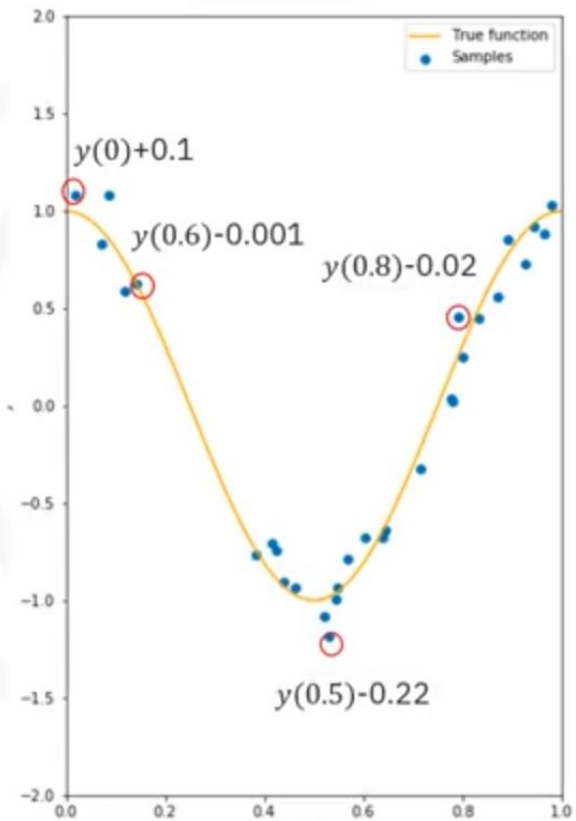
Overfitting Underfitting and Model Selection

In this section, we will **discuss how to pick the best polynomial order** and problems that arise when selecting the wrong order polynomial.

Consider the following function. We assume the training points come from a polynomial function plus some noise.

Model Selection

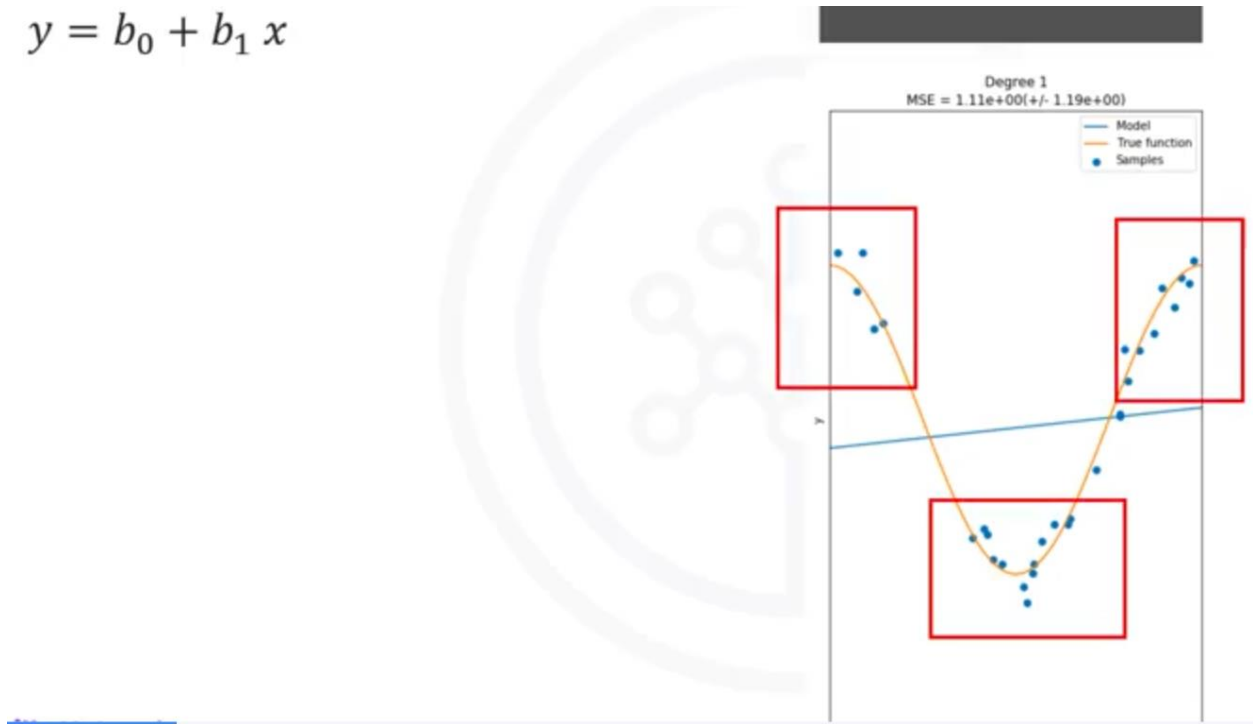
$y(x) + \text{noise}$



The goal of **model selection** is to determine the order of the polynomial to provide the **best estimate** of the function $y(x)$.

If we try and fit the function with a linear function, the line is not complex enough to fit the data.

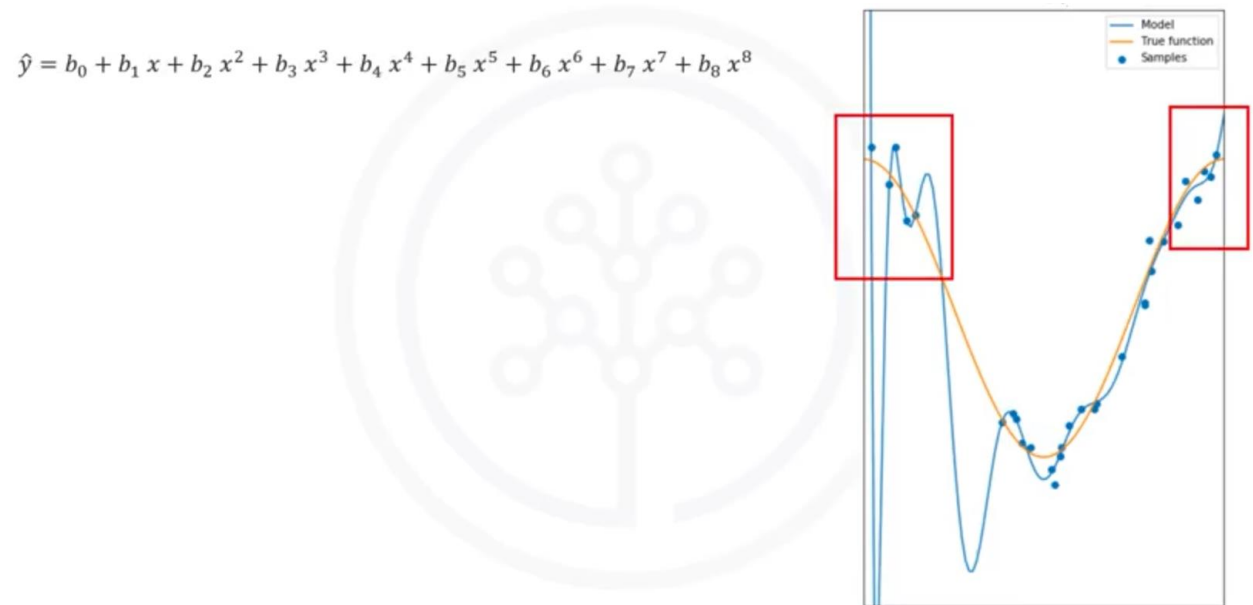
$$y = b_0 + b_1 x$$



As a result, there are many errors. This is called **underfitting**, where the model is too simple to fit the data. If we increase the order of the polynomial, the model fits better. But the model is still not flexible enough and exhibits underfitting.

This is an example of the eighth order polynomial used to fit the data.

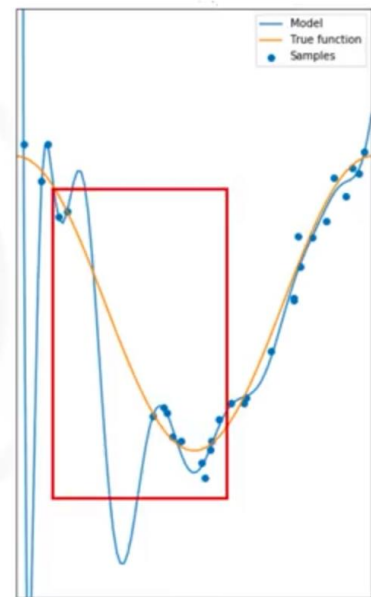
$$\hat{y} = b_0 + b_1 x + b_2 x^2 + b_3 x^3 + b_4 x^4 + b_5 x^5 + b_6 x^6 + b_7 x^7 + b_8 x^8$$



We see the model does well at fitting the data and estimating the function, even at the inflection points.

Increasing it to a 16th order polynomial, the model does **extremely well at tracking the training point**, but **performs poorly at estimating the function**.

$$\hat{y} = b_0 + b_1 x + b_2 x^2 + b_3 x^3 + b_4 x^4 + b_5 x^5 + b_6 x^6 + b_7 x^7 + b_8 x^8 + \dots + b_9 x^9 + b_{10} x^{10} + b_{11} x^{11} + b_{12} x^{12} + b_{13} x^{13} + b_{14} x^{14} + b_{15} x^{15} + b_{16} x^{16}$$



This is especially apparent where there is little training data.

The **estimated function oscillates, not tracking the function**. This is called **overfitting**, where the model is too flexible and fits the noise rather than the function.

Let's look at a plot of the mean square error for the training and testing set of different order polynomials.

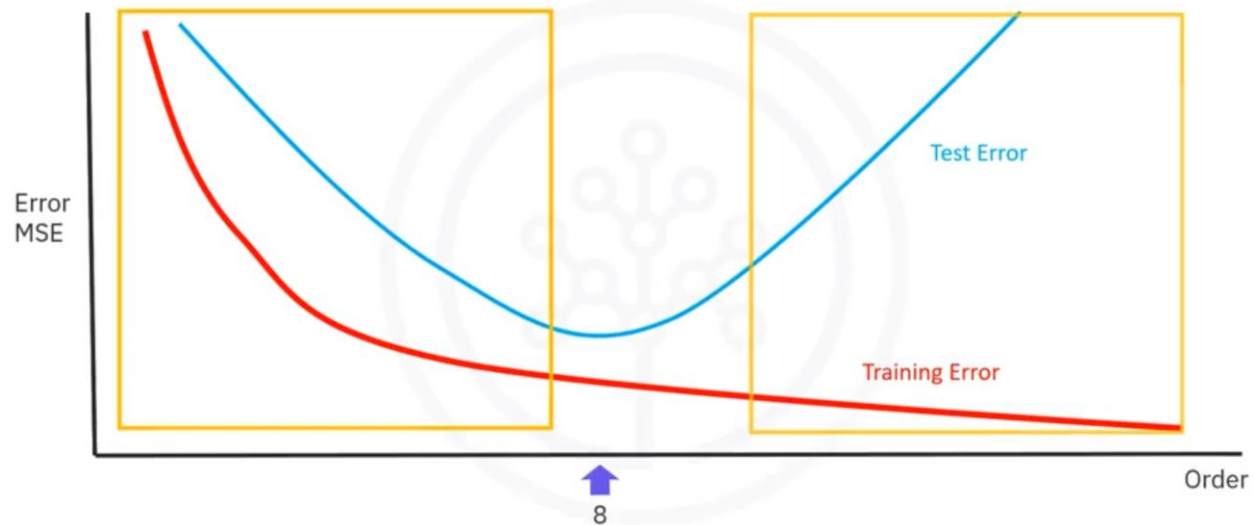
The horizontal axis represents the order of the polynomial. The vertical axis is the mean square error. The **training error decreases with the order of the polynomial**.

The test error is a better means of estimating the error of a polynomial. **The error decreases till the best order of the polynomial is determined**. Then the error begins to increase.

We select the order that minimizes the test error. In this case, it was eight.

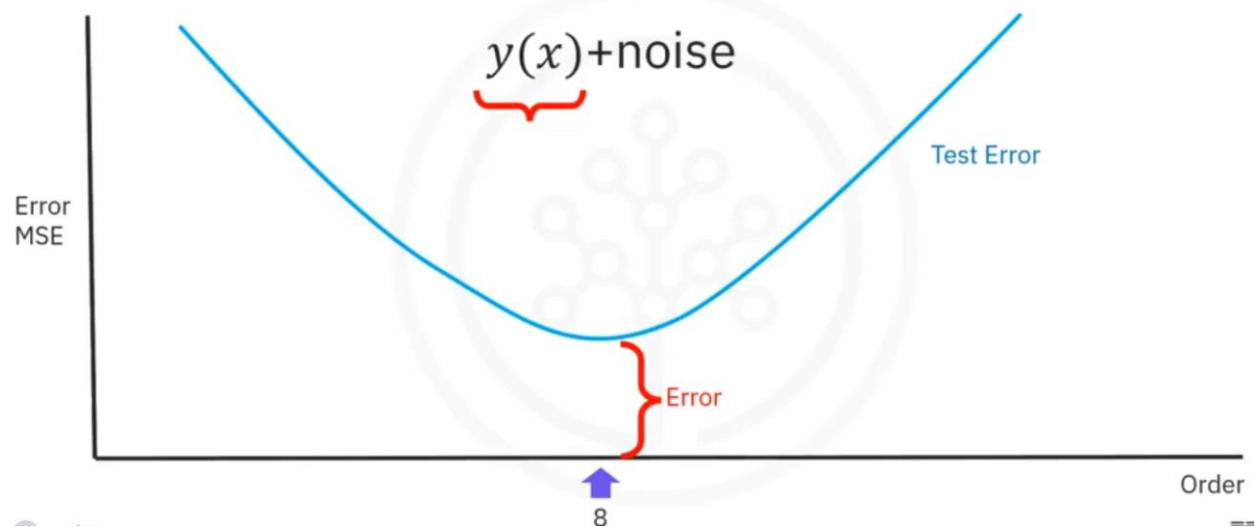
Anything on the left would be considered underfitting. Anything on the right is overfitting.

Model Selection



If we select the best order of the polynomial, we will still have some errors. If you recall the original expression for the training points, we see a noise term. This term is one reason for the error.

Model Selection

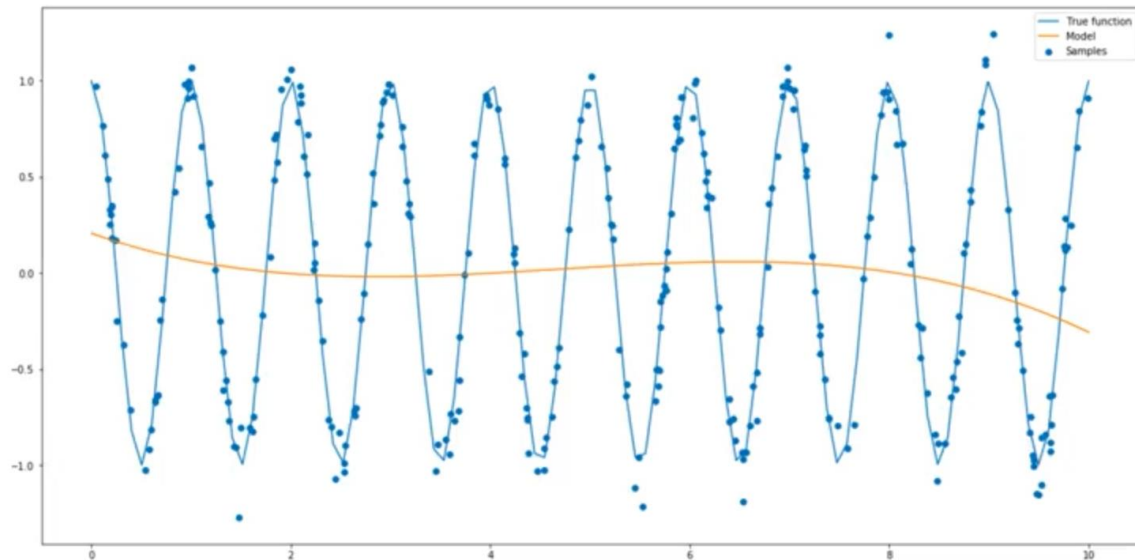


This is because the noise is random and we can't predict it. This is sometimes referred to as **an irreducible error**. There are other sources of errors as well:

- Polynomial assumption may be wrong.
- Our sample points may have come from a different function.

For example, in this plot, the data is generated from a sine wave. The polynomial function does not do a good job of fitting the sine wave.

Model Selection



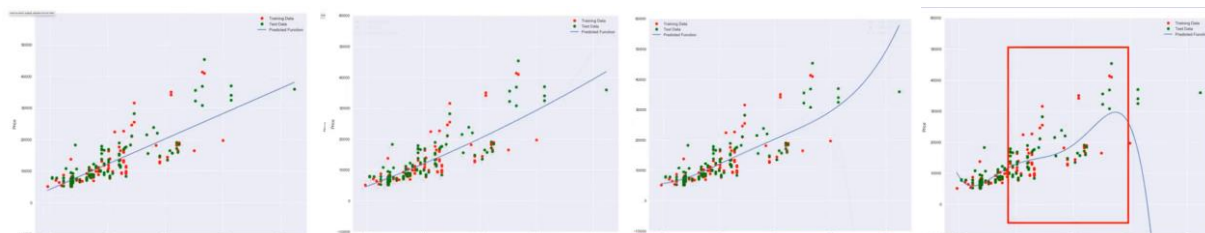
For real data, the model may be too difficult to fit, or we may not have the correct type of data to estimate the function.

Let's try different order polynomials on the real data using horsepower.

The red points represent the training data; the green points represent the test data. If we just use the mean of the data, our model does not perform well.

A linear function does fit the data better.

A second order model looks similar to the linear function.

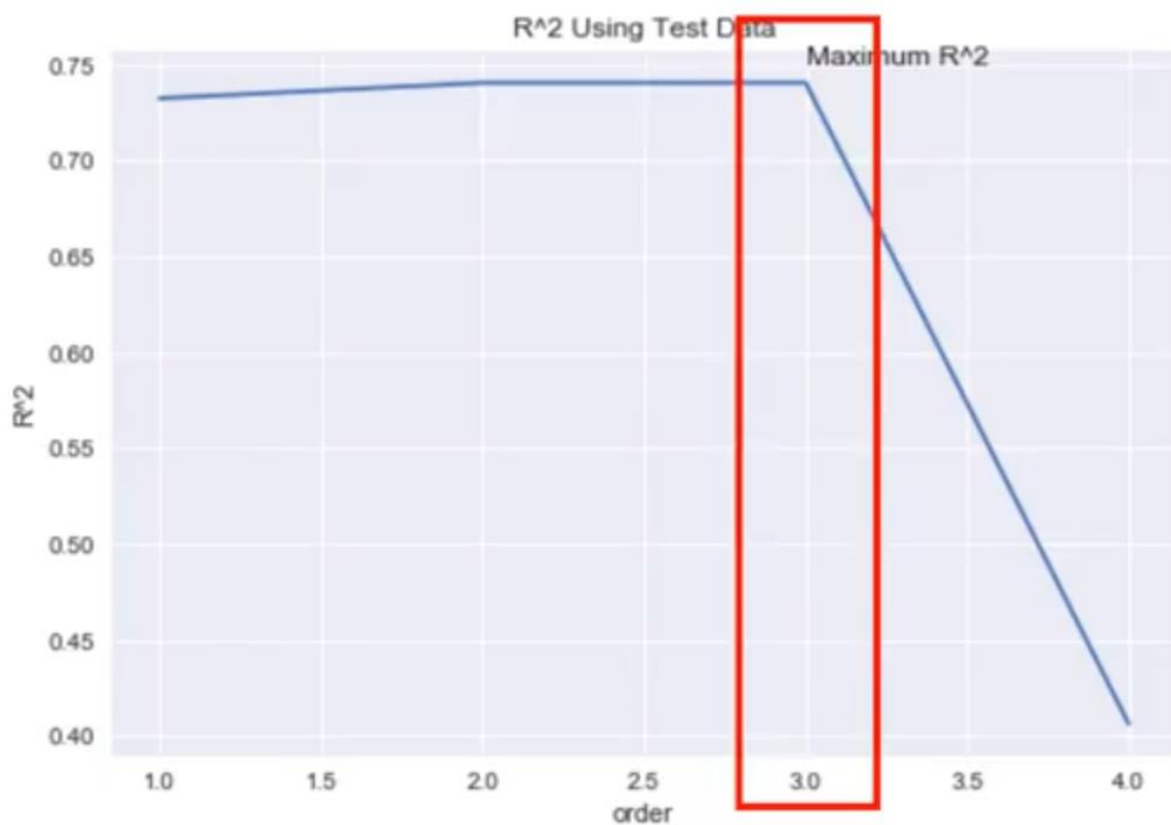


A third order function also appears to increase, like the previous two orders.

Here we see a fourth order polynomial. At around 200 horsepower the predicted price suddenly decreases. This seems erroneous.

Let's use R^2 to see if our assumption is correct.

The following is a plot of the R^2 value. The horizontal axis represents the order of polynomial models. The closer the R^2 is to one, the more accurate the model is. Here we see the R^2 is optimal when the order of the polynomial is three. The R^2 drastically decreases when the order is increased to four, validating our initial assumption.



We can calculate different R^2 values as follows.

```
Rsqu_test=[]  
order=[1,2,3,4]  
for n in order:  
    pr=PolynomialFeatures(degree=n)  
    x_train_pr=pr.fit_transform(x_train[['horsepower']])  
    x_test_pr=pr.fit_transform(x_test[['horsepower']])  
    lr.fit(x_train_pr,y_train)  
    Rsqu_test.append(lr.score(x_test_pr,y_test))
```

Introduction to Ridge Regression

For models with **multiple independent features** and **ones with polynomial feature extrapolation**, it is common to have colinear combinations of features. Left unchecked, this multicollinearity of features can lead the model to overfit the training data. To control this, the **feature sets are typically regularized using hyperparameters**.

Ridge regression is the process of regularizing the feature set using the hyperparameter alpha. The upcoming video shows how Ridge regression can be utilized to regularize and reduce standard errors and avoid over-fitting while using a regression model.

Ridge Regression

Ridge regression prevents overfitting.

In this video, we will focus on

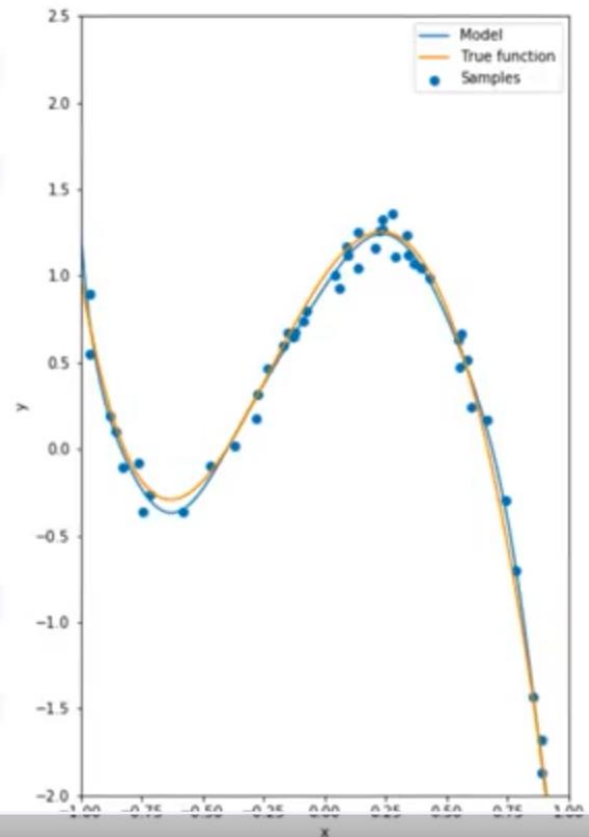
polynomial regression for visualization.

But **overfitting** is also a **big problem** when you have **multiple independent variables** or features.

Consider the following fourth order polynomial in orange. The blue points are generated from this function.

Ridge Regression

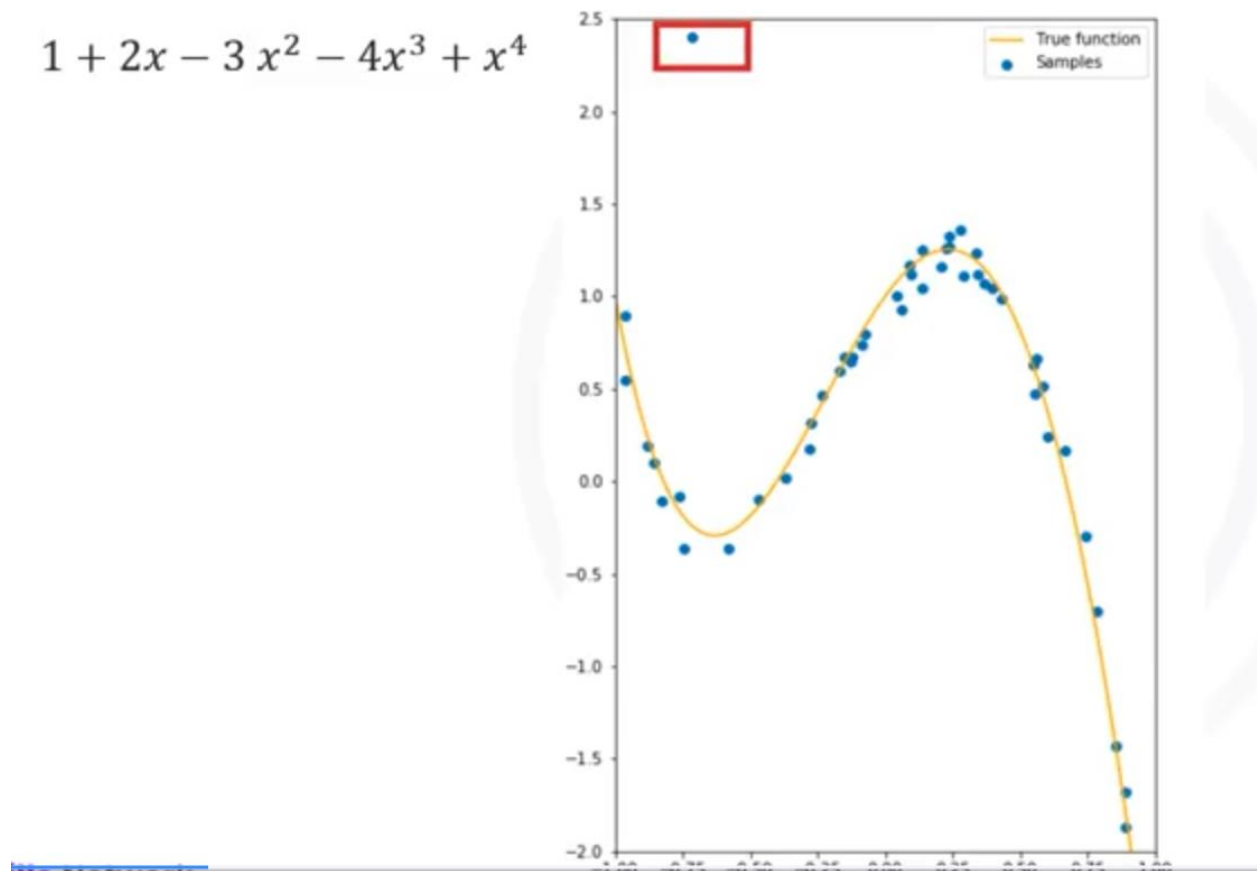
$$y = 1 + 2x - 3x^2 - 4x^3 + x^4$$



We can use a 10th order polynomial to fit the data. The estimated function in blue does a good job at approximating the true function.

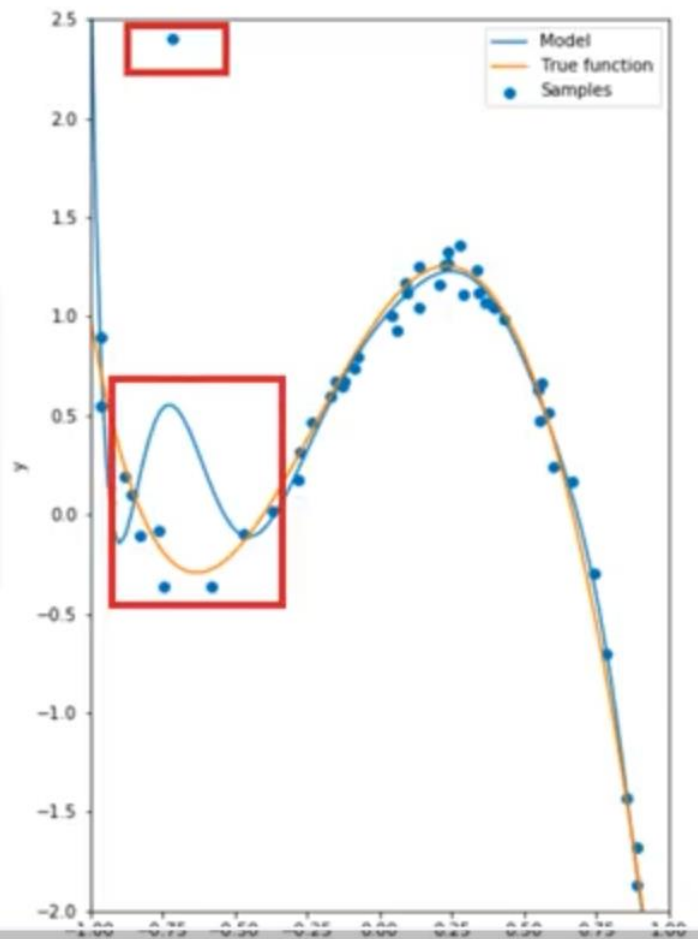
In many cases, real data has outliers. For example, this point shown here does not appear to come from the function in orange.

$$1 + 2x - 3x^2 - 4x^3 + x^4$$



If we use a 10th order polynomial function to fit the data, the estimated function in blue is incorrect and is not a good estimate of the actual function in orange.

$$1 + 2x - 3x^2 - 4x^3 + x^4$$



If we examine the expression for the estimated function, we see the estimated polynomial coefficients have a very large magnitude.

Ridge Regression

$$\hat{y} = 1 + 2x - 3x^2 - 2x^3 - 12x^4 - 40x^5 + 80x^6 + 71x^7 - 141x^8 - 38x^9 + 75x^{10}$$

This is especially evident for the higher order polynomials.

Ridge regression controls the magnitude of these polynomial coefficients by introducing the parameter Alpha. Alpha is a parameter we select before fitting or training the model.

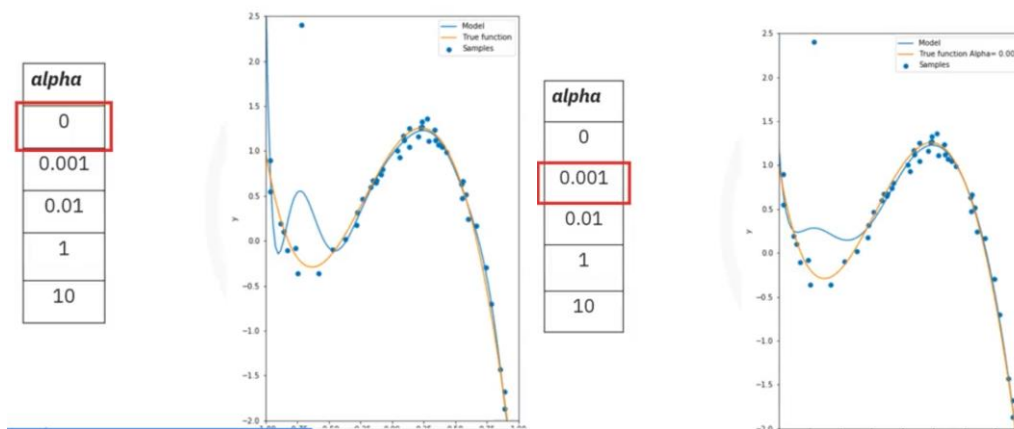
Each row in the following table represents an increasing value of Alpha. Let's see how different values of Alpha change the model. This table represents the polynomial coefficients for different values of Alpha.

Alpha	x	x^2	x^3	x^4	x^5	x^6	x^7	x^8	x^9	x^{10}
0	2	-3	-2	-12	-40	80	71	-141	-38	75
0.001	2	-3	-7	5	4	-6	4	-4	4	6
0.01	1	-2	-5	-0.04	0.15	-1	1	-0.5	0.3	1
1	0.5	-1	-1	-0.614	0.70	-0.38	-0.56	-0.21	-0.5	-0.1
10	0	-0.5	-0.3	-0.37	-0.30	-0.30	-0.22	-0.22	-0.22	-0.17

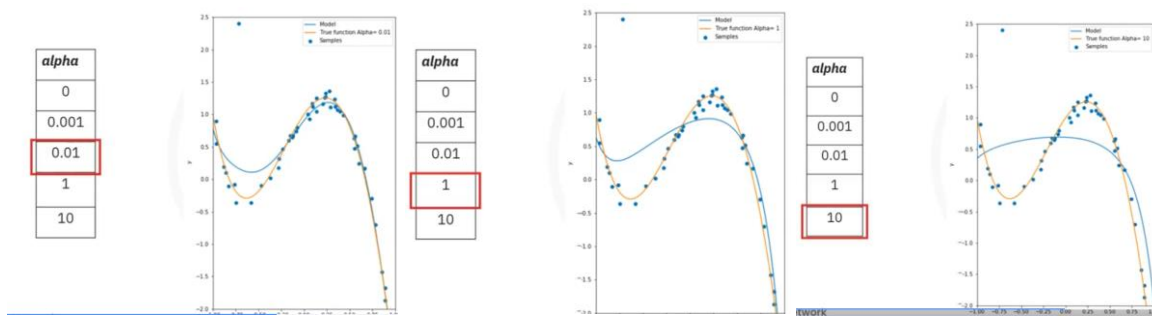
The column corresponds to the different polynomial coefficients, and the rows correspond to the different values of Alpha. As Alpha increases, the parameters get smaller. This is most evident for the higher order polynomial features.

But **Alpha must be selected carefully**. If Alpha is **too large**, the coefficients will approach zero and **underfit the data**.

If Alpha is zero, the overfitting is evident.



For Alpha equal to 0.001 the overfitting begins to subside.



For Alpha equal to 0.01 the estimated function tracks the actual function.

When Alpha equals one, we see the first signs of underfitting. The estimated function does not have enough flexibility.

At Alpha equals to 10, we see extreme underfitting. It does not even track the two points.

In order to **select Alpha, we use cross validation.**

To make a prediction using ridge regression import ridge from sklearn linear models.

Create a ridge object using the constructor. The parameter Alpha is one of the arguments of the constructor. We train the model using the fit method. To make a prediction we use the predict method.

Ridge Regression

```
from sklearn.linear_model import Ridge
```

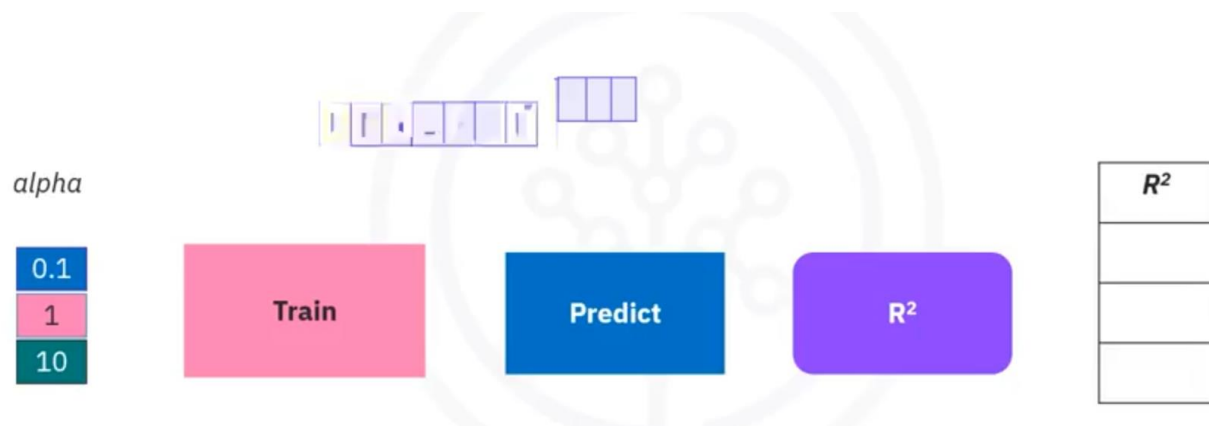
```
RidgeModel=Ridge(alpha=0.1)
```

```
RidgeModel.fit(X,y)
```

```
Yhat=RidgeModel.predict(X)
```

In order to determine the parameter Alpha we use some data for training.

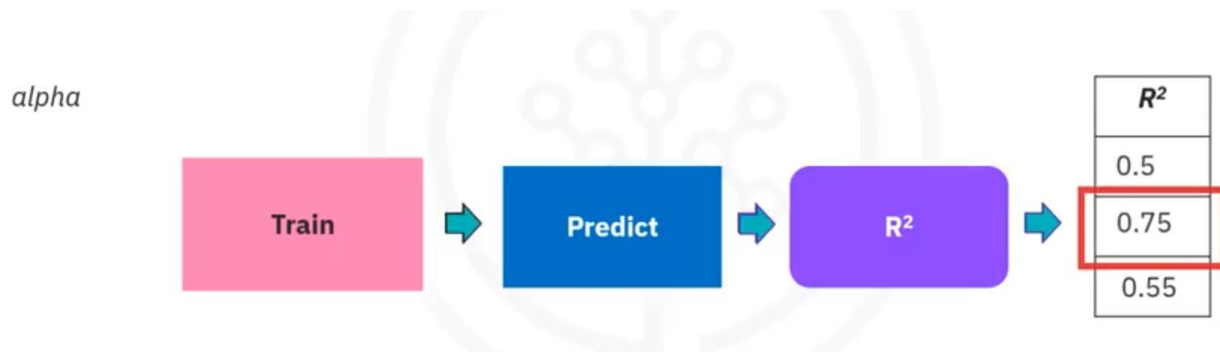
We use a **second set called validation data.**



This is similar to test data, but it is used to select parameters like Alpha.

We start with a small value of Alpha. We train the model, make a prediction using the validation data, then calculate the R^2 and store the values.

Repeat the value for a larger value of Alpha. We train the model again, make a prediction using the validation data, then calculate the R^2 and store the values of R^2 . We repeat the process for a different Alpha value, training the model and making a prediction.



We select the value of Alpha that maximizes the R^2 . Note that we can use other metrics to select the value of Alpha, like mean squared error.

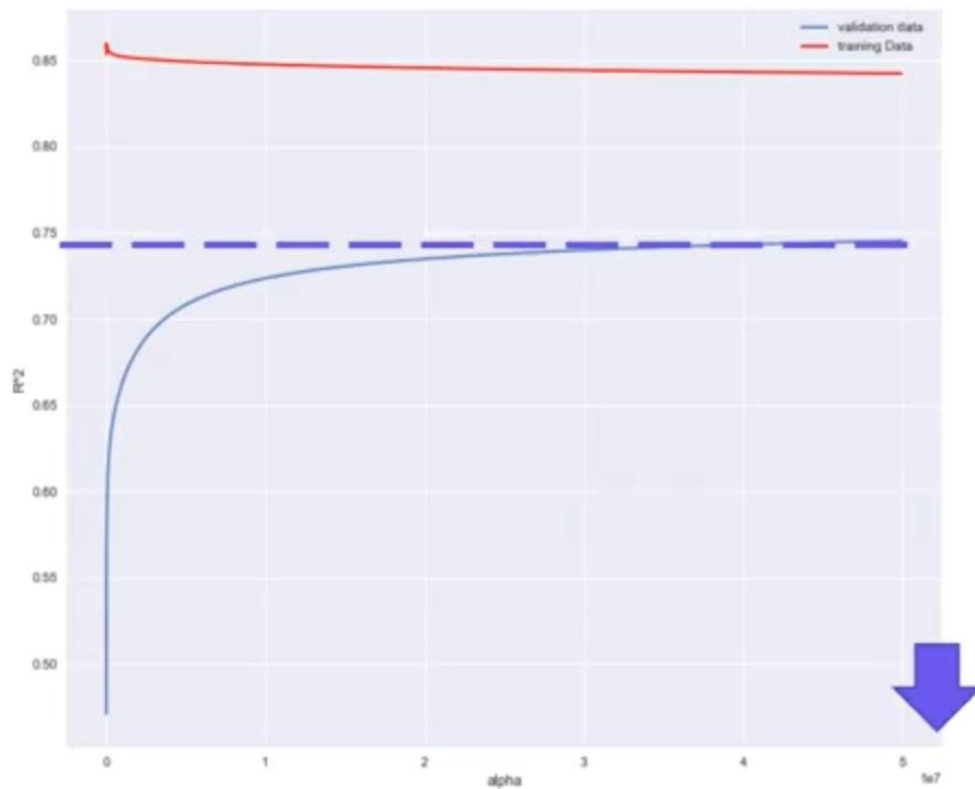
The overfitting problem is even worse if we have lots of features.

The following plot shows the different values of R^2 on the vertical axis. The horizontal axis represents different values for Alpha.

We use several features from our used car data set and a second order polynomial function.

The training data is in red and validation data is in blue.

We see as the value of **Alpha increases**, the value of **R^2 increases** on the validation data and converges at approximately 0.75. In this case, we select the maximum value of Alpha because running the experiment for higher values of Alpha have little impact.



Conversely, as **Alpha increases**, the **R²** decreases on the test data.

This is because the term Alpha prevents overfitting. This may improve the results in the unseen data, but the model has worse performance on the test data.

See the lab to generate this plot

Grid Search

Grid Search allows us to **scan through multiple free parameters with few lines of code**.

Parameters like the alpha term discussed in the previous video are not part of the fitting or training process, these values are called **hyperparameters**.

Hyperparameters

- Alpha in Ridge regression is called a hyperparameter
- Scikit-learn has a means of automatically iterating over these hyperparameters using cross-validation called Grid Search



Scikit-learn has a means of automatically iterating over these hyperparameters using cross-validation. This method is called Grid Search.

Grid Search takes the model or objects you would like to train and different values of the hyperparameters.

It then calculates the mean square error or r squared for various hyperparameter values, allowing you to choose the best values.

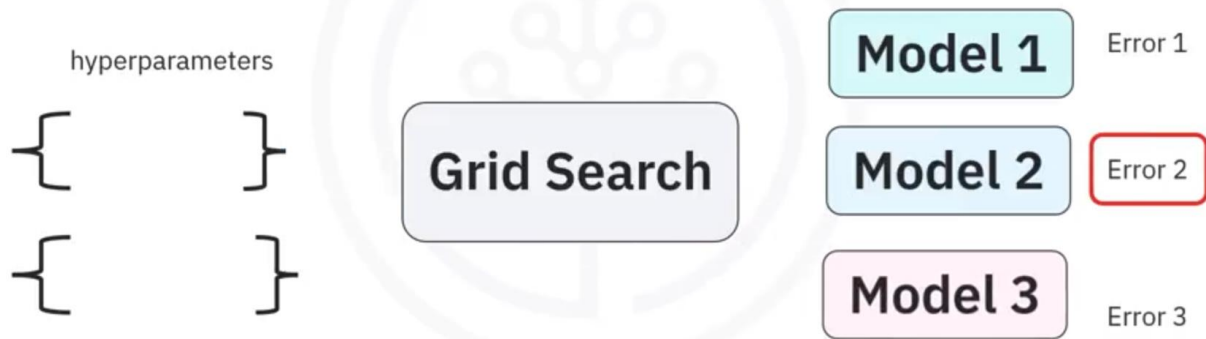
Let the small circles represent different hyperparameters. We start off with one value for hyperparameters and train the model. We use different hyperparameters to train the model.

We continue the process until we have exhausted the different free parameter values.

Each model produces an error, we **select the hyperparameter that minimizes the error.**

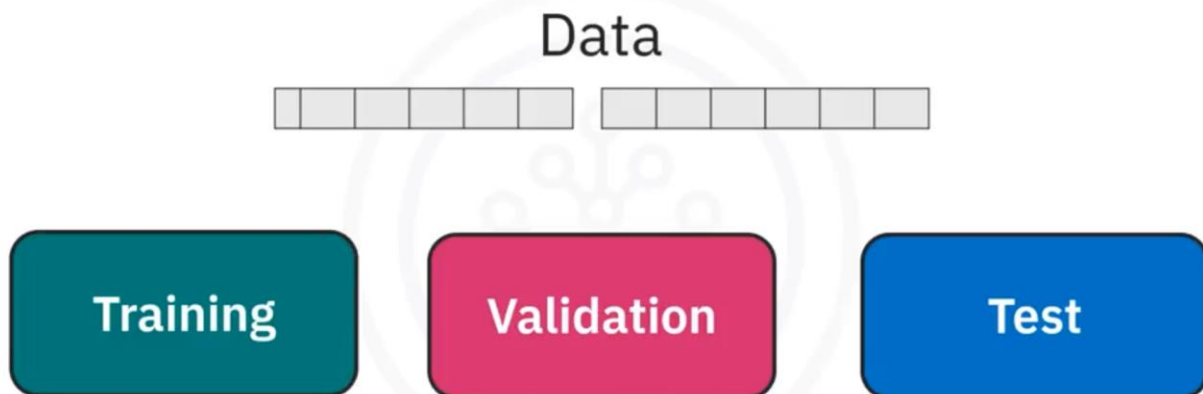
Hyperparameters

- Alpha in Ridge regression is called a hyperparameter
- Scikit-learn has a means of automatically iterating over these hyperparameters using cross-validation called Grid Search



To select the hyperparameter, we split our data set into three parts, the **training** set, **validation** set and **test** set.

Grid Search



We train the model for different hyperparameters, we use the r squared or mean squared error for each model.

We select the hyperparameter that **minimizes the mean squared error or maximizes the r squared on the validation set**.

We finally test our model performance using the test data.

This is the [Scikit-learn webpage](#) where the object constructor parameters are given. It should be noted that the attributes of an object are also called parameters. We will not make the distinction even though some of the options are not hyperparameters per se.

In this module, we will focus on the hyperparameter alpha and the normalization parameter.

Home Installation Documentation ~ Examples

Google Custom Search Search

sklearn.linear_model.Ridge

```
class sklearn.linear_model.Ridge(alpha=1.0, fit_intercept=True, normalize=False, copy_X=True, max_iter=None, tol=0.001, solver='auto', random_state=None) [source]
```

Linear least squares with l2 regularization.

This model solves a regression model where the loss function is the linear least squares function and regularization is given by the l2-norm. Also known as Ridge Regression or Tikhonov regularization. This estimator has built-in support for multi-variate regression (i.e., when y is a 2d-array of shape [n_samples, n_targets]).

Read more in the User Guide.

Parameters: **alpha** : (float, array-like), shape (n_targets)

Regularization strength; must be a positive float. Regularization improves the conditioning of the problem and reduces the variance of the estimates. Larger values specify stronger regularization. Alpha corresponds to λ in other linear models such as LogisticRegression or LinearSVC. If an array is passed, penalties are assumed to be specific to the targets. Hence they must correspond in number.

fit_intercept : boolean

Whether to calculate the intercept for this model. If set to false, no intercept will be used in calculations (e.g. data is expected to be already centered).

normalize : boolean, optional, default False

This parameter is ignored when **fit_intercept** is set to False. If True, the regressors X will be normalized before regression by subtracting the mean and dividing by the l2-norm. If you wish to standardize, please use `sklearn.preprocessing.StandardScaler` before calling **fit** on an estimator with **normalize=False**.

copy_X : boolean, optional, default True

If True, X will be copied; else, it may be overwritten.

max_iter : int, optional

The value of your grid search is a Python **list** that contains a Python **dictionary**, the **key** is the name of the **free parameter**. The **value** of the dictionary is the **different values of the free parameter**.

This can be viewed as a table with various free parameter values, we also have the object or model.

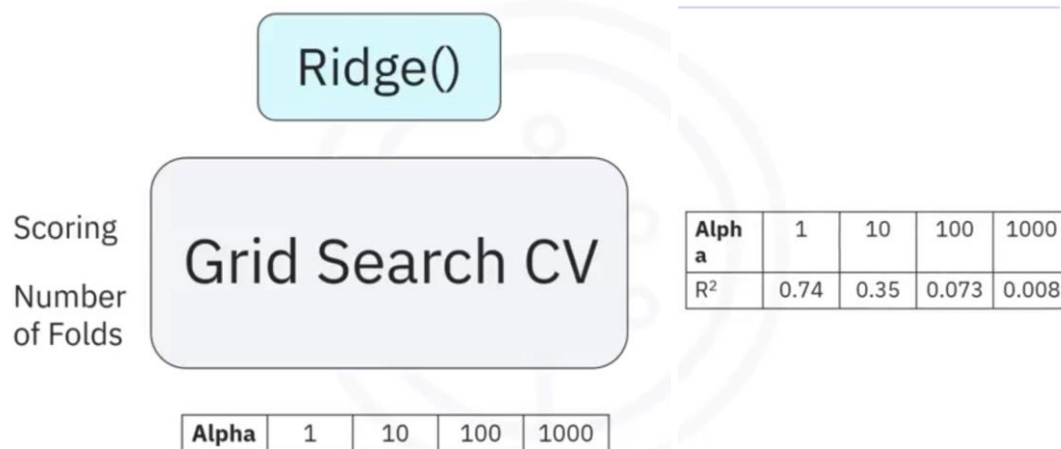
```
parameters = [ {'alpha' : [1, 10, 100, 1000]} ]
```

Alpha	1	10	100	1000
-------	---	----	-----	------

Ridge()

The Grid Search takes on **the scoring method**, in this case r squared, the number of **folds**, the **model** or object, and the free **parameter values**.

Grid Search



Some of the outputs include the different scores for different free parameter values. In this case, **the r squared along with the free parameter values that have the best score**.

First, we import the libraries we need, including Grid Search CV, the dictionary of parameter values. We create a ridge regression object or model, we then create a GridSearchCV object. The inputs are the ridge regression object, the parameter values and the number of folds. We will use r squared, this is the default scoring method. We fit the object, we can find the best values for the free parameters using the attribute best estimator. We can also get information like the mean score on the validation data using the attribute CV result.

```

from sklearn.linear_model import Ridge
from sklearn.model_selection import GridSearchCV

parameters1= [{'alpha': [0.001,0.1,1, 10, 100, 1000,10000,100000,100000]}]

RR=Ridge()

Grid1 = GridSearchCV(RR, parameters1,cv=4)

Grid1.fit(x_data[['horsepower', 'curb-weight', 'engine-size',
'highway-mpg']],y_data)

Grid1.best_estimator_

scores = Grid1.cv_results_
scores['mean_test_score']

```

One of the **advantages of Grid Search** is how quickly we can test multiple parameters.

For example, ridge regression has the option to normalize the data, to see how to standardize, see Module 4.

The term alpha is the first element in the dictionary; the second element is the normalize option. The key is the name of the parameter, the value is the different options in this case, because we can either normalize the data or not.

The values are true or false, respectively. The dictionary is a table or grid that contains two different values.

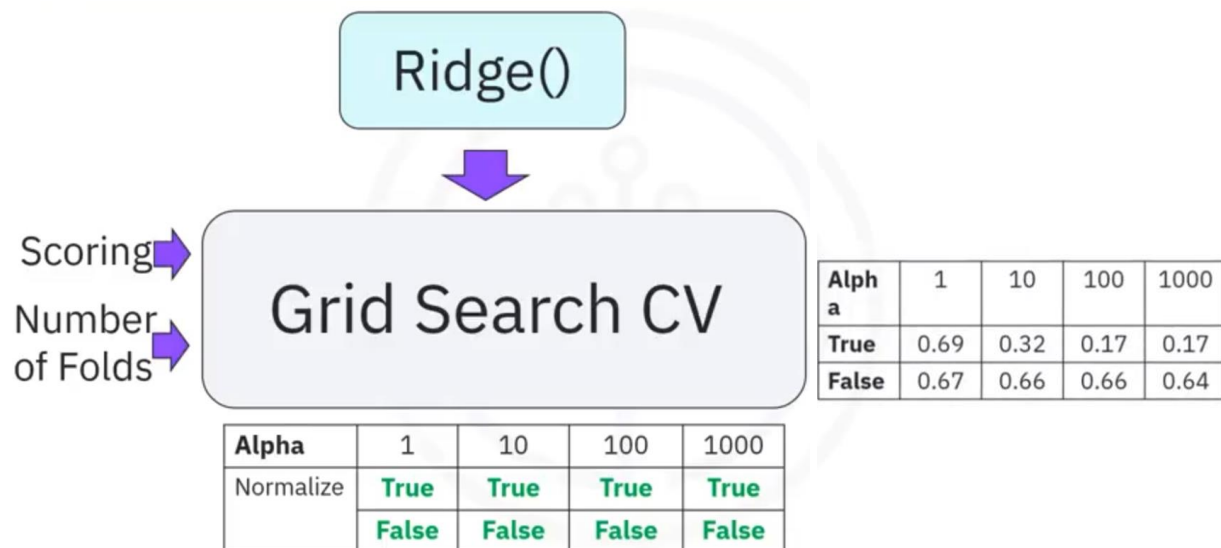
```
parameters = [ {'alpha' : [1, 10, 100, 1000] , 'normalize' : [True, False] } ]
```

Alpha	1	10	100	1000
Normalize	True	True	True	True
	False	False	False	False

Ridge()

As before, we need the ridge regression object or model. The procedure is similar except that we have a table or grid of different parameter values.

The output is the score for all the different combinations of parameter values, the code is also similar.



The dictionary contains the different free parameter values, we can find the best value for the free parameters. The resulting scores of the different free parameters are stored in this dictionary, `Grid1.cv_results_`.

```
from sklearn.linear_model import Ridge
from sklearn.model_selection import GridSearchCV

parameters1= [{'alpha': [0.001,0.1,1, 10, 100, 1000,10000,100000,100000]}]

RR=Ridge()

Grid1 = GridSearchCV(RR, parameters1,cv=4)

Grid1.fit(x_data[['horsepower', 'curb-weight', 'engine-size', 'highway-mpg']],y_data)

Grid1.best_estimator_

scores = Grid1.cv_results_
```

We can print out the score for the different free parameter values. The parameter values are stored as shown here, see the course labs for more examples.


```
for param, mean_val, mean_test in zip(scores['params'], scores['mean_test_score'],
scores['mean_train_score']):
```

```
print(param, "R^2 on test data:", mean_val, "R^2 on train data:", mean_test)
```

```
{'alpha': 0.001, 'normalize': True} R^2 on tesst data: 0.66605547293 R^2 on train data: 0.814001968709
{'alpha': 0.001, 'normalize': False} R^2 on tesst data: 0.665488366584 R^2 on train data: 0.814002698797
{'alpha': 0.1, 'normalize': True} R^2 on tesst data: 0.694175625356 R^2 on train data: 0.810546768311
{'alpha': 0.1, 'normalize': False} R^2 on tesst data: 0.665488937796 R^2 on train data: 0.814002698794
{'alpha': 1, 'normalize': True} R^2 on tesst data: 0.690486934584 R^2 on train data: 0.749104440368
{'alpha': 1, 'normalize': False} R^2 on tesst data: 0.665494127178 R^2 on train data: 0.814002698472
{'alpha': 10, 'normalize': True} R^2 on tesst data: 0.321376875232 R^2 on train data: 0.341856042902
{'alpha': 10, 'normalize': False} R^2 on tesst data: 0.665545680812 R^2 on train data: 0.8140026666
{'alpha': 100, 'normalize': True} R^2 on tesst data: 0.0170551710263 R^2 on train data: 0.0496044796826
{'alpha': 100, 'normalize': False} R^2 on tesst data: 0.666029359996 R^2 on train data: 0.813999791851
{'alpha': 1000, 'normalize': True} R^2 on tesst data: -0.0301961745066 R^2 on train data: 0.005184451599
{'alpha': 1000, 'normalize': False} R^2 on tesst data: 0.668968215369 R^2 on train data: 0.813870488264
{'alpha': 10000, 'normalize': True} R^2 on tesst data: -0.0351687400461 R^2 on train data: 0.000520784757979
{'alpha': 10000, 'normalize': False} R^2 on tesst data: 0.673346359342 R^2 on train data: 0.812583743226
{'alpha': 100000, 'normalize': True} R^2 on tesst data: -0.0356685844558 R^2 on train data: 5.2101975528e-05
{'alpha': 100000, 'normalize': False} R^2 on tesst data: 0.657818838432 R^2 on train data: 0.789541446486
{'alpha': 100000, 'normalize': True} R^2 on tesst data: -0.0356685844558 R^2 on train data: 5.2101975528e-05
{'alpha': 100000, 'normalize': False} R^2 on tesst data: 0.657818838432 R^2 on train data: 0.789541446486
```

Lesson Summary

Congratulations! You have completed this lesson. At this point in the course, you know:

- How to **split your data** using the `train_test_split()` method into training and test sets. You use the training set to train a model, discover possible predictive relationships, and then use the test set to test your model to evaluate its performance.
- How to use the generalization error to measure how well your data does at predicting previously unseen data.
- How to use **cross-validation by splitting the data into folds** where you use some of the folds as a training set, which we use to train the model, and the remaining parts are used as a test set, which we use to test the model. You iterate through the folds until you use each partition for training and testing. At the end, you average results as the **estimate of out-of-sample error**.
- How to pick the best polynomial order and problems that arise when selecting the wrong order polynomial by analyzing models that underfit and overfit your data.
- Select the best order of a polynomial to fit your data by minimizing the test error using **a graph comparing the mean square error to the order of the fitted polynomials**.
- You should use ridge regression when there is a **strong relationship among the independent variables**.
- That ridge regression prevents overfitting.
- **Ridge regression controls the magnitude of polynomial coefficients by introducing a hyperparameter, alpha.**

- To determine alpha, you divide your data into training and validation data. Starting with a small value for alpha, you train the model, make a prediction using the validation data, then calculate the R-squared and store the values. You repeat the value for a larger value of alpha. You repeat the process for different alpha values, training the model, and making a prediction. You select the value of alpha that maximizes R-squared.
- That grid search allows you to scan through multiple hyperparameters using the Scikit-learn library, which iterates over these parameters using cross-validation. Based on the results of the grid search method, you select optimum hyperparameter values.
- The GridSearchCV() method takes in a dictionary as its argument where the key is the name of the hyperparameter, and the values are the hyperparameter values you wish to iterate over.

Lab_model_evaluation_and_refinement_used_cars_pricing

Filtramos las variables numéricas de un dataset con `df._get_numeric_data()` o con `df.select_dtypes(include=[np.number])`

We can use negative squared error as a score by setting the parameter 'scoring' metric to 'neg_mean_squared_error'.

```
MSEcross = -1 * cross_val_score(lre,x_data[['horsepower']], y_data,cv=4,scoring='neg_mean_squared_error')
```

```
Rcross =cross_val_score(lre, x_data[['horsepower']], y_data, cv=4)
```

En Jupyter, hay un widget interact que me ha gustado mucho

Hya una barra de progreso en la parte 3.

Lab_model_evaluation_and_refinement_laptop_pricing

Module 6: Practice Project Overview

Module 1: Importing Data Sets

Understanding the data

Python packages for data Science

Importing and exporting data with Python

Getting Started Analyzing data in Python

Accessing Data Databases with Python

Question 1

Display the data types of each column using the function `dtypes`. Take a screenshot of your code and output. You will need to submit the screenshot for the final project.

```
[12]: #Enter Your Code, Execute and take the Screenshot  
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 21613 entries, 0 to 21612  
Data columns (total 22 columns):  
#   Column                Non-Null Count  Dtype    
---  ---                  
0   Unnamed: 0             21613 non-null  int64    
1   id                     21613 non-null  int64    
2   date                   21613 non-null  object   
3   price                  21613 non-null  float64  
4   bedrooms               21600 non-null  float64  
5   bathrooms              21603 non-null  float64  
6   sqft_living            21613 non-null  int64    
7   sqft_lot               21613 non-null  int64    
8   floors                 21613 non-null  float64  
9   waterfront             21613 non-null  int64    
10  view                   21613 non-null  int64    
11  condition              21613 non-null  int64    
12  grade                  21613 non-null  int64    
13  sqft_above             21613 non-null  int64    
14  sqft_basement          21613 non-null  int64    
15  yr_built               21613 non-null  int64    
16  yr_renovated           21613 non-null  int64    
17  zipcode                21613 non-null  int64    
18  lat                    21613 non-null  float64  
19  long                   21613 non-null  float64  
20  sqft_living15          21613 non-null  int64    
21  sqft_lot15             21613 non-null  int64    
dtypes: float64(6), int64(15), object(1)  
memory usage: 3.5+ MB
```

Question 2

Drop the columns "id" and "Unnamed: 0" from axis 1 using the method `.drop()`, then use the method `.describe()` to obtain a statistical summary of the data. Make sure the `inplace` parameter is set to `True`. Take a screenshot of your code and output. You will need to submit the screenshot for the final project.

```
[21]: #Enter Your Code, Execute and take the Screenshot
df.drop(['id', 'Unnamed: 0'], axis = 1)

[21]:
```

	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	condition	grade	sqft_above	sqft_basement	yr_built	yr_renovated	zipcode	lat	long	sqft_living15
0	20141013T000000	221900.0	3.0	1.00	1180	5650	1.0	0	0	3	7	1180	0	1955	0	98178	47.5112	-122.257	134000
1	20141209T000000	538000.0	3.0	2.25	2570	7242	2.0	0	0	3	7	2170	400	1951	1991	98125	47.7210	-122.319	169000
2	20150225T000000	180000.0	2.0	1.00	770	10000	1.0	0	0	3	6	770	0	1933	0	98028	47.7379	-122.233	272000
3	20141209T000000	604000.0	4.0	3.00	1960	5000	1.0	0	0	5	7	1050	910	1965	0	98136	47.5208	-122.393	136000
4	20150218T000000	510000.0	3.0	2.00	1680	8080	1.0	0	0	3	8	1680	0	1987	0	98074	47.6168	-122.045	180000
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
21608	20140521T000000	360000.0	3.0	2.50	1530	1131	3.0	0	0	3	8	1530	0	2009	0	98103	47.6993	-122.346	153000
21609	20150223T000000	400000.0	4.0	2.50	2310	5813	2.0	0	0	3	8	2310	0	2014	0	98146	47.5107	-122.362	183000
21610	20140623T000000	402101.0	2.0	0.75	1020	1350	2.0	0	0	3	7	1020	0	2009	0	98144	47.5944	-122.299	102000
21611	20150116T000000	400000.0	3.0	2.50	1600	2388	2.0	0	0	3	8	1600	0	2004	0	98027	47.5345	-122.069	141000
21612	20141015T000000	325000.0	2.0	0.75	1020	1076	2.0	0	0	3	7	1020	0	2008	0	98144	47.5941	-122.299	102000

21613 rows x 20 columns

Question 3

Use the method `.value_counts()` to count the number of houses with unique floor values, use the method `.to_frame()` to convert it to a data frame. Take a screenshot of your code and output. You will need to submit the screenshot for the final project.

```
[22]: #Enter Your Code, Execute and take the Screenshot
df['floors'].value_counts().to_frame()

[22]:
```

	count
1.0	10680
2.0	8241
1.5	1910
3.0	613
2.5	161
3.5	8

Question 4

Use the function `boxplot` in the seaborn library to determine whether houses with a waterfront view or without a waterfront view have more price outliers. Take a screenshot of your code and boxplot. You will need to submit the screenshot for the final project.

```
[23]: sns.boxplot(x = 'waterfront', y = 'price', data = df)

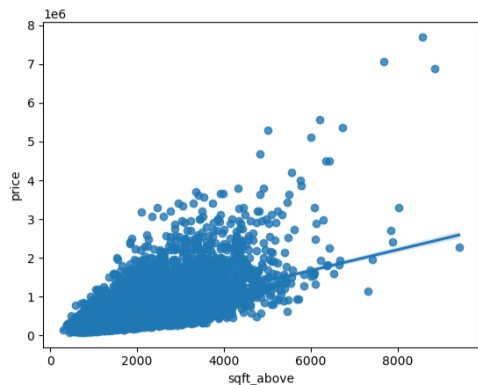
[23]: <AxesSubplot: xlabel='waterfront', ylabel='price'>
```

Question 5

Use the function `regplot` in the seaborn library to determine if the feature `sqft_above` is negatively or positively correlated with price. Take a screenshot of your code and scatterplot. You will need to submit the screenshot for the final project.

```
[24]: #Enter Your Code, Execute and take the Screenshot
sns.regplot(x = 'sqft_above', y = 'price', data = df)
```

```
[24]: <AxesSubplot:xlabel='sqft_above', ylabel='price'>
```



Question 6

Fit a linear regression model to predict the 'price' using the feature 'sqft_living' then calculate the R^2 . Take a screenshot of your code and the value of the R^2 . You will need to submit it for the final project.

```
[27]: #Enter Your Code, Execute and take the Screenshot
X = df[['sqft_living']]
y = df['price']
lre = LinearRegression()
lre.fit(X, y)
print("r2=", lre.score(X, y))
```

```
r2= 0.4928532179037931
```

Question 7

Fit a linear regression model to predict the 'price' using the list of features:

```
[28]: features = ["floors", "waterfront", "lat", "bedrooms", "sqft_basement", "view", "bathrooms", "sqft_living15", "sqft_above", "grade", "sqft_living"]
```

Then calculate the R^2 . Take a screenshot of your code and the value of the R^2 . You will need to submit it for the final project.

```
[30]: #Enter Your Code, Execute and take the Screenshot
X = df[features]
y = df['price']
lre = LinearRegression()
lre.fit(X, y)
print("r2=", lre.score(X, y))
```

```
r2= 0.6576890354915759
```

Question 8

Use the list to create a pipeline object to predict the 'price', fit the object using the features in the list `features`, and calculate the R^2 . Take a screenshot of your code and the value of the R^2 . You will need to submit it for the final project.

```
[33]: #Enter Your Code, Execute and take the Screenshot
my_pipe = Pipeline(Input)
my_pipe.fit(X, y)
print("r2=", my_pipe.score(X, y))
```

```
r2= 0.7512051345272872
```

▼ Question 9

Create and fit a Ridge regression object using the training data, set the regularization parameter to 0.1, and calculate the R^2 using the test data. Take a screenshot of your code and the value of the R^2 . You will need to submit it for the final project.

```
[37]: from sklearn.linear_model import Ridge
```

```
[38]: #Enter Your Code, Execute and take the Screenshot
```

```
RR = Ridge(alpha=0.1)
RR.fit(x_train, y_train)
print("r2=", RR.score(x_test, y_test))
```

```
r2= 0.647875916393907
```

▼ Question 10

Perform a second order polynomial transform on both the training data and testing data. Create and fit a Ridge regression object using the training data, set the regularisation parameter to 0.1, and calculate the R^2 utilising the test data provided. Take a screenshot of your code and the R^2 . You will need to submit it for the final project.

```
[42]: #Enter Your Code, Execute and take the Screenshot
```

```
pt_o2 = PolynomialFeatures(degree = 2)
x_train_pt_o2 = pt_o2.fit_transform(x_train)
x_test_pt_o2 = pt_o2.fit_transform(x_test)
RR_pt_o2 = Ridge(alpha = 0.1)
RR_pt_o2.fit(x_train_pt_o2, y_train)
print("r2=", RR_pt_o2.score(x_test_pt_o2, y_test))
```

```
r2= 0.7002744263583341
```